



Addis Ababa Science and Technology University

College of Engineering

Department of Software Engineering

HackET: A Centralized, Multilingual Platform for End-to-End Hackathon

Management and Discovery in Ethiopia

Group Members:

No.	Name	ID
1.	Ayub Sufian	ETS0238/14
2.	Eyobel Mebratie	ETS0597/14
3.	Hermela Getachew	ETS0792/14
4.	Natnael Shiferahw	ETS1262/14

Advisor Name: Aster Alemu

Signature_____

June, 2026

Acknowledgement

The successful completion of the HackET project has been made possible through the generous support, guidance, and encouragement of many individuals and institutions, to whom we owe our sincere gratitude.

First and foremost, we extend our deepest appreciation to our advisor, Ms. Aster Alemu, whose expert guidance, constructive feedback, and unwavering encouragement throughout the entire project lifecycle shaped both the quality and direction of this work. Their commitment to our academic and professional growth has been invaluable.

We are grateful to the faculty and staff of the Department of Software Engineering at Addis Ababa Science and Technology University (AASTU) for providing an intellectually stimulating environment and the academic foundation necessary to undertake a project of this scope and ambition.

Our sincere thanks go to the hackathon organizers, university tech club leads, and innovation hubs who participated in our on-the-ground research and stakeholder interviews. Their first-hand insights into the daily administrative overhead, information silos, and fragmented coordination workflows of competitive events in Ethiopia gave this platform its practical grounding and relevance.

We also acknowledge the open-source developer communities behind Node.js and PostgreSQL which enabled us to build a performance-optimized, multi-tenant ecosystem within an academic timeframe.

Finally, we are deeply grateful to our families and fellow students for their moral support, patience, and encouragement during the demanding phases of this project. This work is dedicated to everyone who believes that technology, thoughtfully applied, can bridge regional accessibility gaps and empower the next generation of Ethiopian innovators.

Table of contents

Lists of Tables, Figures/Illustrations, Plates/Photographs	5
List of Abbreviations, Symbols, Specialized Nomenclature	6
Abstract.....	8
Chapter One: Introduction	10
1.1. Statement of the Problem.....	11
1.2. Objectives	12
1.2.1. General Objective	12
1.2.2. Specific Objectives	12
1.3. Scope and Limitation	13
1.4. Methodology	15
1.5. Plan of Activities.....	17
1.6. Budget Required	20
1.7. Significance of the Study	21
1.8 Outline of the study.....	23
Chapter Two: Literature Review.....	25
2.1. Study Related Works	25
2.2. Identifying Milestones of the Related Literatures and Finding the Gaps	26
2.3. Lessons Learned from Literatures.....	28
Chapter Three: Problem Analysis and Modeling.....	30
3.1. Existing System and Its Problems.....	30
3.2. Specifying the Requirements of the Proposed Solution.....	32
3.3. System Modeling	33
3.3.1. Functional Requirements	33
3.3.2. Non-Functional Requirements	39
3.3.3. Usecase	42
3.3.4. Dynamic Models of a System	95

3.3.4.1. Sequence Diagrams.....	96
3.3.4.2. State Machine Diagrams	98
3.3.4.3. Activity Diagrams	102
3.3.4.4. Collaboration Diagrams	105
3.4. Model Validation	106
Chapter Four: System Design	109
4.1. Overview	109
4.2. Specifying the Design Goals.....	110
4.3. System Design	111
4.3.1. Proposed Software Architecture	112
4.3.2. Subsystem Decomposition.....	114
4.3.3. Database Design.....	117
4.3.4. Deployment Diagram.....	138
4.3.5. User Interface Design.....	139
4.3.6. System Integration	149
4.3.7. Security design.....	150
4.4. Verifying the Requirements in the Design.....	153
Chapter Five: System Implementation.....	156
5.1. Reviewing the Design Solution.....	156
5.2. Deciding on the Development Tools	157
5.3. Developing the Solution	160
Chapter Six: System Evaluation	168
6.1. Preparing Sample Test Plans.....	168
6.2. Evaluating the Proposed Design and Solutions	168
6.3. Discussing the Results	173
Chapter Seven: Conclusions and Recommendations	175
7.1. Conclusion of the Study	175

7.2. Recommendations of the Study	176
References.....	178
Appendices.....	179

Lists of Tables, Figures/Illustrations, Plates/Photographs

Figure 1.1. HackET Project Timeline and Gantt Chart.....	18
--	----

List of Abbreviations, Symbols, Specialized Nomenclature

Abbreviation	Full Form
AASTU	Addis Ababa Science and Technology University
ACID	Atomicity, Consistency, Isolation, Durability
ADMIN	Administrator
AI	Artificial Intelligence
API	Application Programming Interface
AWS	Amazon Web Services
CDN	Content Delivery Network
CPU	Central Processing Unit
CRUD	Create, Read, Update, Delete
CS	Computer Science
CSV	Comma-Separated Values
CSS	Cascading Style Sheets
EAT	East Africa Time
ERD	Entity Relationship Diagram
FK	Foreign Key
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
ICS	iCalendar (File Format)
ICT	Information and Communication Technology
IP	Internet Protocol
JSON	JavaScript Object Notation
JSONB	JavaScript Object Notation Binary
JWT	JSON Web Token
MVC	Model-View-Controller
NFR	Non-Functional Requirement
OAuth	Open Authorization
ORM	Object-Relational Mapping

OTP	One-Time Password
PDF	Portable Document Format
PKCE	Proof Key for Code Exchange
RAM	Random Access Memory
RBAC	Role-Based Access Control
REST	Representational State Transfer
S3	Simple Storage Service
SDD	System Design Document
SHA-256	Secure Hash Algorithm 256-bit
SMS	Short Message Service
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
SRS	Software Requirements Specification
SSL	Secure Sockets Layer
TC	Test Case
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TTL	Time to Live
UC	Use Case
UCD	User-Centered Design
UI	User Interface
UNQ	Unique Constraint
USD	United States Dollar
UUID	Universally Unique Identifier
UX	User Experience
XLSX	Microsoft Excel Open XML Spreadsheet

Abstract

Ethiopia's rapidly growing digital economy, driven by over 200,000 ICT and engineering students, has seen hackathons emerge as primary catalysts for technology-driven innovation. However, the supporting digital infrastructure remains highly fragmented and decentralized, with over 80% of coordination relying on non-specialized tools like disparate Telegram channels, basic Google Forms, and manual spreadsheets. This reliance creates severe discovery barriers for participants and unsustainable administrative loads for organizers. Existing digital solutions are narrow in scope, causing "Institutional Amnesia" where prototypes and data are lost immediately post-event, while a lack of localized bilingual (Amharic-English) interfaces limits digital inclusion outside the capital.

This project presents HackET, a comprehensive, centralized, multi-tenant web-based ecosystem developed to bridge this critical gap. The platform comprises dedicated, role-based interfaces tailored for different stakeholders: distinct dashboards for Participants, Organizers, and Judges, supplemented by coordination interfaces for mentors. The frontend and backend are built using a unified Node.js and React stack with a PostgreSQL database structure, designed for seamless deployment across scalable production cloud platforms like Vercel and Render.

HackET enables participants to access a centralized discovery hub with advanced filtering by region or theme, utilize a rule-based content-based filtering recommendation engine for personalized event suggestions, and engage an intelligent skill-based matching engine for team formation. To lower cognitive barriers and expand digital inclusion, all static system interfaces provide comprehensive bilingual support in both English and Amharic, optimized for East Africa Time (EAT) and the Ethiopian calendar format. Organizers and judges benefit from an automated judging module featuring multi-criteria scoring and mathematical normalization rules like outlier filtering with a transparent leaderboard, alongside real-time tiered notifications, automated digital certificate issuance, and data-driven analytics tools with exportable PDF and CSV reports.

The system was developed using a structured methodology combining User-Centered Design (UCD) with a sequential Waterfall process. Requirements were systematically gathered through localized field research, including semi-structured interviews with 6–10 hackathon organizers from university tech clubs and hubs, supplemented by comprehensive surveys of 100–120 potential participants. Technical validation involved a tiered quality assurance strategy followed by a usability study with 10–15 local developers and organizers to refine the bilingual user experience. The project demonstrates that centralized multi-tenancy, persistent portfolio archiving, and localized accessibility can be successfully combined into a context-aware infrastructure to meaningfully unify the innovation pipeline and sustainably foster technical talent in developing tech ecosystems.

Keywords: Hackathon Management, Multi-Tenant Architecture, Node.js, Automated Judging, Content-Based Filtering, Bilingual Interface, Ethiopia, Developer Portfolio, Innovation Ecosystem.

Chapter One: Introduction

In recent years, Ethiopia has witnessed a significant surge in its digital economy, driven by a growing population of over 200,000 ICT and engineering students and a national push toward digital transformation. Hackathons—intensive, collaborative events where participants develop technology-driven solutions within a limited timeframe—have emerged as a primary catalyst for this innovation. Globally, these events are recognized as essential engines for creativity, with over 50,000 organized annually [1]. In Ethiopia, the adoption of hackathons by universities and innovation hubs has engaged thousands of young professionals in addressing local challenges in agriculture, health, and finance.

However, the supporting digital infrastructure for these events has not developed at the same pace as the growing interest. While the "innovation culture" is high, the "organizational infrastructure" remains fragmented and largely decentralized. Currently, over 80% of hackathon-related coordination in Ethiopia relies on non-specialized, general-purpose tools such as disparate Telegram channels, basic Google Forms, and manual spreadsheets [2]. This reliance on "Information Silos" creates a severe discovery barrier for participants and an unsustainable administrative load for organizers.

Existing digital solutions remain narrow in scope; most institutional efforts focus only on basic registration web forms but fail to provide a comprehensive, end-to-end ecosystem covering team formation, mentor coordination, automated judging, and project archiving [3]. This leads to "Institutional Amnesia," where valuable prototypes and participant data are lost immediately after an event concludes, preventing long-term community growth and benchmarking [4]. Furthermore, the lack of localized, bilingual (Amharic-English) interfaces remains a critical barrier to digital inclusion for developers outside the capital [6].

Recognizing this gap, the HackET platform is envisioned as a centralized, multi-tenant web-based system designed to unify the Ethiopian innovation ecosystem. By leveraging modern technologies, HackET aims to connect participants with opportunities through persistent developer profiles while offering organizers streamlined tools for automated scoring, real-time feedback, and data-driven analytics. Through this approach, HackET seeks to make hackathon

participation and management more transparent, accessible, and impactful for the next generation of Ethiopian innovators.

1.1. Statement of the Problem

The technology ecosystem in Ethiopia is experiencing a rapid surge, with thousands of ICT and Engineering students graduating annually and a growing number of innovation hubs emerging in cities like Addis Ababa. Hackathons have become the primary catalyst for this talent, serving as high-pressure environments for generating practical solutions to local problems. However, despite their importance, the hackathon landscape in Ethiopia is critically underserved by digital infrastructure, remaining fragmented and decentralized.

Currently, the organization of these events relies on a mix of informal networks and isolated, event-specific websites. While high-profile events may occasionally deploy dedicated landing pages—these platforms function as "Information Silos." They are typically built for a single event cycle, meaning that once the competition ends, the data, participant registries, and project archives become inaccessible or stagnant [4]. This forces the Ethiopian tech community into a cycle of "re-inventing the wheel," where every new organizer must develop or pay for a new, standalone web presence, leading to significant resource redundancy and technical debt.

For the majority of smaller or mid-sized hackathons, coordination remains even more primitive. Over 80% of hackathon-related activity—ranging from announcements to team formation—is conducted via disparate Telegram channels and basic Google Forms [2]. For participants, this creates a "discovery barrier," where high-potential developers miss opportunities because there is no centralized "Source of Truth" to find every upcoming event in the country. For organizers, the lack of a specialized, multi-tenant platform results in massive administrative overhead, as they must manually consolidate registration data, track project submissions, and manage judging scores across disconnected tools [3].

From a strategic perspective, the problem is compounded by "Institutional Amnesia." Because there is no unified, persistent repository, the intellectual property and participant growth tracked during events are not carried over to future opportunities [4]. This lack of a project archive prevents long-term benchmarking and hinders the growth of the innovation community, as new

participants cannot learn from past winners and organizers cannot analyze historical trends to improve event quality.

Furthermore, existing solutions—whether generic social media or one-off event sites—lack integrated bilingual support (Amharic-English), creating a barrier to digital inclusion for developers who prefer localized interfaces [6].

Without a structured, centralized digital system, Ethiopia’s hackathon ecosystem remains a collection of isolated "flash-in-the-pan" events rather than a cohesive innovation pipeline. Therefore, there is an urgent need for HackET, a centralized, multi-tenant web-based platform. Unlike standalone event sites, HackET will provide a permanent ecosystem that consolidates the entire hackathon lifecycle—discovery, persistent developer profiles, automated judging, and project archiving—into a single, efficient, and data-driven infrastructure for all Ethiopian innovators.

1.2. Objectives

1.2.1. General Objective

The general objective of this project is to design and develop a centralized, multi-tenant web platform (HackET) to unify Ethiopia’s fragmented innovation ecosystem. The platform aims to streamline the end-to-end hackathon lifecycle—from unified event discovery and registration to automated judging and secure project archiving—while fostering long-term community growth through persistent developer profiles and data-driven analytics for organizers.

1.2.2. Specific Objectives

The specific objectives of this project are to:

- Develop a Centralized Discovery Hub that enables participants to search, filter, and track upcoming hackathons across Ethiopia through a unified "Source of Truth".
- Implement a Multi-Tenant Management Dashboard for organizers to create and configure events, manage registrations, and coordinate with technical logistics.

- Integrate a Persistent Developer Portfolio System that archives participation history and project submissions, allowing users to build a verifiable digital resume over time.
- Develop an Automated Judging Module supporting multi-criteria scoring, normalization of judge inputs, and real-time result aggregation to eliminate manual entry errors.
- Implement an Intelligent Matching Engine and collaborative discussion modules to facilitate team formation and mentor-led interaction based on skill-based user data.
- Design a Bilingual (Amharic-English) Interface optimized for local accessibility and digital inclusion, supporting both static content and core system interactions.
- Provide Organizer Analytics Tools to monitor event metrics, participant engagement trends, and submission quality to improve future innovation cycles.
- Ensure High System Performance and Localization by designing a responsive architecture that completes actions within 3 seconds while supporting Ethiopian regional time and calendar settings.
- Develop an Automated Communication and Recognition System to manage real-time event notifications and the digital issuance of participation certificates and badges.

1.3. Scope and Limitation

In-Scope

The HackET platform is a centralized, multi-tenant web ecosystem designed to manage the full lifecycle of hackathons in Ethiopia through the following core modules:

- **User and Role-Based Management:** The system provides triple-tier dashboards for Participants, Organizers, and Judges, including a dedicated interface for mentors to log team interactions and provide performance input.
- **Persistent Profiles:** Users can maintain profiles that archive project submissions and participation history while allowing participants to bookmark favorite institutions for filtered event tracking.
- **Unified Discovery Hub:** The platform serves as a centralized source of truth for all Ethiopian hackathons, featuring advanced filtering by region or theme and an integrated calendar view for tracking event-specific deadlines.

- **Organizer Toolkit:** Tools are provided for creating events, setting rules, and managing tiered permissions for staff roles like Technical Leads or Logistics coordinators.
- **Team Formation Engine:** A skill-based matching algorithm helps participants find compatible teammates based on profile data and is complemented by discussion boards for community knowledge sharing.
- **Secure Submission Portal:** A dedicated interface facilitates the upload of project links and documentation that are automatically associated with registered team members.
- **Digital Judging Dashboard:** The system automates scoring by aggregating judge inputs, applying normalization rules like outlier filtering, and displaying a transparent, real-time leaderboard.
- **Bilingual Accessibility and Localization:** All static system interfaces support English and Amharic and are optimized for East Africa Time (EAT) and the Ethiopian Calendar format.
- **Intelligent Personalization:** A content-based filtering recommendation engine delivers personalized event suggestions to participants based on their technical interests and past participation.
- **Automated Communication:** A tiered notification system delivers real-time, in-app alerts for critical deadlines and the automated issuance of post-event digital certificates or badges.
- **Event Analytics:** Data-driven reporting tools allow organizers to evaluate registration trends, submission quality, and judge performance through exportable PDF and CSV reports.

Out-of-Scope

While HackET aims to be a comprehensive solution, the following constraints define the boundaries of the current implementation:

- **Geographical and Network Dependence:** The platform assumes stable internet connectivity, meaning that real-time notifications and heavy media uploads may experience latency in areas with low bandwidth.
- **Mobile Access:** This phase is strictly limited to a responsive web application, with native mobile apps for Android and iOS currently excluded from the development cycle.

- **Dynamic Translation:** While the static UI is bilingual, the platform does not support real-time API-driven translation for user-generated content, requiring organizers and participants to provide bilingual text manually.
- **AI Constraints:** Advanced generative AI for project summaries and lightweight support chatbots are excluded to prioritize the stability of core event management features.
- **Payment and Financials:** The platform does not facilitate event registration fees or prize disbursements, as all financial transactions are required to remain external to the system.
- **Third-Party Integration:** Direct integration with live-coding environments or internal real-time chat systems is out of scope, as the platform focuses specifically on project metadata and file management.

1.4. Methodology

To bring the HackET platform to life, our team has chosen a structured yet user-focused methodology that combines User-Centered Design (UCD) with the Waterfall model. This approach aligns perfectly with our project schedule, where the comprehensive research and documentation phase is completed in the first semester, and full implementation and verification are carried out in the second.

UCD ensures that our design remains centered on the real needs of Ethiopia's fragmented innovation community, while the Waterfall model provides a clear, sequential process for moving from requirements and design to systematic implementation and testing. This combination allows us to balance user empathy with professional project execution through four main phases.

Phase 1: On-the-Ground Research and Design

Everything starts with understanding the people we are building for. This phase is dedicated to laying a solid foundation through localized research before writing any code.

- **Learning from the Community:**

- **Stakeholder Interviews:** We will conduct semi-structured interviews with 6–10 hackathon organizers (e.g., from university tech clubs and hubs like IceAddis) to define their specific workflow needs for event management and judging.
- **Broad Surveys:** We will follow up with surveys for 100–120 potential participants (students and developers) to validate key discovery barriers and language preferences (Amharic vs. English).
- **Making Sense of the Data:**
 - **User Personas:** We will create detailed personas—such as "Selam, the rural CS student" or "Dawit, the club organizer"—to keep user needs at the forefront of every design decision.
 - **User Stories and Use Cases:** From these personas, we will develop user stories that directly link every functional feature, such as the automated judging or team formation engine, back to a real, identified need.
- **Blueprint for the Platform:**
 - **UX/UI Design:** Our philosophy is focused on a straightforward, reliable interface that works well in low-bandwidth environments. We will progress from basic wireframes to high-fidelity, bilingual (Amharic-English) mockups.
 - **Technical Architecture:** We will map the technical backbone, including the client-server model, the PostgreSQL database structure, and the Node.js API design.

Phase 2: Sequential Building and Implementation

With a complete design and full documentation, development begins. Following the Waterfall approach, this phase is carried out sequentially, where each module is fully implemented and verified before moving to the next. The process will emphasize:

- Strict adherence to the approved design and architecture.
- Clear separation of tasks between frontend and backend developers.
- Internal verification and code reviews after the implementation of each core module.

Phase 3: Multi-Layered Testing and User Evaluation

Testing occurs after the development stage is complete to ensure the platform is both technically stable and easy to use.

- **Technical Quality Assurance:** We will employ a tiered strategy including Unit Testing (individual code components), Integration Testing (API-to-Database communication), and End-to-End Testing (simulating a full hackathon cycle from discovery to results).
- **Usability Study:** We will bring in 15–20 local developers and organizers to test-drive the application. By observing their interactions, we will identify and fix any "rough edges" in the bilingual user experience.

Phase 4: Pilot Deployment and Knowledge Continuity

The final phase focuses on the official launch and ensuring the long-term impact of the study.

- **Deployment:** The frontend will be deployed on Vercel, and the backend on Render, ensuring fast, reliable, and scalable access for users across Ethiopia.
- **Monitoring and Feedback:** We will initiate a pilot launch, closely monitoring system performance and user engagement. Feedback from this phase will guide future updates and ensure that HackET continues to bridge the gap in Ethiopia's innovation ecosystem.

1.5. Plan of Activities

This section outlines the detailed work plan for the development of the HackET platform. The project is executed over a seven-month period from October 12, 2025, to May 15, 2026. The schedule is structured to ensure that all documentation and foundational research are completed in the first semester, allowing for a focused implementation phase in the second.

The detailed project timeline is presented below using a Gantt chart, which illustrates the key phases, activities, and their corresponding durations.

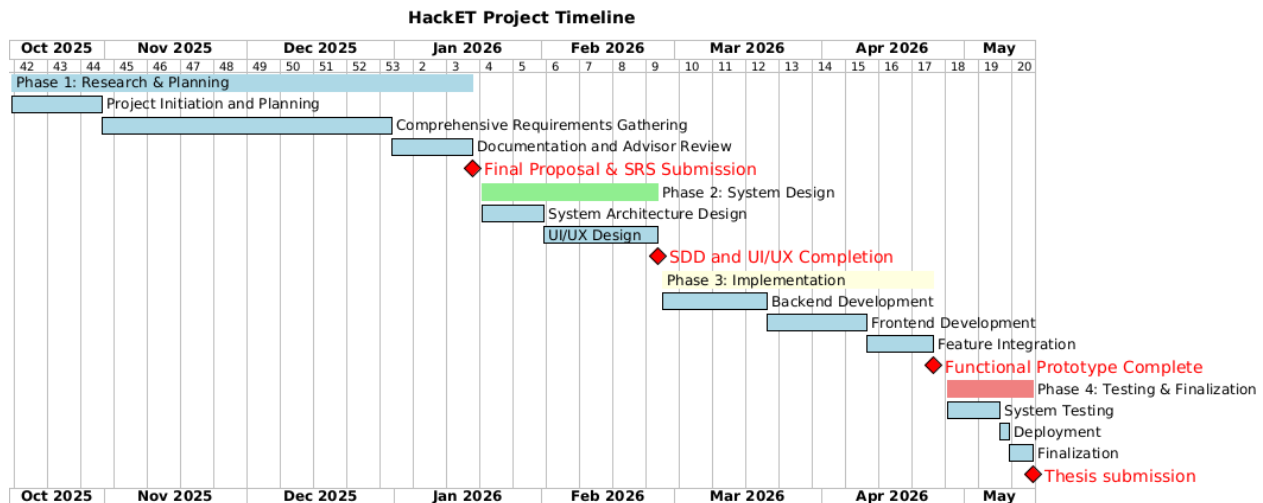


Figure 1.1. HackET Project Timeline and Gantt Chart

Phase 1: Research, Documentation, and Review (October 12 – January 17)

- **Objective:** To identify user needs through research, define system requirements, and finalize all project documentation and planning before technical design.
- **Key Milestone:** Submission of Finalized Project Proposal and Requirement Specification Document.
- **Activities:**
 - **Project Initiation and Planning (October):**
 - Finalize project charter and scope.
 - Set up the GitHub version control environment.
 - **Comprehensive Requirements Gathering (October - December):**
 - Conduct stakeholder interviews with 6–10 hackathon organizers and hubs.
 - Distribute surveys to 100–120 potential participants across Ethiopia.
 - Analyze competing platforms and existing solutions to identify gaps.
 - **Documentation and Advisor Review (December - January 17):**
 - Synthesize research findings into the final project proposal.
 - Perform rigorous editing of Chapters 1, 2, and 3.
 - Conduct advisor reviews and incorporate feedback to ensure the project scope is feasible and academically sound.

Phase 2: System Design and Architecture (January 19 – February 25)

- **Objective:** To transform requirements into a technical blueprint, designing the database and user interface.
- **Key Milestone:** Completion of System Design Document (SDD) and High-Fidelity Prototypes.
- **Activities:**
 - **System Architecture Design (January - February):**
 - Define the Entity Relationship Diagram (ERD) and database schema.
 - Map out the API endpoints and client-server communication protocols.
 - **UI/UX Design (February):**
 - Create low-fidelity wireframes for the web app.
 - Create high-fidelity prototypes ensuring accessibility for users outside Addis Ababa.

Phase 3: Core Development and Implementation (February 26 – April 24)

- **Objective:** To implement the functional system components sequentially using the Node.js stack.
- **Key Milestone:** A fully functional HackET web application with frontend and backend.
- **Activities:**
 - **Backend Development (February - March):**
 - Implement user authentication, RESTful APIs, and database models.
 - **Frontend Development (March - April):**
 - Build the responsive web dashboards for Participants, Organizers, and Judges using React.
 - **Feature Integration (April):**
 - Connect the automated judging module, the skill-based matching engine, and the bilingual translation support.

Phase 4: Testing, Deployment, and Finalization (April 27 – May 15)

- **Objective:** To validate the system through testing, deploy it to a production environment, and finalize the academic thesis.
- **Key Milestone:** Successful Deployment of Live Platform and Final Thesis submission.
- **Activities:**
 - **System Testing (April - May):**
 - Perform unit and integration testing to validate individual components and API responses.
 - Conduct a usability study with 15–20 representative users.
 - **Deployment (May):**
 - Launch the frontend on Vercel and the backend on Render for the pilot study.
 - **Finalization (May):**
 - Analyze pilot performance metrics and implement final refinements.
 - Compile the final project thesis, technical documentation, and presentation materials.

Deliverables:

- Project Proposal and Requirements Specification Document.
- System Design Blueprint and UI/UX Prototypes.
- Fully Functional HackET Software Package (Frontend & Backend).
- System Evaluation and Testing Reports.
- Final Project Thesis and Defense Presentation.

1.6. Budget Required

The following budget outlines the estimated financial resources required for the successful development and deployment of the HackET platform. The budget is categorized into Operational Costs and Contingency.

Category	Item Description	Quantity / Duration	Unit Cost (ETB)	Total Cost (ETB)	Justification
Operational Costs	Mobile Data	7 Months	1,800	12,600	Reliable internet for team collaboration, code deployment (GitHub), and remote research.
Operational Costs	Communication & Field Research	7 Months	600	4200	Airtime and transport costs for interviewing hackathon organizers, and coordinating usability testing.
Contingency	Miscellaneous / Reserve	—	—	1,680	~10% buffer to cover unforeseen fees.
TOTAL				18,480 ETB	

Table 1.1. HackET Project Estimated Budget and Financial Rationale

1.7. Significance of the Study

The significance of this study lies in its potential to transform Ethiopia’s fragmented innovation landscape into a cohesive, professional, and inclusive digital ecosystem. As the nation accelerates toward its Digital Ethiopia 2030 objectives, there is a critical need for infrastructure

that not only hosts competition but also preserves intellectual capital and nurtures technical talent across all regions. By introducing a centralized, multi-tenant platform, this research addresses the systemic inefficiencies currently hindering the growth of local startups and the professional development of young innovators.

At a practical level, HackET provides a much-needed departure from the "organizational chaos" of manual coordination. By replacing disparate communication channels like Telegram and spreadsheets with an integrated Organizer Toolkit, the platform significantly reduces administrative errors and ensures a higher standard of event execution. The introduction of an automated scoring system further professionalizes the ecosystem by applying normalization rules that eliminate judging bias, thereby ensuring that the most impactful solutions—rather than just the most visible—receive the recognition they deserve.

Furthermore, the study champions Digital Inclusion by dismantling the linguistic and geographical barriers that often centralize innovation in Addis Ababa. By providing a bilingual interface in both Amharic and English, the platform empowers developers from regional universities and rural hubs to participate in the digital economy on an equal footing. This linguistic accessibility is not merely a convenience; it is a vital tool for ensuring that Ethiopia's diverse pool of talent can contribute to solving local problems without being sidelined by language proficiency.

Beyond the immediate scope of event management, the research offers a long-term solution for Talent Archiving and Knowledge Continuity. Traditionally, the intellectual output of a hackathon is lost the moment the event ends. HackET changes this trajectory by building a Persistent Developer Identity, allowing recruiters, incubators, and investors to track the progress of innovators over time. This creates a sustainable "pipeline" of talent, turning weekend prototypes into viable startups and connecting local developers with global opportunities.

Finally, this study contributes significantly to the field of Software Engineering within the Ethiopian context. It serves as a technical case study on the implementation of Multi-Tenant Architectures and rule-based evaluation systems in environments characterized by variable internet connectivity. By demonstrating how localized tech platforms can be both robust and

user-centric, this research provides a blueprint for future developers seeking to build digital solutions that are culturally relevant, technically sound, and nationally impactful.

1.8 Outline of the study

This study is organized into seven chapters, each building upon the previous to provide a comprehensive understanding of the development and evaluation of the HackET platform. The chapters are as follows:

Chapter One – Introduction:

Introduces the background of the study, outlines the problem statement, objectives, scope, and methodology, and explains the significance of the study. It sets the foundation for understanding why the HackET platform is needed and what it aims to achieve.

Chapter Two – Literature Review:

Examines previous studies and existing systems related to educational technology platforms, bilingual translation tools, and mobile application development. It identifies the gaps that this study seeks to fill and summarizes key lessons from earlier works.

Chapter Three – Problem Analysis and Modeling:

Analyzes the weaknesses of current educational platforms in Ethiopia, particularly in terms of language accessibility and user engagement. It defines both the functional and non-functional requirements of the proposed solution and presents system models and diagrams to visualize how the solution will work.

Chapter Four – System Design:

Describes the overall design of the HackET platform, including its architecture, database design, subsystems, deployment model, and user interface. This chapter explains how each component contributes to solving the identified problem.

Chapter Five – System Implementation:

Details how the design was turned into a working system. It discusses the tools, programming languages, and technologies used to develop the platform, including frontend and backend development, integration of translation APIs, and notification services.

Chapter Six – System Evaluation:

Focuses on how the system was tested and evaluated. It includes the results of usability and performance tests to determine whether the HackET platform meets its intended objectives, such as user satisfaction, translation accuracy, and system reliability.

Chapter Seven – Conclusions and Recommendations:

Summarizes the main findings, reflects on the overall contribution of the study, and provides recommendations for further improvements and future research.

Chapter Two: Literature Review

This literature review presents an overview of existing studies and systems related to hackathon management, digital collaboration, multilingual platforms, and local innovation ecosystems. Its primary purpose is to synthesize existing knowledge, identify global best practices, and pinpoint the specific gaps—particularly fragmentation, lack of coordination, and linguistic barriers—that the HackET platform aims to address within the Ethiopian context. By reviewing key works, this section establishes the academic and technical foundation for the proposed system, ensuring its design is informed by both global insights and local realities.

2.1. Study Related Works

Studies relevant to the design and development of unified hackathon management systems, digital collaboration platforms, and multilingual interfaces are discussed below.

The first study, generally associated with Devpost, represents the leading platform for managing global hackathons. While not an academic study, its model of end-to-end event management serves as the industry standard, offering integrated registration, submission, and judging tools [1]. Its success lies in providing a single, unified workflow. However, Devpost's model is primarily English-centric and lacks features for robust localization and integration with regional collaboration tools common in contexts like Ethiopia.

Similarly, Unstop (formerly Dare2Compete), a major Indian platform, emphasizes gamification and community engagement through leaderboards and achievement badges [2]. While it supports a diverse user base, its core interface primarily uses English, creating an accessibility barrier for users in non-English-dominant regions. This reinforces the need for HackET to prioritize local accessibility.

In the Ethiopian context, research by Tadesse and Gebrehiwot (2020) analyzed the challenges of machine translation for Amharic, noting that while neural models are improving, contextual accuracy in technical domains (like hackathon themes) remains a significant hurdle [5]. Their work highlights that simple API translation is often insufficient without careful integration. Complementing this, Alemu (2018) studied user acceptance of Amharic interfaces in Ethiopia,

finding a strong preference for localized platforms, which significantly improves digital inclusion and reduces cognitive load [6].

Beyond localization, studies on general web-based event management offer insights into workflow. Müller et al. (2022) examined digital platforms for academic conferences, focusing on the necessity of user-role-based dashboards for different stakeholders (organizers, reviewers, attendees) [3]. This justifies HackET's design of distinct dashboards for Participants, Organizers, and Judges. Similarly, Chen and Lin (2021) emphasized integrated communication channels and knowledge archiving to ensure that project outcomes persist beyond a single event, directly addressing the problem of "scattered data" in the Ethiopian ecosystem [4].

Following this direction, Verma et al. (2022) developed a service-finding platform in Nigeria, designed to connect users with specialized providers in a resource-constrained environment [7]. While the domain differs, the proximity-based matching and user feedback systems are highly relevant to HackET's need for skill-based team formation. Furthermore, Adetunji et al. (2023) proposed an ecosystem model implemented with Flutter and Laravel that integrates mobile notifications and user reviews [8]. Despite its innovation, it depends heavily on constant connectivity, unlike the low-bandwidth optimizations planned for HackET.

Finally, foundational work by Deshpande and Karypis (2004) on Content-Based Filtering (CBF) provides the technical rationale for HackET's recommendation engine [9]. By matching item features (event tags) to user profile data (past participation), this rule-based approach serves as a practical, low-resource alternative to complex collaborative filtering, making it ideal for the Ethiopian technical context.

2.2. Identifying Milestones of the Related Literatures and Finding the Gaps

The reviewed studies demonstrate the gradual evolution of digital innovation management systems and identify several critical milestones and persistent gaps in the existing research.

Key Milestones

- **Centralized End-to-End Management:** Platforms like Devpost and Unstop established the foundational blueprint for digital innovation hubs by integrating the entire event

lifecycle—registration, submission, and judging—under a single digital roof, proving that centralization significantly boosts organizational efficiency [1], [2].

- **Role-Based Access and Workflow:** Research by Müller et al. (2022) confirmed the necessity of providing distinct user types (Organizers, Judges, and Participants) with tailored dashboards to streamline their specific tasks and prevent data overlap [3].
- **Sustainability and Knowledge Archiving:** Studies on collaborative learning platforms highlighted that the retention of project metadata and the creation of persistent discussion boards are key to building sustainable communities that outlast the duration of a single event [4].
- **Validated Need for Localization:** Empirical evidence from Alemu (2018) showed that providing localized language interfaces, such as Amharic, is a critical factor for achieving digital inclusion and lowering the cognitive barrier for users in diverse linguistic contexts [6].
- **Cost-Effective Personalization:** Foundational work on Content-Based Filtering (CBF) by Deshpande and Karypis (2004) demonstrated that effective user-event matching can be achieved through rule-based systems, offering a reliable and scalable alternative to resource-heavy AI models [9].

Identified Gaps

Despite this progress, the following gaps persist in the existing literature and current systems:

- **Lack of Unified Local Ecosystems:** Current systems in Ethiopia remain highly fragmented. While global solutions exist, there is no centralized cross-platform ecosystem optimized for the local innovation pipeline, forcing hubs to rely on disconnected general-purpose tools like Telegram and Google Forms [2].
- **Limited Low-Resource Language Support:** There is a notable lack of professional hackathon management systems that integrate comprehensive Amharic support. Global platforms remain English-centric, and the literature confirms that simple, non-integrated translation tools fail to bridge the accessibility gap for technical users [5].

- **Weak Post-Event Continuation:** Existing literature on hackathons often focuses on the "event-day" logistics, but there is a significant gap in automated mechanisms for post-hackathon project archiving and persistent developer portfolio tracking [4].
- **Absence of Automated Scoring Normalization:** While many systems handle submissions, few integrate automated mathematical normalization to eliminate judge bias or "outlier" scoring, a common pain point in large-scale technical evaluations.
- **Scalability for Multi-Provider Networks:** Many existing solutions are built as "one-off" websites for specific events. There is a lack of multi-tenant architectures that allow various independent universities and hubs to manage their own events under a shared, secure infrastructure.
- **Limited Local Academic Research:** Ethiopian academic studies primarily focus on general traffic management or broad ICT adoption. There is a specific lack of research into specialized software engineering frameworks designed to unify the "innovation lifecycle" for local developers, organizers, and judges within the Ethiopian context.

These gaps form the foundation for HackET, which aims to build a centralized, bilingual, and multi-tenant innovation hub optimized for the unique needs of the Ethiopian tech community.

2.3. Lessons Learned from Literatures

From these studies, several key lessons emerge that directly inform the HackET design and development strategy:

- **Centralization is the Core of Efficiency:** The success of global industry standards like Devpost confirms that unifying the entire event lifecycle—from registration to project archiving—into a single digital environment is the most effective solution to fragmented data. HackET must prioritize this end-to-end integration to eliminate the "Information Silos" typical of the current Ethiopian landscape [1].
- **Bilingual Integration Drives Inclusivity:** Research into the Ethiopian digital context emphasizes that local language interfaces are not merely a feature but a necessity for user acceptance. For HackET, providing a seamless Amharic-English experience is essential to bridge the digital divide for innovators across regional hubs [6].

- **Persistent Data Sustains Innovation:** Lessons from collaborative learning studies show that innovation is lost when data is not archived. Building a "Persistent Developer Identity" ensures that the platform provides long-term value to participants and recruiters, filling the gap of "Institutional Amnesia" identified in prior event-specific efforts [4].
- **Rule-Based Personalization is Practical:** Global research into recommendation engines demonstrates that high-value user matching does not always require complex AI. Implementing a robust, rule-based matching system allows HackET to remain performant and reliable in low-resource settings while still delivering personalized event discoveries [9].
- **Role-Specific Tools Ensure Professionalism:** Effective management depends on specialized workflows for different stakeholders. By integrating dedicated dashboards—particularly an automated Judging Module—HackET can reduce manual errors and ensure the transparency required to build trust within the local tech ecosystem [3].

Overall, the literature demonstrates strong progress toward digitized innovation management but lacks unified, bilingual, and context-aware systems for the East African region. HackET builds on these insights by integrating a centralized multi-tenant architecture, persistent developer portfolios, and localized accessibility to deliver a more efficient and inclusive hackathon experience for Ethiopia's growing tech community.

Chapter Three: Problem Analysis and Modeling

This chapter presents a detailed analysis of the existing hackathon ecosystem in Ethiopia. It identifies the inherent inefficiencies, communication gaps, and technological limitations that affect both organizers and participants, demonstrating how these problems result in a fragmented innovation community. This analysis provides the necessary foundation for defining the functional and non-functional requirements of the proposed solution, HackET. By systematically dissecting the problem, this chapter establishes a clear link between the identified shortcomings and the specific design features required for a centralized, multilingual, and efficient hackathon management platform.

3.1. Existing System and Its Problems

The existing ecosystem for organizing and participating in hackathons in Ethiopia is a blend of deeply entrenched manual practices and isolated, event-specific digital tools, both of which present significant limitations to the growth of a unified innovation pipeline.

The Traditional Manual System

The predominant method for managing hackathons remains fragmented and ad-hoc. Most organizers—ranging from university tech clubs to regional innovation hubs—rely on a patchwork of general-purpose tools to bridge organizational gaps. This approach is characterized by several critical problems:

- **Information Fragmentation:** Event announcements are scattered across dozens of disparate Telegram channels and Facebook groups. This creates a high "discovery barrier," where potential participants frequently miss opportunities simply because they are not subscribed to a specific niche channel.
- **Inefficient Workflow Management:** Coordination relies heavily on manual data entry. Organizers use Google Forms for registration, Excel for scoring, and Google Drive for submissions. Without an integrated system, staff must act as "human APIs," manually moving data between these tools, which leads to significant administrative overhead and data errors.

- **Judging and Transparency Risks:** Scoring is often conducted via printed scorecards or private spreadsheets. This lack of a real-time, automated leaderboard delays results and can lead to perceptions of bias or a lack of transparency in how winners are selected.
- **Institutional Amnesia:** Once an event concludes, the data—including project prototypes and participant contact info—is often archived in local files or lost. This prevents long-term community growth, as there is no central repository for others to learn from past winning solutions.

Isolated Digital Solutions: Event-Specific Silos

In response to the growing tech scene, high-profile events (such as the *NASA Space Apps Addis* or *A2SV AI for Impact*) occasionally deploy dedicated landing pages or use global platforms like Devpost.

Synthesized Limitations of Existing Systems

While these digital efforts are an improvement, a combined analysis reveals persistent and critical gaps that form the central problem HackET addresses:

- **Lack of Participant Empowerment:** Existing tools are "one-way" registration forms. They do not offer features for skill-based team formation or personalized recommendations, leaving participants to find teammates through chaotic Telegram group chats.
- **Absence of Bilingual Inclusion:** Leading global platforms and local landing pages are almost exclusively English-centric. This creates a linguistic barrier for talented developers in regional universities (e.g., Bahir Dar, Hawassa) who prefer or require Amharic interfaces for technical engagement.
- **Limited Tools for Organizers:** General-purpose forms do not provide automated scoring normalization (to handle outlier judges) or real-time analytics to track event health, forcing organizers to perform complex manual calculations.
- **Geographical and Economic Centralization:** Most structured hackathon activity is confined to Addis Ababa. The lack of a unified national hub leaves innovators in remote regions disconnected from the central innovation "Source of Truth."

- **Gaps in Data Continuity:** There is no Persistent Developer Portfolio. Unlike global standards, local participants cannot build a verifiable digital resume of their hackathon history across different organizers, hindering their employment and networking prospects.

These shared limitations highlight the urgent need for a next-generation platform that moves beyond "flash-in-the-pan" event sites to create a comprehensive, multi-tenant, and widely accessible innovation ecosystem.

3.2. Specifying the Requirements of the Proposed Solution

The shortcomings of the existing systems directly inform the foundational requirements for the HackET platform. The proposed solution is designed to be an integrated ecosystem that addresses the multifaceted needs of both participants and organizers through a set of carefully specified features.

A core requirement is the development of a real-time, centralized event discovery hub. This will allow participants to instantly find and request registration for upcoming hackathons, or filter future events by specific technical themes, from a network of verified university hubs and innovation centers, directly tackling the uncertainty and fragmentation of current social media channels. To further reduce wasted effort and missed opportunities, the platform will include a bilingual interface feature. This will not be a complex linguistic model but a simple, integrated tool allowing users to view the entire system in Amharic or English. This gives participants crucial accessibility to help them understand event rules and submission criteria before initiating their registration.

To fill the significant gap in participant empowerment, the system will incorporate a rule-based recommendation engine. This "virtual mentor" is a key differentiator, designed to analyze the user's skill profile and past participation to suggest relevant events and possible teammates. By providing clear, automated suggestions for skill-matching, this feature will reduce the "disorganized workflow" of team formation, saving participants time and improving project quality.

To support the growth of the local ecosystem and encourage platform adoption, HackET introduces a multi-tenant organizer model. Subscribed organizers gain priority listing of their events, access to advanced analytics dashboards, and enhanced visibility to the national developer community. This ensures that institutions not only attract top talent but can also make data-driven decisions to optimize their judging process, manage project archives, and strategically grow their local innovation chapters.

Building a foundation of trust and transparency is paramount. Therefore, the system will implement a multi-layered trust and evaluation framework. This begins with a mandatory verification process for all organizers before they can host an event on the platform. To maintain quality and foster a reliable community, this will be complemented by a standardized judging and scoring system, allowing judges to record scores digitally and help participants receive formal feedback. To further enhance the user experience and reduce anxiety during the high-pressure submission window, the platform will also provide real-time status tracking of a project's submission and evaluation progress.

Finally, to ensure the platform is a valuable partner for the tech community, HackET will provide a comprehensive dashboard for participants and organizers. This goes beyond simple notifications and includes tools for portfolio and project management, as well as an analytics module. This module will offer insights into skill trends, participation history, and performance metrics, empowering innovators and hubs to optimize their participation and grow their influence in the Ethiopian tech landscape.

3.3. System Modeling

System modeling provides a visual and structural representation of the HackET platform to validate system requirements and functionalities. The following models describe the system's structure, interactions, and processes.

3.3.1. Functional Requirements

This section outlines the functional requirements of the proposed HackET system. Each requirement defines what the system should do to meet user and business needs. The functions

are grouped by major modules, with each having a priority level and clear operational description.

1. User and Role Management

ID	Requirement Description	Priority
FR-USR-01	The system shall allow Participants to register using Full Name, valid Email (regex-validated), Technical Skills/Interests, and Password.	High
FR-USR-02	The system shall allow Organizers to register using Organization Name, Representative Name, Institutional Email, Password, and a mandatory hub/university description.	High
FR-USR-03	The system must validate all input data for completeness and format (e.g., non-empty fields, valid email structure) before allowing submission.	High
FR-USR-04	The system must hash passwords (using BCrypt or similar) and encrypt sensitive personal data before storage in the database.	High
FR-USR-05	The system must prevent duplicate account creation by checking the database for existing email addresses during the registration process.	High
FR-USR-06	The system must verify login credentials and restrict system access to authenticated users only.	High
FR-USR-07	The system shall redirect users to their specific role-based dashboard (Participant, Organizer, or Judge) within 2 seconds of successful authentication.	High

FR-USR-08	The system must automatically terminate user sessions after 30 minutes of inactivity to ensure account security.	High
FR-USR-09	The system should allow Participants to maintain a profile that archives all project submissions and participation history as a unified portfolio.	Medium
FR-USR-10	The system shall allow Organizers to update and manage their event host profiles and institutional contact details.	Medium
FR-USR-11	The system may allow Participants to "bookmark" specific Organizers or Institutions for filtered tracking of their future events.	Medium

Table 3.1. User and Role Management Requirements

2. Event Discovery and Management

ID	Requirement Description	Priority
FR-EVT-01	The system shall provide a centralized listing of all current and upcoming Ethiopian hackathons on a single unified platform.	High
FR-EVT-02	The system shall provide search and filter functionality based on Region, Theme, Technical Category (e.g., AI, Web), and Schedule.	High
FR-EVT-03	The system shall provide an integrated calendar view displaying event-specific deadlines (Registration, Submission, Judging).	High
FR-EVT-04	The system shall enable users to register for specific events directly through the platform interface.	High

FR-EVT-05	The system must automatically link the user_id to the event_instance_id upon registration for lifecycle tracking.	High
FR-EVT-06	The system must allow Organizers to create, configure parameters, and publish new hackathon events.	High
FR-EVT-07	The system must incorporate a Rule-Based Categorization Module to auto-assign technical tags based on keywords found in the event description.	High

Table 3.2. Event Discovery and Management Requirements

3. Collaboration and Community

ID	Requirement Description	Priority
FR-COL-01	The system shall provide tools for the creation and management of teams (inviting members, defining team names).	High
FR-COL-02	The system should implement a matching algorithm to suggest teams/partners based on a match between "Participant Skills" and "Team Needs."	High
FR-COL-03	The system shall provide a mentor coordination interface where mentors can log interactions and provide input on assigned teams.	High
FR-COL-04	The system should host Discussion Boards and a post-event rating system for each event.	Medium
FR-COL-05	The system shall provide a structured form for Participants to submit feedback on organization, rules, and judging.	Medium

Table 3.3. Collaboration and Community Requirements

4. Submission and Evaluation

ID	Requirement Description	Priority
FR-SUB-01	The system shall provide a secure submission interface for project links (GitHub/GitLab), PDF documentation, and video links.	High
FR-SUB-02	The system must automatically associate every submission with the registered team and all individual team members.	High
FR-SUB-03	The system shall provide a permission-based Judging Dashboard accessible only to users with the 'Judge' role.	High
FR-SUB-04	The dashboard must support score input and text comments based on the specific criteria configured by the Organizer.	High
FR-SUB-05	The system must automatically aggregate and normalize scores (e.g., dropping high/low scores) based on the Organizer's predefined rules.	High
FR-SUB-06	The system shall display a Transparent Judging Scoreboard to Participants once rankings are finalized.	High
FR-SUB-07	The system shall provide a mechanism for Judges to deliver structured, written feedback to teams regarding their performance.	High

Table 3.4. Submission and Evaluation Requirements

5. Core System Enhancements

ID	Requirement Description	Priority
----	-------------------------	----------

FR-SYS-01	The system must support a bilingual (Amharic and English) interface for all user-facing components.	High
FR-SYS-02	The system should provide a manual toggle for organizers to provide translations.	Medium
FR-SYS-03	The system must allow users to toggle languages at any time, updating the UI dynamically without a full page reload.	High
FR-SYS-04	The system should implement a Content-Based Filtering Recommendation Engine to match event categories with user profile data.	Medium
FR-SYS-05	The system should deliver personalized event recommendations to users based on their interests and past participation.	Medium
FR-SYS-06	The system must provide Tiered Notifications (Reminders) for critical deadlines based on hard-coded time triggers.	High
FR-SYS-07	Notifications must be delivered in real-time within the application (in-app notifications).	High
FR-SYS-08	The system shall automatically trigger notifications to Participants upon the issuance of post-event Badges or Certificates.	High

Table 3.5. Core System Enhancements Requirements

6. System Analytics and Reporting

ID	Requirement Description	Priority
FR-ANA-01	The system shall provide an Analytics Dashboard for Organizers to track Registration Rates, Submission Rates, and Judge Performance.	High

FR-ANA-02	The system must allow the export of all generated reports in PDF and CSV formats.	High
FR-ANA-03	The system must automatically track and visualize participant engagement trends across different event cycles.	High

Table 3.6. System Analytics and Reporting Requirements

3.3.2. Non-Functional Requirements

This section defines the quality attributes and constraints that govern the performance, usability, reliability, and security of the proposed HackET system. These requirements ensure that the system not only meets functional expectations but also delivers a consistent, efficient, and secure user experience across different environments.

1. Performance and Scalability

ID	Requirement Description	Priority
NFR-PER-01	The system shall ensure that 90% of key user actions (page loads, form submissions, and score calculations) respond within 3 seconds under normal network conditions.	High
NFR-SCA-01	The system must support at least 1,000 concurrent users (Participants, Organizers, and Judges) during peak windows without performance degradation.	High
NFR-SCA-02	The system shall allow for the addition of cloud resources (CPU/RAM) to accommodate increasing user loads and event volumes.	High
NFR-SCA-03	The Node.js architecture should allow future integration of additional modules (e.g., payment gateways, real-time chat) without structural redesign.	Medium

Table 3.7. Performance and Scalability Requirements

2. Availability and Reliability

ID	Requirement Description	Priority
NFR-AVA-01	The platform shall maintain at least 95% uptime, ensuring stability during high-stakes submission and judging periods.	Medium
NFR-AVA-02	The system must maintain redundant database backups to prevent the loss of project archives and participant histories.	High
NFR-REL-01	The system shall log all runtime errors, including timestamps and user session details, for diagnostic purposes.	Medium

Table 3.8. Availability and Reliability Requirements

3. Security Requirements

ID	Requirement Description	Priority
NFR-SEC-01	Access must be permission-based with unique credentials and distinct dashboards for Organizers, Judges, and Participants.	High
NFR-SEC-02	The system shall support granular permissions within the Organizer role (e.g., Technical Lead vs. Logistics Staff).	High
NFR-SEC-03	Sensitive data (passwords, submission metadata) must be encrypted and transmitted via secure HTTPS connections.	High
NFR-SEC-04	The system shall use PostgreSQL to ensure strong ACID compliance for reliable profiles, event logs, and archives.	High

NFR-SEC-05	The system must enforce mandatory automatic session expiration for Organizer and Judge roles after a defined period of inactivity.	High
NFR-SEC-06	The system shall maintain activity logs for all administrative actions, including organizer verification and scoring modifications.	Medium

Table 3.9. Security Requirements

4. Usability and Localization

ID	Requirement Description	Priority
NFR-USA-01	The interface shall use clear icons and readable typography for both English and Amharic script.	High
NFR-USA-02	The web application must be responsive and function properly on slow internet connections typical in regional areas of Ethiopia.	High
NFR-USA-03	New users should be able to complete primary tasks (registration/submission) within 10 minutes of first use without formal training.	Medium
NFR-USA-04	The entire interface and all system notifications must be available in both English and Amharic.	High
NFR-LOC-01	Date and time formats shall follow Ethiopian regional settings (EAT) and support the Ethiopian calendar for scheduling.	Medium

Table 3.10. Usability and Localization Requirements

5. Maintainability and Portability

ID	Requirement Description	Priority
-----------	--------------------------------	-----------------

NFR-MTN-01	All modules shall include inline documentation and follow consistent naming conventions for iterative improvements.	Medium
NFR-MTN-02	Software updates (bug fixes/enhancements) shall be deployable with minimal downtime during active hackathon cycles.	Medium
NFR-POR-01	The responsive web application must function smoothly across various mobile and desktop platforms.	High
NFR-POR-02	The application shall be compatible with the latest versions of Chrome, Edge, and Firefox.	Medium
NFR-POR-03	The backend should support deployment on scalable cloud platforms such as Render or Vercel.	Medium

Table 3.11. Maintainability and Portability Requirements

3.3.3. Usecase

This section identifies the main actors and use cases of the proposed HackET system. Use cases describe how different users interact with the system to accomplish specific goals. They also help visualize the system's boundaries, its functionalities, and the relationships between actors and processes.

The HackET platform involves six main actors: the Participant, the Organizer, the Judge, the Mentor, the Administrator, and the Timer / Clock.

HackET Usecase Diagram

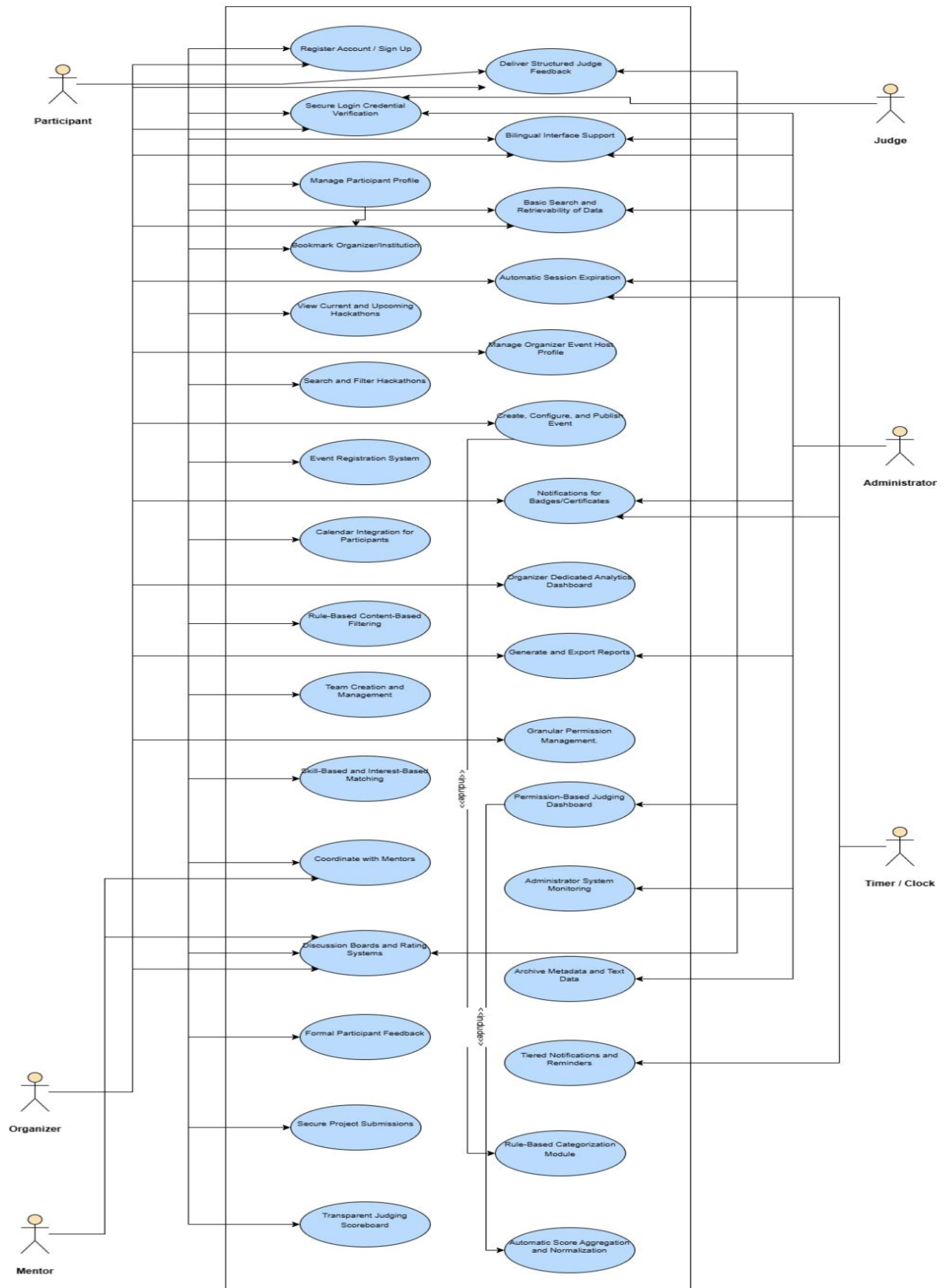


Figure 3.1: Use Case Diagram of HackET

Actor	Type	Description and Role
Participant	Primary	A registered individual who uses the HackET web application to discover and register for hackathons, form or join teams, submit projects, participate in event discussions, request mentorship, and receive certificates upon completion.
Organizer	Primary	A verified organization representative who creates and configures hackathons, manages the full event lifecycle from draft to archival, invites and manages event staff, reviews participant registrations, and exports analytics reports. Organizer accounts require administrator approval of their organization before gaining full platform access.
Event Staff	Supporting	An existing platform user assigned a scoped staff role for a specific hackathon via a StaffAssignment record. Staff roles include CO_ORGANIZER (full event co-management), TECHNICAL_LEAD (judging configuration and score normalization), LOGISTICS (participant check-in and scheduling), COMMUNICATIONS (announcements and content publishing), FINANCE (prize management), SPONSOR (participant data visibility), JUDGE and MENTOR

Administrator	Primary	A platform operator who verifies and approves organizer accounts, suspends or reactivates user accounts, reviews flagged discussion content, manages system-wide broadcasts, and generates platform usage and analytics reports.
----------------------	---------	--

Table 3.12. Actor Descriptions and Roles in HackET

Use Case ID	UC0001	
Use Case Name	Register Account / Sign Up	
Description	The user (Participant or Organizer) registers by providing personal and credential information to create a new account; Organizers must provide additional institutional verification data.	
Trigger	The user selects the “Sign Up” option on the application.	
Actors	Participant, Organizer	
Pre-conditions	The user must not have an existing account.	
Main Flow	User: 1. Selects “Sign Up.” 3. Fills in personal and credential details. 5. Submits registration.	System: 2. Displays registration form for personal details, role selection (Participant/Organizer), and credentials. 4. Validates input data.

		6. Stores user data securely in the database. 7. Returns a success confirmation message.
Alternative Flow	AF1: Email Already Registered Condition: User enters an email already linked to an existing account. Steps: 1. System detects duplicate email. 2. System displays a message: “This email is already registered. Please log in or use another.” 3. User is redirected to Login or allowed to enter a different email. AF2: Weak Password Condition: Password does not meet security criteria. Steps: 1. System validates password strength and detects a weak one. 2. System displays: “Your password is too weak. Please use at least X characters and include letters/numbers/special symbols.” 3. User re-enters a stronger password, returning to Step 4 of Main Flow. AF3: Missing Required Fields Condition: User leaves one or more mandatory fields blank. Steps: 1. System detects incomplete fields. 2. System highlights missing fields and shows an error message.	

	3. User fills in missing fields, returning to Step 4 of Main Flow. AF4: Server or Network Error Condition: System fails to store data due to server or network issues. Steps: 1. System attempts to save data. 2. System fails due to an internal error. 3. System displays: “Registration failed due to server problem. Please try again later.” 4. User retries the process once the system is available.
Exception Conditions	The user enters invalid or duplicate information.
Post-condition	A new user account is created and stored in the system.

Table 3.3: Use Case Specification: Register Account / Sign Up (UC0001)

Use Case ID	UC0002
Use Case Name	Secure Login Credential Verification
Description	The user authenticates by providing valid credentials to access the system and their role-based dashboard.
Trigger	The user presses the “Login” button after entering credentials.
Actors	Participant, Organizer, Judge, Mentor, Administrator
Pre-conditions	Users must have a valid registered account.

Main Flow	User: 1. Selects “Login.” 2. Inserts credentials.	System: 3. Checks the database for matching credentials. 4. If valid, identifies the user's role. 5. Opens the corresponding Role-Based Dashboard.
Alternative Flow	AF1: Incorrect Credentials Condition: User enters the wrong password or email/username. Steps: 1. System detects credential mismatch. 2. System displays: “Incorrect credentials. Please try again.” 3. The user may retry the login or choose “Forgot Password.” AF2: Account Not Found Condition: Email/username does not match any registered account. Steps: 1. System displays: “Account not found. Please create an account.” 2. The user may go to Sign Up. AF3: Forgot Password Flow Condition: User selects “Forgot Password.” Steps: 1. System asks for email/username. 2. System sends reset link/OTP.	

	3. User resets the password and returns to login (Main Flow Step 1). AF4: Server/Network Issue Condition: System fails to check user credentials due to an internal error. Steps: 1. System detects a communication failure. 2. System displays: “Unable to process login. Please try again later.” 3. The user retries once the system recovers.
Exception Conditions	The user entered incorrect credentials.
Post-condition	The user will be logged in and able to access their role-specific dashboard and services.

Table 3.4: Use Case Specification: Secure Login Credential Verification (UC0002)

Use Case ID	UC0003
Use Case Name	Logout
Description	Ends the current user session immediately and clears all local session data to prevent unauthorized access.
Trigger	The user selects the "Logout" option from the dashboard or navigation menu.
Actors	Participant, Organizer, Judge, Mentor, Administrator
Pre-conditions	The user must be logged in and have an active session.

Main Flow	User: 1. Selects "Logout" from the user menu.	System: 2. Invalidates the active authentication token on the server. 3. Clears local session storage, authentication cookies, and temporary cache. 4. Redirects the user to the public landing page or login screen. 5. Displays a confirmation message: <i>"You have been successfully logged out."</i>
Alternative Flow	AF1: Automatic Logout via Expiration Condition: The user remains inactive for the period defined in the non-functional requirements. Steps: 1. System detects session timeout. 2. System automatically performs Main Flow steps 2–4. 3. System displays: <i>"Session expired due to inactivity. Please log in again."</i>	

Exception Conditions	Server Connection Failure
Post-condition	The user session is terminated; all protected dashboards are inaccessible without re-authentication.

Table 3.4: Use Case Specification: Logout (UC0003)

Use Case ID	UC0004	
Use Case Name	Automatic Session Expiration	
Description	Ends the active session for Organizer and Judge roles securely after a period of inactivity.	
Trigger	The Timer / Clock detects inactivity for a predefined duration for an Organizer or Judge session.	
Actors	Organizer, Judge, Timer / Clock	
Pre-conditions	The Organizer or Judge must be logged in and inactive for the predefined duration.	
Main Flow	Timer / Clock: 1. Signals inactivity timeout.	System: 2. Automatically ends the user session and clears temporary data. 3. Displays message: “Your session has expired due to inactivity. Please log in again.” 4. Redirects to the login screen.

Alternative Flow	AF1: Inactivity Warning Condition: The user approaches the inactivity timeout limit (e.g., 5 minutes before expiration). Steps: 1. System displays a warning: “Your session will expire in 5 minutes due to inactivity. Click to continue.” 2. User interacts with the app, returning to an active session. AF2: Participant Role Exemption Condition: The inactive user is a Participant. Steps: 1. System identifies the user as a Participant. 2. System allows a longer or permanent session duration (does not trigger session expiration).
Exception Conditions	System failure while clearing the session.
Post-condition	Organizer/Judge session terminated successfully due to inactivity.

Table 3.5: Use Case Specification: Automatic Session Expiration (UC0004)

Use Case ID	UC0005
Use Case Name	Manage Participant Profile
Description	Allows the Participant to view and update their profile information and view their current project submission status and list of past participation.
Trigger	The Participant opens the “Profile” section.

Actors	Participant	
Pre-conditions	Participant is logged in.	
Main Flow	Participant: 1. Opens “Profile.” 3. Updates profile details. 4. Confirms changes.	System: 2. Displays current profile details, project submission status, and list of past participation (metadata only). 5. Validates and saves changes to the database.
Alternative Flow	AF1: View Past Participation Details Condition: Participant selects to view a past participation item. Steps: 1. System displays key details for the selected past participation (metadata: event name, date, role). AF2: Profile Data Validation Failure Condition: Participant enters invalid data (e.g., email format, phone number). Steps: 1. System detects invalid data in Step 5. 2. System displays an error message highlighting the incorrect fields. 3. Participant corrects the data, returning to Step 4 of Main Flow.	
Exception Conditions	Database error during update.	
Post-condition	Participant profile details are updated, and status/history is viewed.	

Table 3.6: Use Case Specification: Manage Participant Profile (UC0005)

Use Case ID	UC0006	
Use Case Name	Manage Organizer Event Host Profile	
Description	Allows the Organizer to view and update their organization's host profile information.	
Trigger	The Organizer opens the “Host Profile” section.	
Actors	Organizer	
Pre-conditions	Organizer is logged in.	
Main Flow	Organizer: 1. Opens “Host Profile.” 3. Updates profile details. 4. Confirms changes.	System: 2. Displays current organization profile details. 5. Validates and saves changes to the database.
Alternative Flow	AF1: Data Validation Failure Condition: Organizer submits invalid or incomplete mandatory information. Steps: 1. System detects validation errors in Step 5.	

	2. System displays an error message highlighting the issues. 3. Organizer corrects the data, returning to Step 4 of Main Flow. AF2: Leave Without Changes Condition: Organizer leaves the profile page without submitting changes. Steps: 1. System detects unsaved changes. 2. System prompts: “Discard unsaved changes?” Organizer confirms discard. 3. System returns to the previous screen.
Exception Conditions	Database error during update.
Post-condition	Organizer Event Host profile details are updated.

Table 3.7: Use Case Specification: Manage Organizer Event Host Profile (UC0006)

Use Case ID	UC0007
Use Case Name	Bookmark Organizer/Institution
Description	Allows the Participant to add or remove Organizers/Institutions to a “Favorite” list.
Trigger	The Participant selects the “Bookmark” or “Favorite” action on an Organizer’s profile or event listing.
Actors	Participant
Pre-conditions	Participant is logged in. Organizer/Institution must exist.

Main Flow	Participant: - Views an Organizer's profile or event. - Selects the "Bookmark" option.	System: - Adds the Organizer to the Participant's list of favorites. - Displays confirmation of the update.
Alternative Flow	AF1: Remove Bookmark Condition: Participant selects the bookmark icon again for an already bookmarked Organizer. Steps: - System removes the Organizer from the favorites list. - System confirms removal. AF2: Reaching Max Favorites Condition: Participant attempts to bookmark when the maximum allowed limit is reached. Steps: - System detects the limit is reached. - System displays: "Maximum number of favorites reached. - Please remove an existing one to add a new one." - Bookmark action is prevented.	
Exception Conditions	Database error prevents saving the update to the favorites list.	
Post-condition	The Organizer/Institution is added to or removed from the Participant's favorite list.	

Table 3.8: Use Case Specification: Bookmark Organizer/Institution (UC0007)

Use Case ID	UC0008
--------------------	--------

Use Case Name	View Current and Upcoming Hackathons	
Description	Displays a centralized, unified list of all current and upcoming hackathons across Ethiopia.	
Trigger	The user opens the main “Events” or “Discover” page.	
Actors	Participant, Organizer, Judge, Mentor, Administrator, Public User	
Pre-conditions	User is logged in or accessing the public event list.	
Main Flow	User: 1. Opens the event discovery page.	System: 2. Retrieves and displays the centralized list of current and upcoming hackathons.
Alternative Flow	AF1: No Upcoming Events Condition: The database contains no currently active or upcoming published events. Steps: 1. System displays: “There are no active or upcoming hackathons currently listed.”	
Exception Conditions	Database retrieval error.	
Post-condition	A list of current and upcoming hackathons is displayed.	

Table 3.9: Use Case Specification: View Current and Upcoming Hackathons (UC0008)

Use Case ID	UC0009
--------------------	--------

Use Case Name	Search and Filter Hackathons	
Description	Allows users to search and filter hackathons by region, theme, category, and schedule.	
Trigger	The user enters a search term or selects a filter option.	
Actors	Participant, Organizer, Judge, Mentor, Administrator, Public User	
Pre-conditions	A list of hackathons is displayed.	
Main Flow	User: 1. Enters search text or selects filter criteria (region, theme, category, schedule).	System: 2. Queries event data based on search/filter criteria. 3. Updates the displayed list to show only matching events.
Alternative Flow	AF1: No Results Found Condition: The applied filter or search yields zero events. Steps: 1. System displays a message: “No events found matching your criteria.” 2. System may suggest removing filters. AF2: Search Auto-Correction/Suggestion Condition: User misspells a search term (e.g., region name). Steps: 1. System detects possible misspelling. 2. System displays search results for the corrected term and shows a message: “Showing results for [Corrected Term]. Search for [Original Term] instead?”	

Exception Conditions	Database search error.
Post-condition	The displayed list of hackathons is refined according to the search and filter criteria.

Table 3.10: Use Case Specification: Search and Filter Hackathons (UC0009)

Use Case ID	UC0010	
Use Case Name	Event Registration System	
Description	Enables a Participant to register for a selected hackathon event.	
Trigger	The Participant presses the “Register” button on an event's detail page.	
Actors	Participant	
Pre-conditions	Participant is logged in. The event is open for registration.	
Main Flow	Participant: 1. Selects an event. 2. Presses “Register.”	System: 3. Validates registration eligibility (e.g., not already registered). 4. Records the Participant's registration in the event roster. 5. Displays a confirmation message.

Alternative Flow	AF1: Registration Closed Condition: Registration deadline has passed, or the event capacity is reached. Steps: 1. System displays: “Registration for this event is closed (or capacity reached).” 2. Registration is blocked. AF2: Already Registered Condition: The Participant attempts to register for an event they are already registered for. Steps: 1. System detects existing registration. 2. System displays: “You are already registered for this event.” AF3: Pre-requisite Failure Condition: The event has required pre-requisites (e.g., specific age/skill), and the Participant does not meet them. Steps: 1. System validates pre-requisites. 2. If failed, system displays: “You do not meet the required criteria to register for this event.” 3. Registration is blocked.
Exception Conditions	Database error during registration.
Post-condition	The Participant is successfully registered for the event.

Table 3.11: Use Case Specification: Event Registration System (UC0010)

Use Case ID	UC0011
--------------------	--------

Use Case Name	Regional Calendar and Deadline Integration	
Description	Allows Participants to view and add key event deadlines to their personal calendars, with native support for toggling between Gregorian and Ethiopian Calendar formats.	
Trigger	The Participant views event details or selects the “Add to Calendar” option.	
Actors	Participant	
Pre-conditions	Participant is logged in and viewing an event with defined deadlines (Registration, Submission, Judging).	
Main Flow	Participant: - Views the event timeline. - Toggles between Gregorian and Ethiopian calendar views to verify dates. - Selects the “Add to Calendar” option for a specific deadline. - Saves the entry to their external personal calendar.	System: - Updates the UI to display the equivalent dates in the selected format. - Generates a standardized calendar file (.ics).
Alternative Flow	AF1: Calendar Generation Failure Condition: The system fails to generate the calendar file due to an internal error. Steps: System displays: “Unable to generate the calendar file. Please try again later.”	

Exception Conditions	File generation error; Date conversion logic failure.
Post-condition	The Participant has successfully viewed deadlines in their preferred local format and/or exported the entry.

Table 3.12: Use Case Specification: Regional Calendar and Deadline Integration (UC0011)

Use Case ID	UC0012	
Use Case Name	Create, Configure, and Publish Event	
Description	Enables the Organizer to define, configure all details (rules, deadlines, criteria), and publish a new hackathon event.	
Trigger	The Organizer selects “Create New Event” from the dashboard.	
Actors	Organizer	
Pre-conditions	Organizer is logged in.	
Main Flow	Organizer: 1. Selects “Create New Event.” 3. Enters event details, deadlines, scoring criteria, and rules. 4. Confirms save. 5. Selects “Publish.”	System: 2. Displays event configuration forms 6. Validates event data. 7. Stores event configuration in the database. 8. Marks the event as published, making it visible in event discovery.

Alternative Flow	AF1: Save as Draft Condition: Organizer saves the event without publishing. Steps: 1. Organizer selects “Save Draft” in Step 4. 2. System saves the configuration with a “Draft” status. 3. Event remains invisible to Participants. AF2: Missing Mandatory Fields Condition: Organizer attempts to publish with missing mandatory fields (e.g., description, key deadlines). Steps: 1. System detects incomplete mandatory fields in Step 6. 2. System highlights missing fields and blocks publishing. 3. Organizer fills in missing fields, returning to Step 4 of Main Flow. AF3: Duplicate Event Name Condition: Organizer uses an event name that is identical to a recent or highly prominent existing event. Steps: 1. System detects a potential naming conflict. 2. System displays a warning. 3. Organizer may choose to override the warning or edit the name, returning to Step 4 of Main Flow.
Exception Conditions	Input validation failure (e.g., deadlines in the past).
Post-condition	A new hackathon event is created, configured, and either saved as a draft or published.

Table 3.13: Use Case Specification: Create, Configure, and Publish Event (UC0012)

Use Case ID	UC0013
--------------------	---------------

Use Case Name	Rule-Based Categorization Module
Description	Automatically assigns tags and categories to an event based on rules applied to the event description.
Trigger	An Organizer saves or updates the event description (Step 6 of UC0011).
Actors	System (Internal Module)
Pre-conditions	The event description text is submitted or updated. Rule-based categorization is configured.
Main Flow	System: 1. Receives event description text. 2. Applies configured rules (e.g., keyword matching, NLP checks). 3. Assigns tags/categories to the event. 4. Saves the assigned tags/categories to the database.
Alternative Flow	AF1: Low Confidence Tagging Condition: The system cannot confidently assign a required tag based on the description. Steps: 1. System flags the tag as low confidence or pending review. 2. System notifies the Organizer. 3. Organizer manually reviews and approves/modifies the tag.
Exception Conditions	Categorization logic error.
Post-condition	The event has automatically assigned tags/categories.

Table 3.14: Use Case Specification: Rule-Based Categorization Module (UC0013)

Use Case ID	UC0014	
Use Case Name	Rule-Based Content-Based Filtering	
Description	The system suggests relevant hackathons to Participants based on their profile, history, or bookmarked items.	
Trigger	Participant opens the “Recommended” or “Discover” section.	
Actors	Participant, System (Internal Module)	
Pre-conditions	Participant is logged in. System has relevant event and profile data.	
Main Flow	Participant: 1. Opens the discovery page.	System: 2. Analyzes Participant data (skills, interests, past participation, bookmarked Organizers). 3. Applies content-based filtering rules to match Participant data with event tags, themes, and Organizer information. 4. Displays a list of recommended events.

Alternative Flow	AF1: Insufficient Profile Data Condition: Participant's profile lacks sufficient data (e.g., skills, interests) for quality matching. Steps: 1. System uses general popular events. 2. System displays a prompt to the Participant: “Update your profile for better recommendations.” AF2: No Matching Events Condition: The filtering criteria result in zero recommended events. Steps: 1. System displays: “No specific recommendations found. Displaying popular or featured events instead.”
Exception Conditions	System error during recommendation generation.
Post-condition	A list of recommended events is displayed to the Participant.

Table 3.15: Use Case Specification: Rule-Based Content-Based Filtering (UC0014)

Use Case ID	UC0015
Use Case Name	Team Creation and Management
Description	Allows Participants to create a new team, invite/remove members, and manage team details.
Trigger	The Participant selects “Create Team” or opens the “Team Management” section.

Actors	Participant	
Pre-conditions	Participant is registered for an event.	
Main Flow	Participant: 1. Selects "Team Management." 2. Creates a new team (as captain). 3. Enters team details (name, event). 4. Invites other Participants.	System: 5. Validates team details. 6. Records the new team and sends invitations. 7. Displays confirmation.
Alternative Flow	AF1: Team Size Limit Reached Condition: Team Captain attempts to invite a member, but the team is full. Steps: 1. System detects the team is at max capacity. 2. System displays: "Team size limit reached (X members)." 3. Invitation is blocked. AF2: Invited Member Rejects/Ignores Invite Condition: An invited Participant rejects or ignores the invitation. Steps: 1. System notifies the Team Captain of the rejection or expiration of the invite. 2. The Captain may invite another member. AF3: Participant Leaves Team Condition: A Participant chooses to leave the team. Steps: 1. Participant confirms leaving. 2. System removes the Participant from the team roster. 3. System notifies the Team Captain.	

Exception Conditions	Database error during team update.
Post-condition	A new team is created, or an existing team's membership/details are updated.

Table 3.16: Use Case Specification: Team Creation and Management (UC0015)

Use Case ID	UC0016	
Use Case Name	Skill-Based and Interest-Based Matching	
Description	Provides potential team members to Participants based on skill sets and interests for team formation.	
Trigger	The Participant opens the “Team Matching” or “Find Members” feature.	
Actors	Participant, System (Internal Module)	
Pre-conditions	Participant is logged in and not yet on a full team.	
Main Flow	Participant: 1. Opens the “Find Members” feature.	System: 2. Analyzes Participant profile (skills, interests). 3. Matches with other Participants seeking teams based on complementary criteria. 4. Displays a list of suggested potential teammates.

Alternative Flow	AF1: No Suitable Matches Found Condition: No Participants with complementary skills/interests are currently seeking a team. Steps: 1. System displays: “No suitable matches found at this time. Try again later or broaden your search criteria.” AF2: Participant Not Seeking Team Condition: A suggested match is already on a full team or has set their status to "Not Seeking Team." Steps: 1. System automatically filters out unavailable Participants from the suggestions list.
Exception Conditions	Matching algorithm error.
Post-condition	A list of suggested potential teammates is displayed.

Table 3.17: Use Case Specification: Skill-Based and Interest-Based Matching (UC0016)

Use Case ID	UC0017
Use Case Name	Mentor-Participant Interaction and Logging
Description	Provides a coordination mechanism for Participants to seek guidance and a dedicated interface for Mentors to log team interaction details and provide non-scoring performance input.
Trigger	A Participant initiates contact or a Mentor accesses the logging dashboard.

Actors	Participant, Mentor		
Pre-conditions	Both Participant and Mentor are authenticated and associated with the same active event.		
Main Flow	Participant: 1. Selects a Mentor to contact for technical or logistical guidance. 2. Sends a message or requests a meeting.	Mentor: 3. Receives the request and interacts with the team. 4. Accesses the dedicated interaction logging interface. 5. Enters interaction details (duration, topic) and non-scoring performance input regarding the team's progress.	System: 6. Persists the log and links it to the team's profile.

Alternative Flow	AF1: Mentor Logs Interaction Condition: Mentor initiates logging of an interaction that occurred outside the in-app coordination mechanism. Steps: Follows Steps 4–6 of the Main Flow. AF2: Mentor Unavailable Condition: Selected Mentor is offline or has reached their daily interaction limit. Steps: 1. System displays: “Mentor is currently unavailable. Try another mentor or send an asynchronous message.”
Exception Conditions	Data persistence error; Unauthorized access to the log interface.
Post-condition	The interaction is facilitated, and a verifiable record of the team's mentorship history is stored.

Table 3.18: Use Case Specification: Mentor-Participant Interaction and Logging (UC0017)

Use Case ID	UC0018
Use Case Name	Discussion Boards and Rating Systems
Description	Allows users to participate in discussion boards and submit post-event ratings (text-only data persistence).
Trigger	The user selects the “Discussion Board” or submits a post-event rating.
Actors	Participant, Organizer, Judge, Mentor, Administrator

Pre-conditions	User is logged in. Event must be active or completed (for rating).	
Main Flow	User: 1. Navigates to the relevant discussion board or rating page. 2. Submits a post, comment, or rating score/text.	System: 3. Validates and stores the post/comment/rating (text-only persistence). 4. Displays the new content to relevant users.
Alternative Flow	AF1: Content Moderation Required Condition: User submits content containing flagged keywords (e.g., profanity). Steps: 1. System detects flagged content. 2. System displays: “Your post is under review.” 3. System temporarily hides the post until reviewed by an Administrator/Organizer. AF2: Rating Period Closed Condition: User attempts to submit a post-event rating after the deadline. Steps: 1. System detects the deadline is passed. 2. System displays: “The rating period for this event is closed.” 3. Submission is blocked.	
Exception Conditions	Database error during data storage.	
Post-condition	Discussion content or rating is successfully submitted and persisted.	

Table 3.19: Use Case Specification: Discussion Boards and Rating Systems (UC0018)

Use Case ID	UC0019	
Use Case Name	Formal Participant Feedback	
Description	Provides a structured mechanism for Participants to give formal feedback on event organization, rules, and judging.	
Trigger	The Participant selects “Submit Feedback” after the event completion.	
Actors	Participant	
Pre-conditions	Participant is logged in. The event is complete.	
Main Flow	Participant: 1. Selects “Submit Feedback.” 2. Fills out the structured feedback form (on organization, rules, judging). 3. Submits the form.	System: 4. Validates and stores the structured feedback data. 5. Notifies the Organizer.
Alternative Flow	AF1: Missing Mandatory Sections Condition: Participant attempts to submit the form without completing all mandatory sections. Steps: 1. System detects incomplete sections.	

	2. System displays an error message. 3. Participant completes the sections, returning to Step 3 of Main Flow. AF2: Already Submitted Feedback Condition: Participant attempts to submit feedback for the same event a second time. Steps: 1. System detects previous submission. 2. System displays: “You have already submitted feedback for this event.” 3. Submission is blocked.
Exception Conditions	Database error during data storage.
Post-condition	Structured feedback from the Participant is successfully submitted and stored.

Table 3.20: Use Case Specification: Formal Participant Feedback (UC0019)

Use Case ID	UC0020
Use Case Name	Secure Project Submissions
Description	Provides a secure interface for a Participant (on behalf of a team) to upload and submit their hackathon project.
Trigger	The Participant selects “Submit Project” on the submission deadline page.
Actors	Participant

Pre-conditions	Participant is logged in and belongs to a team. Submission window is open.	
Main Flow	Participant: 1. Selects “Submit Project.” 2. Uploads project files/links and enters submission metadata. 3. Confirms submission.	System: 4. Secures and stores the submission files/metadata. 5. Automatically links the submission to the correct team and its members. 6. Displays a confirmation message.
Alternative Flow	AF1: Submission Deadline Passed Condition: Participant attempts to submit after the critical deadline. Steps: 1. System checks the deadline. 2. If passed, system displays: “Submission deadline has passed. Submission is locked.” 3. Submission is blocked. AF2: File Upload Failure Condition: A required file fails to upload due to network interruption or size limit. Steps: 1. System detects file upload failure. 2. System displays an error message. 3. Participant retries the upload, returning to Step 2 of Main Flow. AF3: Update Submission Condition: Participant submits a revised version before the deadline. Steps: 1. System archives the previous version's metadata and stores the new submission as the current active version, completing Main Flow.	

Exception Conditions	File upload failure or deadline is passed.
Post-condition	Project submission is securely stored and correctly linked to the team.

Table 3.21: Use Case Specification: Secure Project Submissions (UC0020)

Use Case ID	UC0021	
Use Case Name	Permission-Based Judging Dashboard	
Description	Provides Judges with a secure dashboard to input scores and comments based on configurable criteria.	
Trigger	The Judge logs in during the judging period.	
Actors	Judge	
Pre-conditions	Judge is logged in. The judging period is open. Projects have been submitted.	
Main Flow	Judge: 1. Logs in and accesses the Judging Dashboard. 2. Selects a project to evaluate. 4. Inputs scores and comments for each criterion. 5. Submits the evaluation.	System: 3. Displays project details and the configurable scoring criteria/form. 6. Validates and securely stores the scores and comments.

Alternative Flow	AF1: Scoring Criteria Not Loaded Condition: The system fails to retrieve the judging criteria for the selected event. Steps: 1. System displays: “Scoring criteria unavailable. Please try again.” 2. Judge is prevented from scoring until the criteria are available. AF2: Judge Has Already Scored Condition: Judge selects a project they have already submitted final scores for. Steps: 1. System displays the previous score and feedback. 2. System displays: “You have already scored this project.” 3. Judge may view but cannot resubmit unless explicitly permitted by an Administrator/Organizer.
Exception Conditions	Data submission failure.
Post-condition	Judge's evaluation (scores and comments) is securely stored for the selected project.

Table 3.22: Use Case Specification: Permission-Based Judging Dashboard (UC0021)

Use Case ID	UC0022
Use Case Name	Automatic Score Aggregation and Normalization
Description	Automatically calculates and normalizes judge scores, applying defined scoring rules (e.g., dropping high/low scores).

Trigger	A Judge submits a final score (UC0020), or the judging deadline is met.
Actors	System (Internal Module)
Pre-conditions	Multiple judge scores exist for a submission. Scoring rules are defined.
Main Flow	System: 1. Retrieves all scores for a submission. 2. Applies defined scoring rules (e.g., drops the highest and lowest scores). 3. Calculates the final aggregated and normalized score. 4. Stores the final score.
Alternative Flow	AF1: Insufficient Scores for Normalization Condition: The number of scores submitted is less than the required minimum for normalization rules (e.g., dropping high/low scores). Steps: 1. System bypasses the normalization step. 2. System calculates the simple average. 3. System logs a warning for the Organizer/Administrator.
Exception Conditions	Calculation error.
Post-condition	The final aggregated and normalized score for a project is calculated and stored.

Table 3.23: Use Case Specification: Automatic Score Aggregation and Normalization (UC0022)

Use Case ID	UC0023
--------------------	--------

Use Case Name	Transparent Judging Scoreboard	
Description	Displays the real-time or final judging scoreboard accessible to Participants.	
Trigger	The Participant opens the “Scoreboard” section.	
Actors	Participant	
Pre-conditions	Judging is underway or complete. The scoreboard is set to be visible by the Organizer.	
Main Flow	Participant: 1. Opens the “Scoreboard” section.	System: 2. Retrieves the latest aggregated score data. 3. Displays the transparent scoreboard (ranking, final score, etc.) to the Participant.
Alternative Flow	AF1: Scoreboard Visibility Restricted Condition: The Organizer has set the scoreboard to be private or released only after a certain date/time. Steps: 1. System checks visibility settings. 2. System displays: “Scoreboard is currently private. Check back after [Date/Time].” AF2: Scoreboard Data Lag Condition: The latest scores have not yet been aggregated or normalized. Steps: 1. System retrieves the available data.	

	2. System displays a warning: “Scores are currently being processed. Displayed data may be slightly delayed.”
Exception Conditions	Database retrieval error.
Post-condition	The current or final judging scoreboard is successfully displayed.

Table 3.24: Use Case Specification: Transparent Judging Scoreboard (UC0023)

Use Case ID	UC0024
Use Case Name	Deliver Structured Judge Feedback
Description	Provides a mechanism for Judges to deliver structured, written feedback to participating teams, made accessible to Participants.
Trigger	The judging period is complete, and the Organizer activates feedback release.
Actors	Judge, Participant
Pre-conditions	Structured feedback from the Judge is submitted and stored (UC0020).

Main Flow	Organizer/System: 1. Initiates the release of Judge feedback.	System: 2. Retrieves the stored written feedback for a team. 3. Makes the structured feedback available for the Participant to view on their team/submission dashboard.	Participant: 4. Views the structured feedback.
Alternative Flow	AF1: Judge Provided No Feedback Condition: A submission received scores but the Judge did not input written feedback. Steps: 1. System displays a message to the Participant: “No written feedback was provided by the Judge for this submission.” AF2: Feedback Release Delayed Condition: The Organizer attempts to release feedback, but the system prevents it due to a mandatory time lock. Steps: 1. System displays: “Feedback release is locked until all final results are published.”		
Exception Conditions	Database retrieval error.		
Post-condition	Structured Judge feedback is successfully delivered to the participating team.		

Table 3.25: Use Case Specification: Deliver Structured Judge Feedback (UC0024)

Use Case ID	UC0025	
Use Case Name	Bilingual Interface Support	
Description	Allows the user to switch the display language for static content only.	
Trigger	The user selects the language switcher option.	
Actors	Participant, Organizer, Judge, Administrator, Mentor	
Pre-conditions	The interface is currently displayed in one language.	
Main Flow	User: 1. Selects the language switcher (e.g., Amharic/English).	System: 2. Reloads the static content of the page using the selected language assets.
Alternative Flow	AF1: Dynamic Content Not Translated Condition: The user views content generated by users (e.g., discussion posts, event descriptions). Steps: 1. System displays user-generated content in the original language.	

	2. System displays a note: “Dynamic content is not translated.” AF2: Language Asset Missing Condition: The system fails to load the correct translation asset for a static content element. Steps: 1. System defaults the missing text block to the secondary language (e.g., English). 2. System logs a translation error for the Administrator.
Exception Conditions	Language asset file missing.
Post-condition	The interface is displayed in the user’s selected language for static content.

Table 3.26: Use Case Specification: Bilingual Interface Support (UC0025)

Use Case ID	UC0026
Use Case Name	Tiered Notifications and Reminders
Description	Delivers real-time and in-app notifications and reminders based on hard-coded rules for critical deadlines.
Trigger	The Timer / Clock reaches a critical deadline defined by hard-coded rules (e.g., 24 hours before submission closes).
Actors	Timer / Clock, System
Pre-conditions	The event has critical deadlines. Users are registered for the event.

Main Flow	Timer / Clock: 1. Triggers the notification rule.	System: 2. Generates the corresponding tiered notification/reminder message. 3. Sends the real-time and in-app notification to the relevant user group (e.g., all registered Participants).
Alternative Flow	AF1: User Opt-Out Condition: The user has disabled certain types of notifications in their settings. Steps: 1. System checks user preferences. 2. System prevents real-time delivery (e.g., email/SMS) but still displays the in-app notification. AF2: Notification Failure Condition: The real-time delivery mechanism (e.g., email server) fails. Steps: 1. System logs the failure and attempts retry within a short period. 2. System confirms the in-app notification was successfully stored.	
Exception Conditions	Notification delivery failure.	
Post-condition	Relevant users receive a tiered notification/reminder about a critical deadline.	

Table 3.27: Use Case Specification: Tiered Notifications and Reminders (UC0026)

Use Case ID	UC0027
--------------------	--------

Use Case Name	View and Manage Earned Badges/Certificates	
Description	Notifies Participants of issued awards and provides a dedicated interface to view and download digital post-event certificates or badges.	
Trigger	The System (via Timer/Clock) or Administrator/Organizer confirms the final results and issues recognition.	
Actors	Participant, Timer/Clock, Administrator, Organizer	
Pre-conditions	The hackathon has concluded, and the evaluation/scoring phase is complete.	
Main Flow	System: 1. Triggers the issuance of badges/certificates based on event completion or ranking. 2. Sends a real-time notification to the Participant. 6. Generates and serves the secure digital file.	Participant: 3. Navigates to their "Achievements" or "Portfolio" section. 4. Views the specific details of the earned badge or certificate. 5. Selects the "Download" option to save a high-resolution digital copy.
Alternative Flow	AF1: Notification Link Failure Condition: The link within the notification to the badge/certificate repository fails to resolve. Steps: 1. System logs the link failure. 2. System attempts to correct the link and sends a follow-up notification with the corrected link. AF2: Bulk Issuance by Administrator	

	Condition: The Administrator or Organizer manually initiates the recognition process for a group of participants instead of waiting for a system-timed trigger. Steps: 1. Administrator selects a specific list of winners or eligible participants from the event management dashboard. 2. Administrator triggers the “Bulk Issuance” command. 3. System validates the completion status of the selected participants. 4. System executes the Main Flow (Steps 1–6) for every selected user to generate badges/certificates and send individual notifications.
Exception Conditions	Notification delivery failure; PDF/Image generation error.
Post-condition	The Participant has received notification and successfully accessed/archived their digital recognition.

Table 3.28: Use Case Specification: View and Manage Earned Badges/Certificates (UC0027)

Use Case ID	UC0028
Use Case Name	Organizer Dedicated Analytics Dashboard
Description	Enables the Organizer to view event-specific metrics and analytics (registration rates, team sizes, judge performance metrics, etc.).
Trigger	The Organizer opens the “Analytics” tab for a specific event.
Actors	Organizer
Pre-conditions	Organizer is logged in and associated with the event.

Main Flow	Organizer: 1. Opens the “Analytics” dashboard.	System: 2. Fetches event-specific data (usage, registration, submission, judge performance metrics). 3. Displays the metrics in the dashboard view.
Alternative Flow	AF1: Data Retrieval Timeout Condition: The system takes longer than the acceptable threshold to generate complex metrics. Steps: 1. System displays a loading indicator. 2. After timeout, system displays: “Report generation timed out. Try again later or use the simpler report generation.”	
Exception Conditions	Database retrieval error.	
Post-condition	Event-specific analytics are successfully displayed to the Organizer.	

Table 3.29: Use Case Specification: Organizer Dedicated Analytics Dashboard (UC0028)

Use Case ID	UC0029
Use Case Name	Generate and Export Reports
Description	Allows the Organizer or Administrator to generate and export reports on key event or system metrics.

Trigger	The user selects “Generate Report” or “Export Data.”	
Actors	Organizer, Administrator	
Pre-conditions	Monitoring data is available.	
Main Flow	User: 1. Selects report type/criteria (usage, registration, submission, judge performance). 2. Selects “Generate” or “Export.”	System: 3. Compiles relevant data based on criteria. 4. Generates the report/export file. 5. Makes the report/export available for display/download.
Alternative Flow	AF1: Cancel Report Generation Condition: User decides not to generate the report. Steps: 1. The user clicks “Cancel” before the report is generated. 2. System aborts the process and returns to the previous page or dashboard. AF2: Missing or Incomplete Data Condition: The selected report criteria yields zero or incomplete data. Steps: 1. System displays a warning: “Report generated but data is incomplete or empty based on current criteria.”	

	AF3: Export File Format Failure Condition: The system fails to correctly format the export file (e.g., CSV, PDF). Steps: 1. System logs the error. 2. System displays: “File export failed. Please try a different format.”
Exception Conditions	Missing or incomplete data for the requested report.
Post-condition	The requested report is generated and made available to the user.

Table 3.30: Use Case Specification: Generate and Export Reports (UC0029)

Use Case ID	UC0030
Use Case Name	Administrator System Monitoring and Governance
Description	A privileged user manages and monitors the entire HackET system, overseeing accounts, governing core application utilities, and provisioning secondary Administrator accounts to ensure platform continuity.
Trigger	Administrator opens the “Monitoring Dashboard” or “System Settings.”
Actors	Administrator (Root or Existing)
Pre-conditions	Administrator is logged in with high-level system privileges (initially set up via secure server-side provisioning).

Main Flow	Administrator: 1. Opens the monitoring dashboard. 3. Administrator reviews system-wide audit logs to ensure security compliance across all hackathon events.	System: 2. Fetches platform-wide logs, cross-tenant user accounts, and displays real-time usage metrics and performance data.
Alternative Flow	AF1: Provision Secondary Administrator Condition: The existing Admin needs to delegate platform-wide governance to another trusted user. Steps: 1. Administrator selects "Add New Administrator." 2. Administrator enters the email and credentials for the new privileged user. 3. System generates a secure, encrypted invitation/activation link. 4. System logs this creation event for security auditing. AF2: Account Suspension/Management Condition: Administrator identifies a user or organizer account for disciplinary action. Steps: 1. Administrator selects a specific account from the global user list. 2. Administrator modifies the account status (e.g., Suspend, Deactivate). 3. System validates and implements the change, immediately revoking the target's session tokens. AF3: Filter/Export Logs Condition: Administrator needs to perform a deep audit or export data for reporting.	

	Steps: 1. Administrator selects filters (date range, user type, or event category). 2. System generates an export file (PDF/CSV) for download.
Exception Conditions	Database Connection Failure; Privilege Conflict.
Post-condition	Platform-wide governance is maintained, and new administrative roles are successfully created and logged.

Table 3.31: Use Case Specification: Administrator System Monitoring and Governance (UC0030)

Use Case ID	UC0031
Use Case Name	Archive Data
Description	Performs data persistence by archiving data for later retrieval.
Trigger	The Administrator initiates the archiving process, or an event is automatically marked for archiving.
Actors	Administrator, System (Internal Module)
Pre-conditions	Data for a completed event is ready for archiving.

Main Flow	Administrator/System: 1. Initiates archiving process.	System: 2. Selects non-archived data. 3. Moves/copies the selected data to the archival storage. 4. Marks the data as archived in the main database.
Alternative Flow	AF1: Partial Archiving Failure Condition: A portion of the data fails to move to archival storage. Steps: 1. System logs the failure for the specific data block. 2. System attempts to retry the failed block. 3. System notifies the Administrator of the partial success/failure.	
Exception Conditions	Database or storage write failure.	
Post-condition	Event metadata and text-only discussion data are successfully archived.	

Table 3.32: Use Case Specification: Archive Data (UC0031)

Use Case ID	UC0032
Use Case Name	Basic Search and Retrievability of Data
Description	Allows users to perform a basic search and retrieve current and archived metadata only.
Trigger	The user performs a search action.
Actors	Participant, Organizer, Judge, Administrator, Mentor, Public User

Pre-conditions	Data (current or archived metadata) exists.	
Main Flow	User: 1. Enters a search query.	System: 2. Queries the current and archived metadata databases. 3. Retrieves and displays the relevant metadata and links.
Alternative Flow	AF1: Search No Results Condition: Search yields no results in either current or archived databases. Steps: 1. System displays: “No matching results found.”	
	AF2: Archived Data Retrieval Delay Condition: Retrieving data from the archived storage takes longer than normal. Steps: 1. System displays a warning: “Retrieving archived results, may take a few moments.”	
Exception Conditions	Search index failure.	
Post-condition	Relevant current or archived metadata is successfully retrieved and displayed.	

Table 3.33: Use Case Specification: Basic Search and Retrievalability of Data (UC0032)

Use Case ID	UC0033
--------------------	--------

Use Case Name	Manage Event Staff and Permissions		
Description	Enables the Lead Organizer to onboard event experts (Judges, Mentors) via secure invitation and manage their specific access levels (e.g., Technical Lead, Logistics Staff) within the event ecosystem.		
Trigger	The Lead Organizer accesses the “Staff Management” or “Permissions” dashboard.		
Actors	Organizer (Lead), Invitee (Organizer (Staff)/Judge/Mentor)		
Pre-conditions	The Lead Organizer is authenticated and the specific event is active in the multi-tenant dashboard.		
Main Flow	Lead Organizer: 1. Navigates to the Staff Management section. 2. Initiates an invitation by entering the staff member's email address. 3. Selects the specific Permission Level or Role (e.g., Judge, Mentor, Technical	System: 4. Generates a secure, one-time invitation token. 5. Sends an automated email invitation to the recipient. 7. Confirms the account and grants access to the specific event dashboard based on the pre-assigned permissions.	Invitee: 6. Clicks the secure link, completes their profile, and sets their credentials.

	Lead) for that individual.		
Alternative Flow	AF1: Revoke or Modify Existing Permissions Condition: The Lead Organizer needs to change the status of currently active staff. Steps: 1. Lead Organizer selects an existing staff member from the list. 2. Lead Organizer updates the role or selects "Revoke Access." 3. System updates the permissions in the database immediately. AF2: Attempt to Modify Own Permissions Condition: The Lead Organizer attempts to remove their own Lead status. Steps: 1. System detects the self-modification attempt. 2. System displays: "Cannot modify your own lead role. Assign another Lead first." 3. Action is blocked.		
Exception Conditions	Invalid Email Format; Token Expiration.		
Post-condition	The staff member is successfully onboarded or their permissions are updated, ensuring secure access to event utilities.		

Table 3.34: Use Case Specification: Manage Event Staff and Permissions (UC0033)

3.3.4. Dynamic Models of a System

A dynamic model of the HackET system illustrates how different components interact and evolve over time in response to user inputs and system events.

3.3.4.1. Sequence Diagrams

A sequence diagram represents the interaction between system components and actors over time, emphasizing the order of message exchange in response to a given scenario. It helps visualize how various system entities communicate to complete a task, ensuring that system operations are logically ordered and synchronized.

Below are the key sequence diagrams developed for the HackET system.

1. Sequence Diagram for “Secure Project Submissions”

This diagram shows how a participant selects the submission option, how the system verifies the submission window and secures the uploaded project files, and how the submission is automatically linked to the correct team and its members.

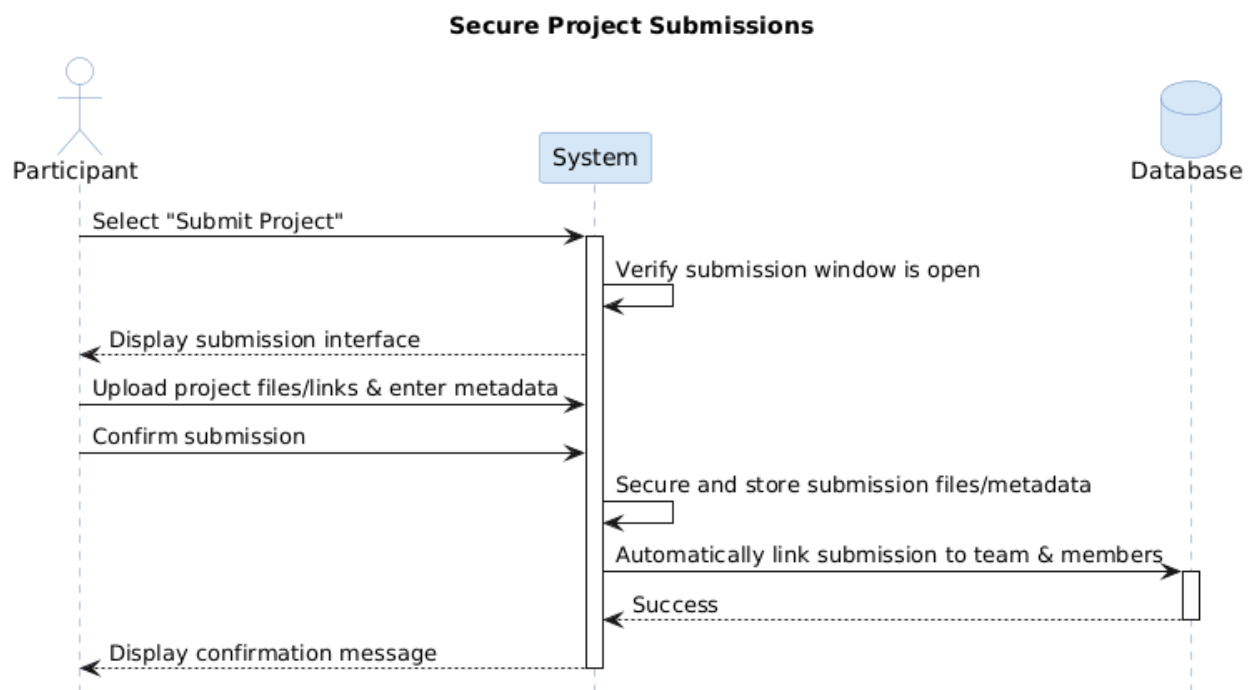


Figure 3.2: Sequence Diagram for Secure Project Submissions

2. Sequence Diagram for “Automatic Score Aggregation and Normalization”

This diagram shows how a judge submits a final score, how the Internal Scoring Module retrieves all project scores from the database, and how the system applies defined normalization rules to calculate and store the final aggregated result.

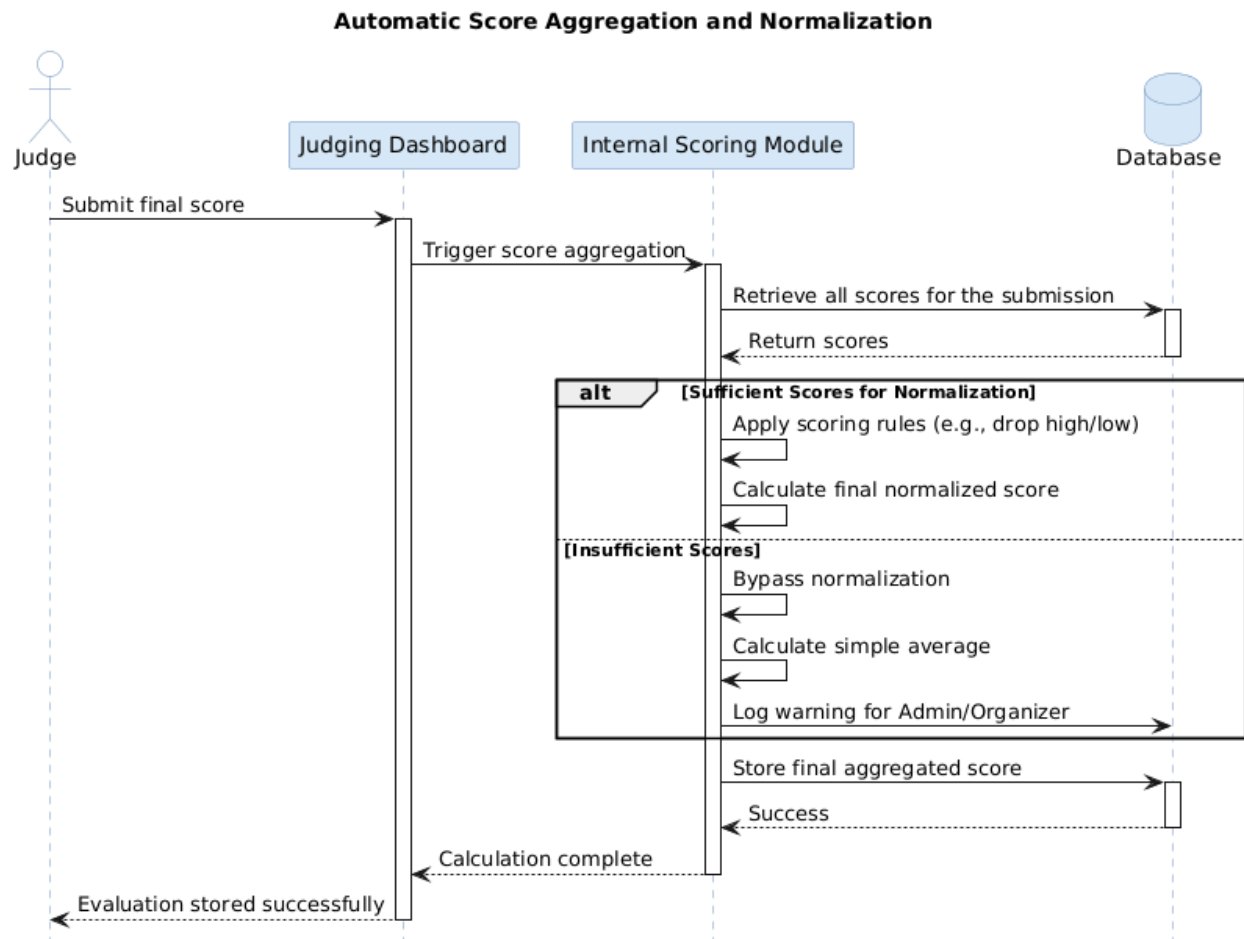


Figure 3.3: Sequence Diagram for Automatic Score Aggregation and Normalization

3. Sequence Diagram for “Automatic Session Expiration”

This diagram shows how the timer / clock signals an inactivity timeout, how the system clears temporary data for Organizer or Judge sessions, and how the request is finalized by redirecting the user to the login screen with an expiration notification.

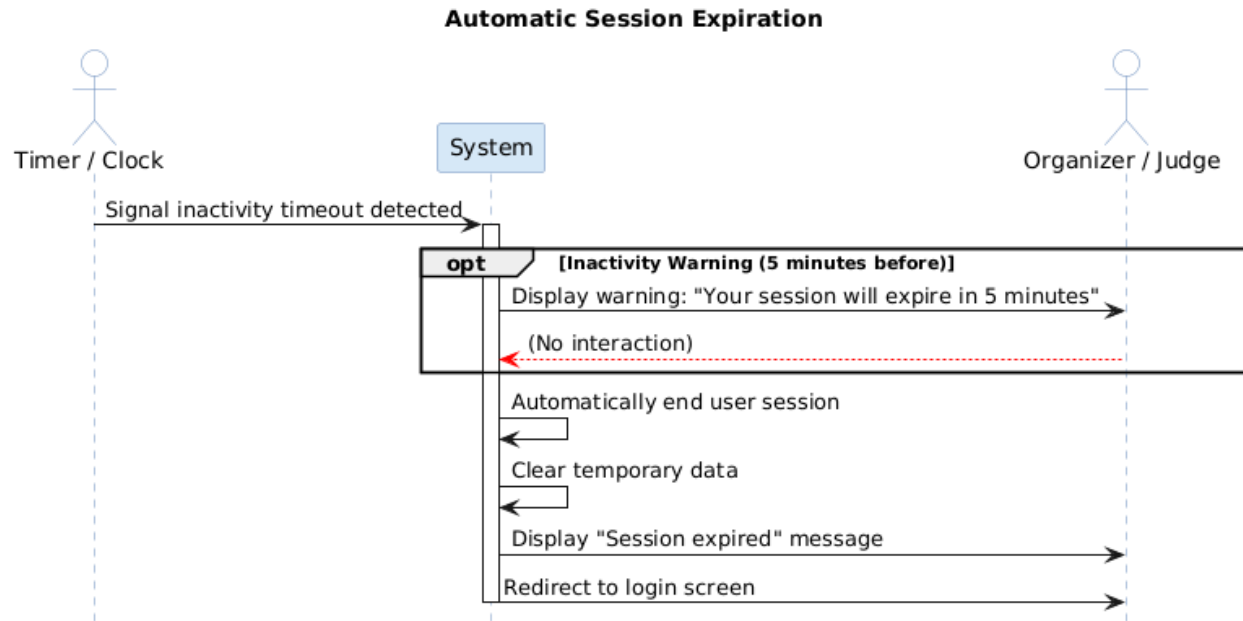


Figure 3.4: Sequence Diagram for Automatic Session Expiration

3.3.4.2. State Machine Diagrams

A state machine diagram models how an object transitions between different states in response to specific events. It helps visualize the system’s dynamic behavior and ensures that every possible event leads to a well-defined state transition.

Below are the key state machine diagrams developed for the HackET system.

1. State Machine Diagram for “Hackathon Event Life Cycle”

This diagram follows the progression from event creation by the Organizer to automatic triggers by the Timer / Clock and eventual archiving by the Administrator or System.

Hackathon Event Life Cycle

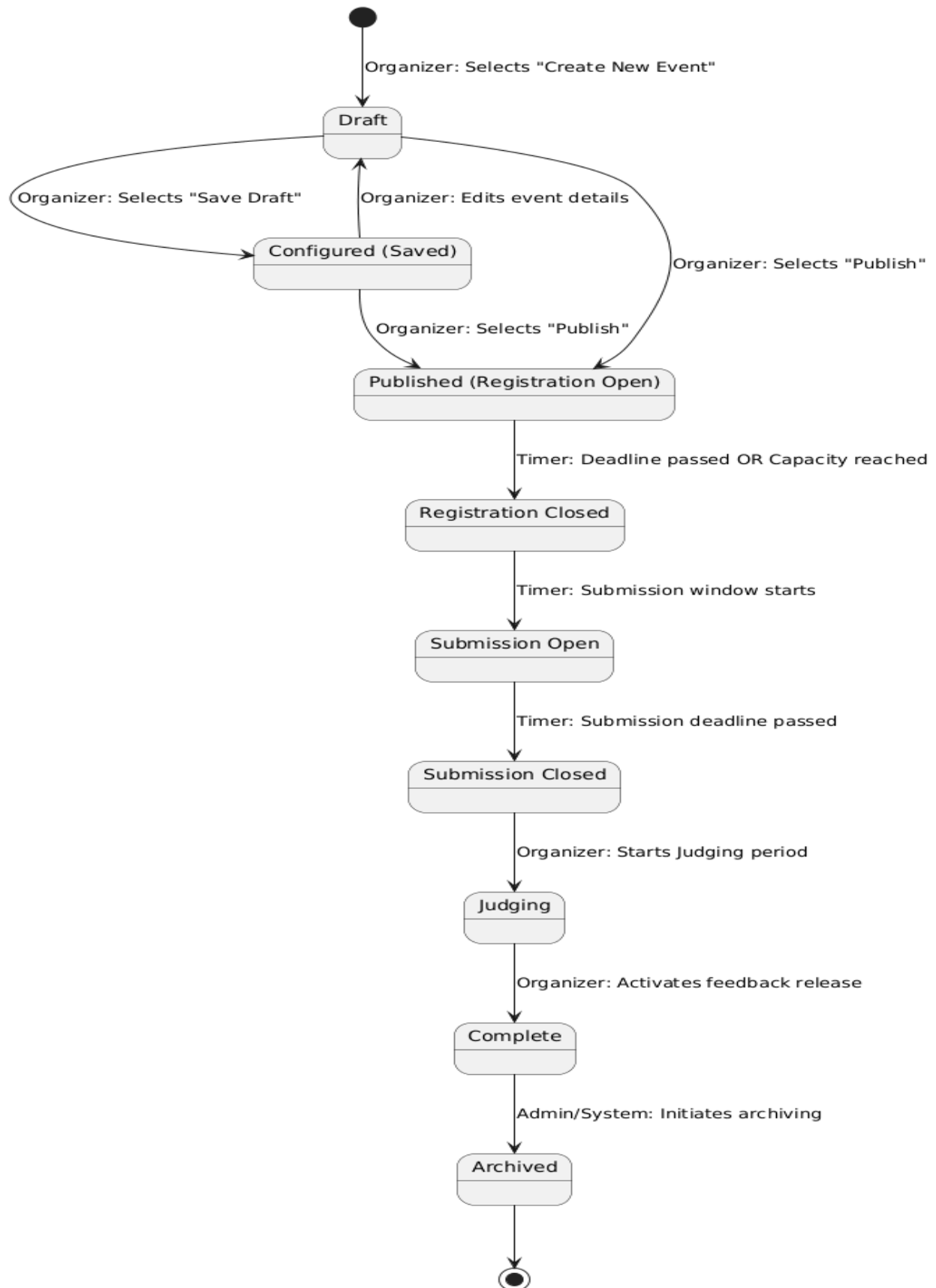


Figure 3.5: State Machine Diagram for Hackathon Event Life Cycle

State	Description
Draft	The initial stage where an Organizer has started entering event details but has not yet saved or finalized them.
Configured (Saved)	The event details are saved in the database but are not yet visible to the public or potential participants.
Published (Registration Open)	The event is live and visible in the Event Discovery section; participants can now register for the hackathon.
Registration Closed	The period for new participants to join has ended, triggered either by reaching a deadline or the event's maximum capacity.
Submission Open	The window is open for registered teams or individuals to upload their project files and documentation.
Submission Closed	The deadline for project uploads has passed; the system no longer accepts new submissions.
Judging	Judges evaluate the submissions based on set criteria; the Organizer manages the scoring and feedback process.
Complete	The hackathon has ended, winners have been declared, and the Organizer has activated the release of results and feedback.

Archived	The final state where event metadata and text-only discussion data are moved to long-term storage for historical reference.
-----------------	---

Table 3.35: Explanation of state for Hackathon Event Life Cycle

2. State Machine Diagram for “Participant Account Status”

This diagram focuses on the lifecycle of a user account, covering registration, authentication, and administrative oversight.

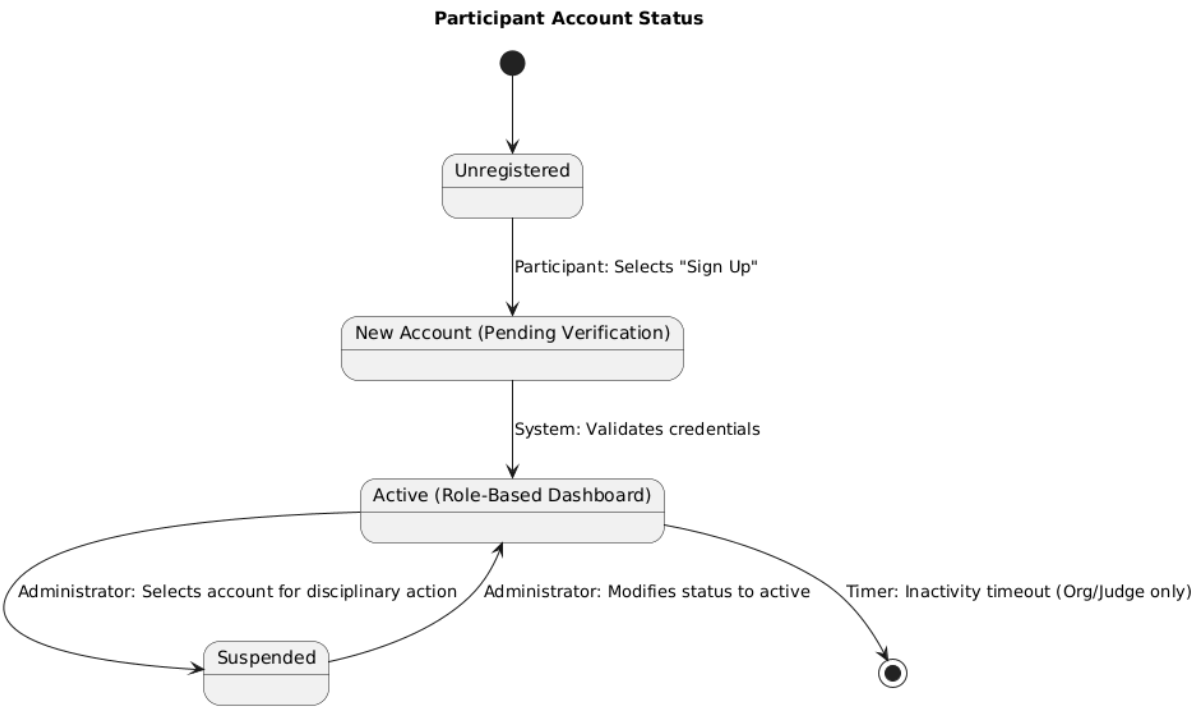


Figure 3.6: State Machine Diagram for Participant Account Status

State	Description
-------	-------------

Unregistered	A visitor who has not yet created an account but may be browsing public event listings.
New Account (Pending Verification)	The user has submitted their registration details; the account exists but is waiting for email or credential validation.
Active (Role-Based Dashboard)	The account is fully verified. Upon login, the user is directed to a dashboard tailored to their specific role (Participant, Organizer, or Judge).
Suspended	An Administrator has temporarily disabled the account due to disciplinary reasons or violation of platform policies.

Table 3.36: Explanation of state for Participant Account Status

3.3.4.3. Activity Diagrams

Activity diagrams represent the workflow of the system, showing decision points, parallel processes, and the flow of control from one activity to another. They provide a visual representation of the business and operational workflows that define the HackET platform.

Below are the key activity diagrams that illustrate the main processes within the system.

1. Activity Diagram for “Create, Configure, and Publish Event”

This diagram models the workflow where an Organizer defines event details and criteria, while the System performs validation checks and manages the transition between "Draft" and "Published" states based on data completeness and naming rules.

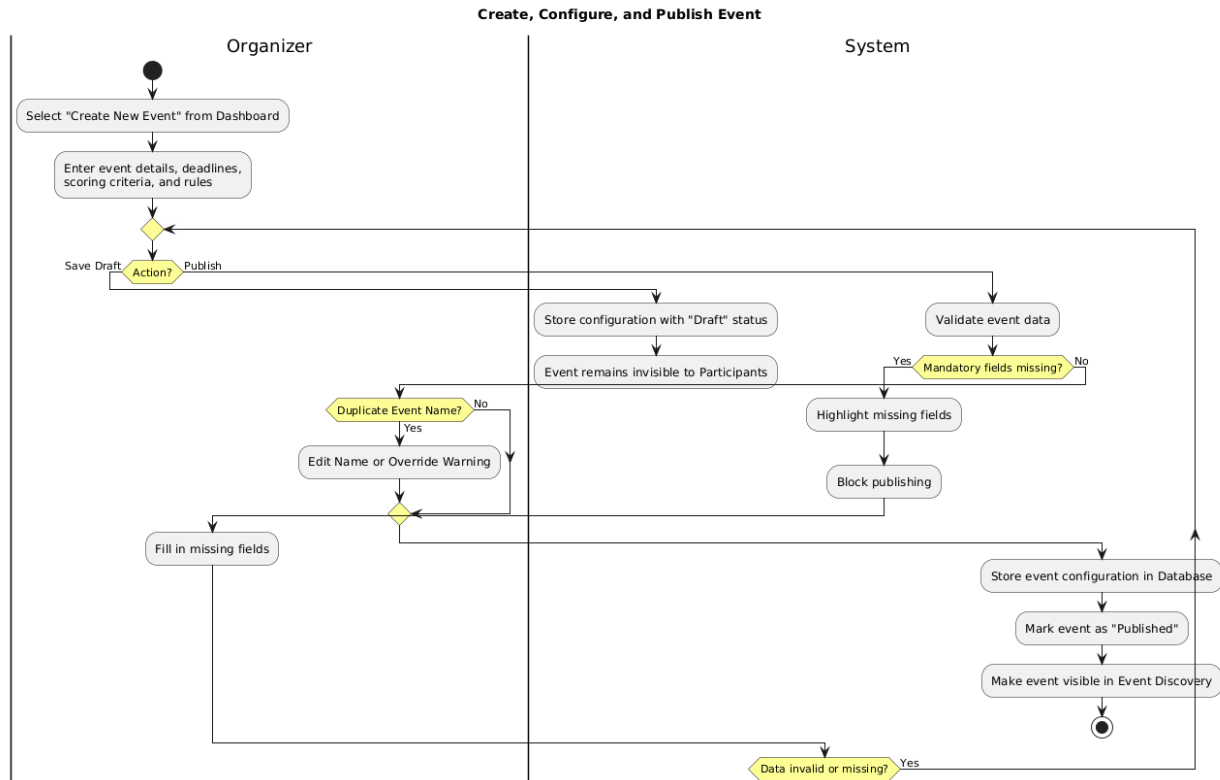


Figure 3.7: Activity Diagram for Create, Configure, and Publish Event

2. Activity Diagram for “Team Creation and Matching Workflow”

This diagram illustrates the process where a Participant manages team formation and invitations while the System's Internal Module concurrently analyzes profile data to suggest compatible teammates based on skills and interests.

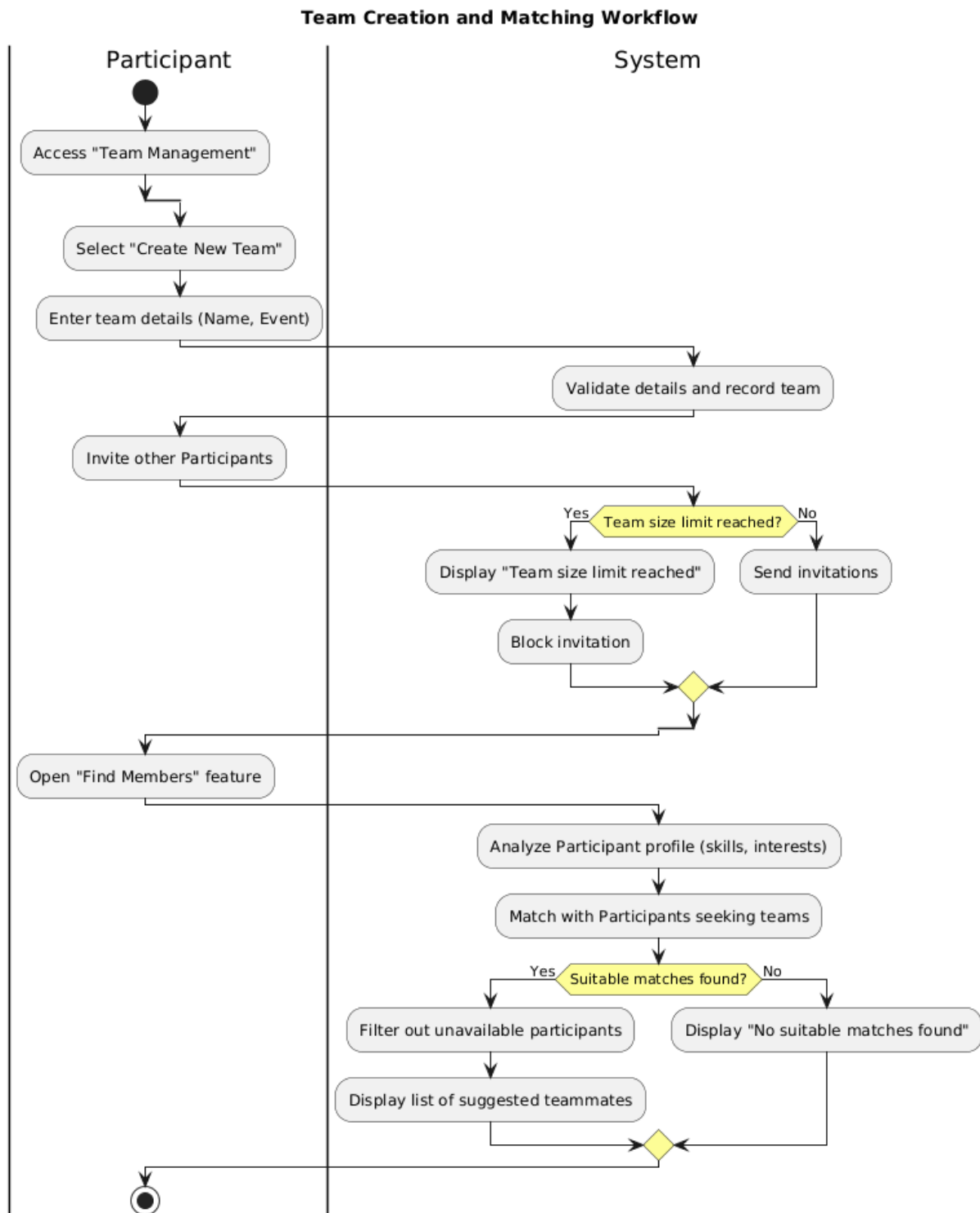


Figure 3.8: Activity Diagram for Team Creation and Matching Workflow

3. Activity Diagram for "Register Account / Sign Up"

This diagram models the onboarding flow for Participants and Organizers, tracing the path from initial data entry through multiple system validation loops for email uniqueness, password strength, and field completion.

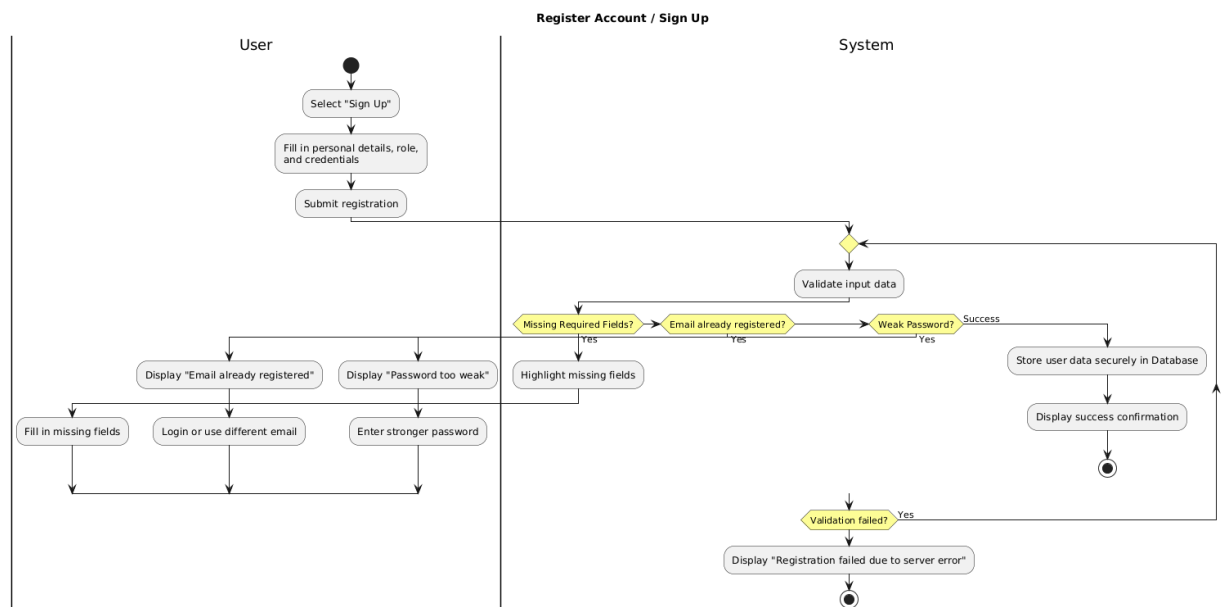


Figure 3.9: Activity Diagram for Register Account / Sign Up

3.3.4.4. Collaboration Diagrams

Collaboration diagrams (also known as communication diagrams) illustrate how objects interact with each other to accomplish specific tasks. They emphasize the structural organization of objects and the messages exchanged between them, showing the relationships and interactions in a spatial layout.

Below are the key collaboration diagrams that illustrate the main processes within the system.

1. Collaboration Diagram for “End-to-End Hackathon Process”

This diagram illustrates the core lifecycle of a hackathon, from event creation to the final scoreboard viewing, involving the Organizer, Participant, Mentor, Judge, and System.

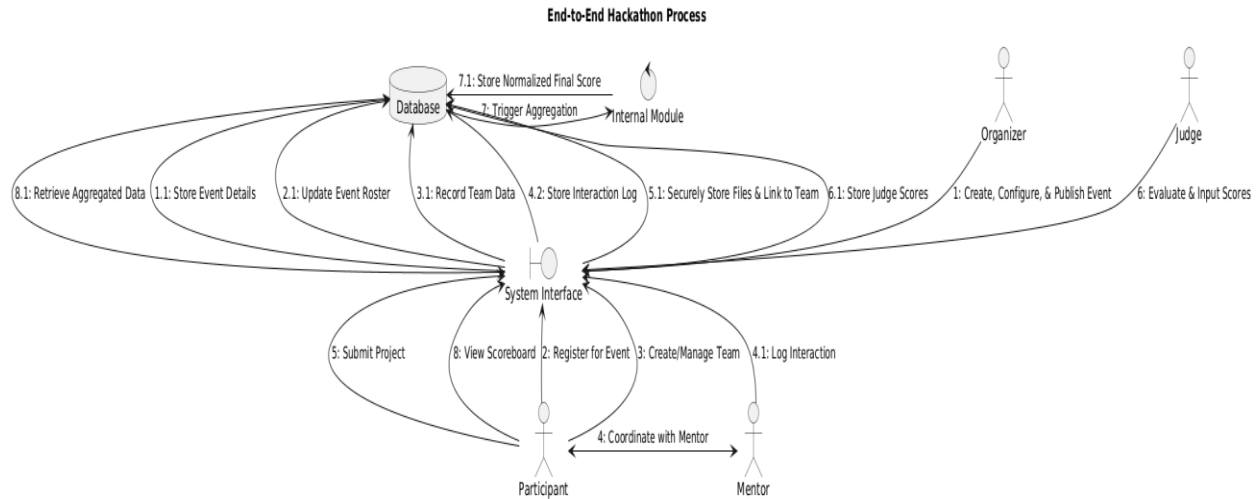


Figure 3.10: Collaboration Diagram for End-to-End Hackathon Process

2. Collaboration Diagram for “System Maintenance and Administration”

This diagram focuses on the Administrator's role in monitoring the platform, executing the archiving process for completed events, and retrieving data from the archival storage.

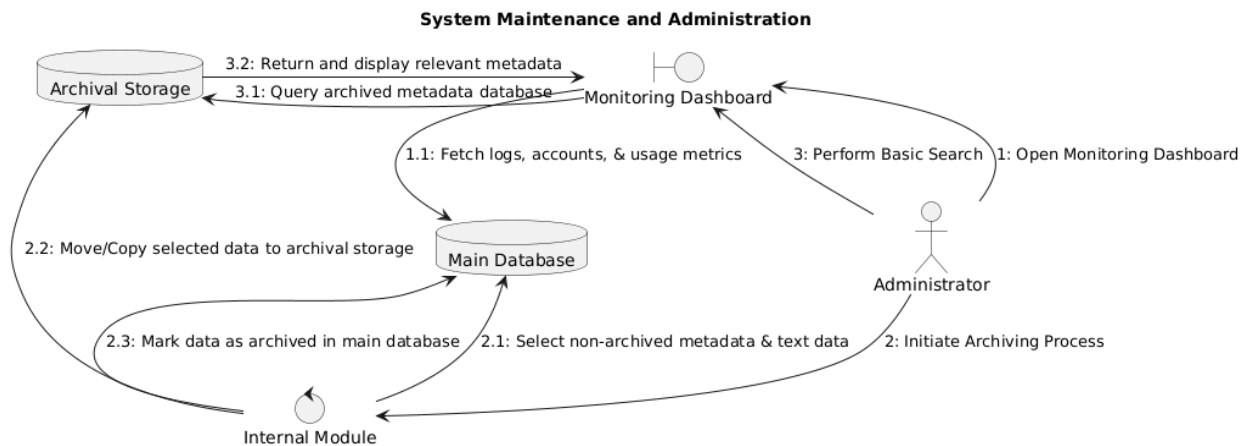


Figure 3.11: Collaboration Diagram for System Maintenance and Administration

3.4. Model Validation

The verification and validation of the problem analysis and system models for HackET ensure that the proposed solution is both correct and complete in addressing the fragmented Ethiopian

hackathon ecosystem. This section confirms that the functional and non-functional requirements derived from stakeholder needs are accurately reflected in the system models.

Verification of Requirements

To ensure the correctness of the system, the requirements were verified against the initial pain points identified during the problem analysis phase:

- **Centralization:** The Centralized Event Listing and Discovery (UC0006) requirement directly addresses the problem of information fragmentation by replacing scattered social media announcements with a single, unified platform.
- **Accessibility:** The Bilingual Interface Support (UC0023) requirement validates the need for digital inclusion by providing Amharic and English options, thereby removing linguistic barriers identified in the participant surveys.
- **Efficiency:** The Judging and Scoring Dashboard (UC0021) and Automatic Score Aggregation (UC0022) modules verify the shift from manual, error-prone spreadsheets to a streamlined, automated workflow for organizers and judges.

Validation of System Models

The system models were validated through iterative reviews to ensure they logically support the end-to-end hackathon lifecycle:

- **Functional Completeness:** Use case specifications, such as Register Account (UC0001) and Secure Project Submissions (UC0020), were cross-checked with the main and alternative flows to ensure all user actions and potential system errors (e.g., submission deadlines, duplicate emails) are handled.
- **Dynamic Consistency:** Sequence diagrams, such as the one for Automatic Session Expiration, validate that the system correctly manages security transitions and role-based access over time.
- **Workflow Accuracy:** Activity diagrams for processes like Team Creation and Matching were validated to confirm that the internal matching module correctly suggests teammates based on the skills and interests defined in the functional requirements.

Stakeholder Review and Refinement

The models were further validated through a feedback loop involving the elicitation techniques used earlier:

- **Organizer Validation:** The Organizer Analytics Dashboard and granular permission settings were confirmed with organizers to ensure they provide the necessary oversight for event management.
- **Participant Validation:** The Rule-Based Content-Based Filtering model was validated against participant survey data, confirming that personalized recommendations would enhance event discovery and engagement.

This comprehensive validation process ensures that the HackET platform's design is robust, technically feasible, and aligned with the strategic goal of fostering a collaborative national innovation ecosystem in Ethiopia.

Chapter Four: System Design

4.1. Overview

This chapter presents the system design of the proposed HackET platform, a centralized, multi-tenant web-based system designed to unify Ethiopia's fragmented hackathon ecosystem. The purpose of this chapter is to translate the previously defined functional and non-functional requirements into a coherent and implementable system design that ensures reliability, scalability, and usability in a real-world operational environment.

System design plays a critical role in achieving the objectives of the project, as it defines how the various system components interact to deliver secure project submissions, automated judging, and persistent developer archiving. Given the involvement of multiple stakeholders—including participants, organizers, mentors, and judges—the design must support distinct yet interconnected workflows while maintaining data integrity and consistent system behavior across diverse user roles.

This chapter outlines the architectural decisions, data management strategies, and interface considerations adopted to support key system features such as centralized event discovery, skill-based team matching, automated scoring normalization, and bilingual (Amharic-English) interface support. Emphasis is placed on aligning design decisions with the constraints of low-bandwidth environments, Ethiopian regional time and calendar settings, and the specific needs of the local innovation community.

The design approach prioritizes modularity and separation of concerns to ensure maintainability and future extensibility. Core system functions such as multi-tier authentication, project lifecycle management, automated notifications, and multi-tenant event configuration are designed as loosely coupled components that communicate through well-defined RESTful interfaces. This allows individual subsystems to evolve independently without compromising overall system stability.

Additionally, this chapter ensures that both functional requirements (such as event creation, registration, and secure submission) and non-functional requirements (including performance,

security, availability, and localization) are explicitly addressed within the design. By grounding architectural and data modeling decisions in these requirements, the proposed design provides a robust foundation for implementation and evaluation.

The remainder of this chapter details the specific design goals, the proposed system architecture, database design (ERD), deployment considerations, user interface design, system integration strategy, and security mechanisms, concluding with a verification of how the design satisfies the stated project requirements.

4.2. Specifying the Design Goals

The HackET project will be developed with the following areas of focus that meet both the functional & non-functional requirements for the centralized hackathon management and discovery service.

Performance

The performance of the HackET system is designed to minimize latency when processing event registrations, project submissions, and real-time leaderboard updates. We want to create a seamless user experience for participants and organizers during high-traffic periods, such as final submission deadlines, and believe that this can be accomplished with optimized backend workflows, reducing unnecessary database round-trip communications and caching frequently accessed data—such as active event listings and user profiles—to ensure rapid response times even in low-bandwidth environments.

Scalability

The system will allow for future increases in the number of concurrent users, hosted hackathons, project repositories, and judging transactions. The platform is architected with a modular service-oriented approach which allows the backend to be stateless, and there will also be the ability to scale resources and optimize the platform’s database architecture to accommodate higher loads as the ecosystem grows to include more institutions and organizations across Ethiopia.

Security

Security is a priority during the design of the HackET project to prevent unauthorized access to administrative panels, ensure the integrity of project submissions, and protect sensitive user information. Role-based access controls (RBAC), secured communication protocols, and rigorous server-side validation will enforce the security of privileged operations, such as score entry by judges and system configuration by administrators. The design will ensure that submission data is protected and that user authentication remains robust against common vulnerabilities.

Availability

The system's goal is to ensure minimal downtime, particularly during critical hackathon windows and submission phases. This is accomplished through a robust architectural foundation, a reliable method of data persistence, and an organized approach to error handling and graceful degradation that will allow for the continuity of core services—such as project uploading and event discovery—during peak usage or intermittent connectivity issues.

Usability

The interface of the system is designed to be easy to understand and operate so that participants, organizers, and judges can interact with the platform with as little training as possible. The screen flow has been developed with simplicity and consistency in mind, incorporating multilingual support (English and Amharic) to ensure accessibility within the Ethiopian context. The various modules provide clear, adequate feedback regarding registration status, submission confirmations, team formation progress, and evaluation results.

4.3. System Design

The proposed HackET system adopts a layered client–server architecture, which organizes the entire platform into distinct, well-defined layers based on their specific responsibilities. This classic architectural style promotes separation of concerns, making the system easier to understand, develop, test, maintain, and scale over time.

4.3.1. Proposed Software Architecture

HackET follows a layered client-server architecture in which a single React web client communicates with a unified Express.js backend exclusively over REST. The backend is decomposed into domain-scoped service modules, each owning a well-defined bounded context: auth, events, teams, submissions, judging, matching, mentorship, notifications, analytics, audit, archiving, storage, discussions, profile, organization, bookmark, and email. Each module follows a controller → service → Prisma pattern in which controllers handle HTTP concerns and delegate all business logic to their corresponding service. The Express server mounts twenty route namespaces under the /api/v1 prefix and applies a layered middleware pipeline — Helmet security headers, CORS, JSON body parsing, Morgan logging, global rate limiting, and active-user metrics — before any request reaches a controller.

An in-process SchedulerWorker handles all background operations, polling PostgreSQL every sixty seconds to advance hackathon state-machine transitions (UPCOMING → REGISTRATION_OPEN → REGISTRATION_CLOSED → IN_PROGRESS → JUDGING → COMPLETED) based on configured temporal deadlines, invalidating the events:active Redis cache key on any state change, and emitting deadline:upcoming events for approaching submission windows.

Cross-cutting side effects — audit logging, notification dispatch, and transactional email — are decoupled from primary business logic through a Node.js EventEmitter-based internal event bus. The audit.service, notification.service, and email.service register their listeners at server startup and react to named bus events independently, preventing tight coupling between domain controllers and infrastructure concerns.

PostgreSQL, accessed through Prisma ORM, serves as the primary relational store for all platform entities. Redis provides session persistence, leaderboard caching, and active-event cache invalidation. Both services are containerized via Docker Compose for local development and connected to the Express server at startup with graceful shutdown handling for both.

AWS S3 serves as the unified object store for two distinct concerns. For blob storage, project submission assets, profile media, and certificate files are managed through the storage service; the /api/v1/storage route exposes presigned URL generation so clients upload directly to S3

without proxying binary data through the API server. For archival, when a hackathon reaches ARCHIVED status the archiving service serializes the full hackathon record — metadata, teams, submissions, and scores — as a structured JSON object written to S3. The federated search endpoint performs concurrent queries against both active PostgreSQL records and these archived S3 JSON objects, merging results into a single unified response.

The React frontend is the sole client tier, communicating with the same REST API scoped by role-specific JWT claims and Express middleware permission classes.

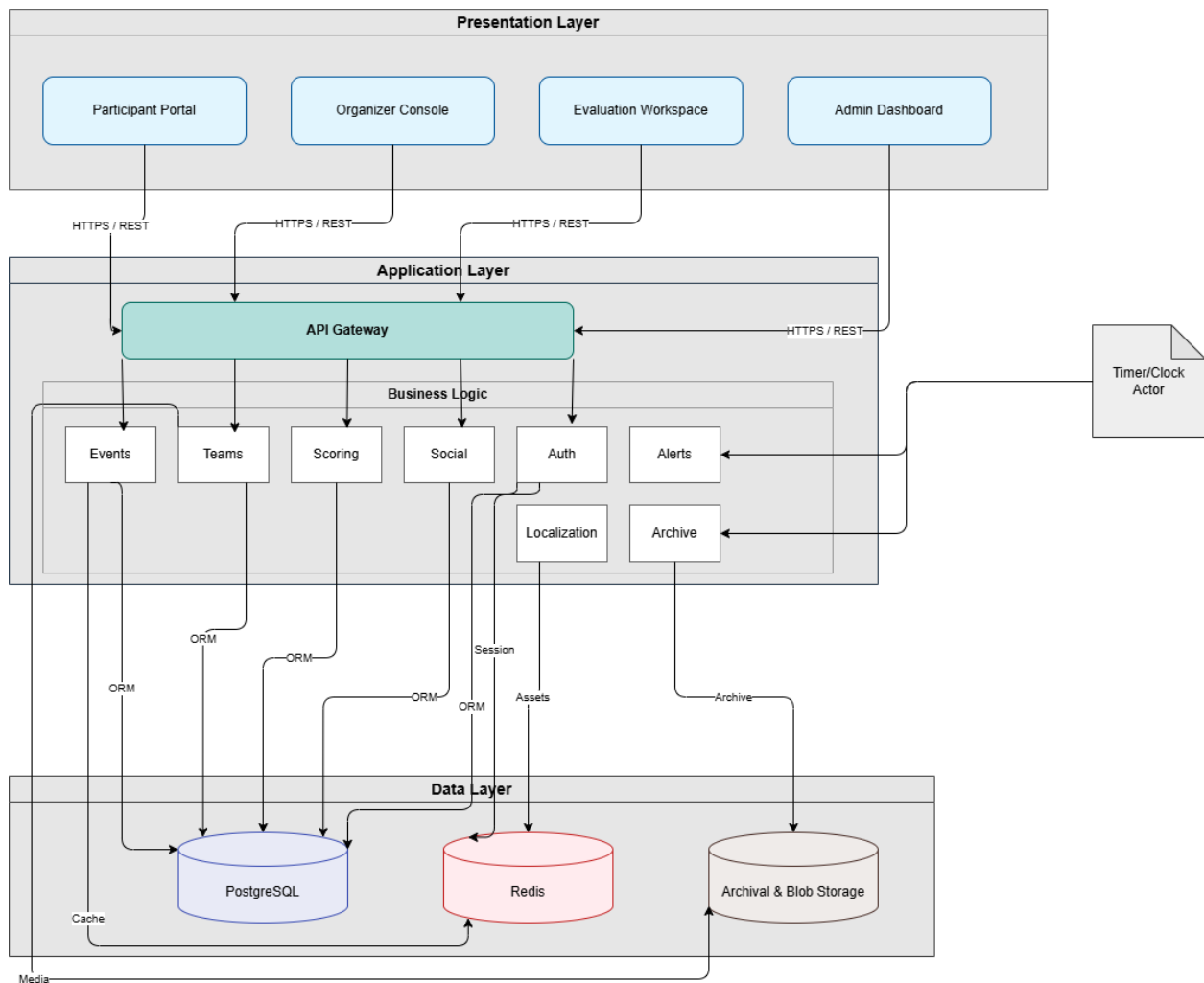


Figure 4.1: HackET System Architecture Diagram

4.3.2. Subsystem Decomposition

The backend is organized into ten route-and-service modules, each encapsulating a distinct responsibility boundary. The table below summarizes each module's scope, primary responsibility, and key Prisma models.

Subsystem / App	Responsibility	Key Models
auth	User registration, email verification, OAuth (Google/GitHub) and local authentication, session lifecycle, password reset, RBAC middleware guards	User, Session, EmailVerificationToken, PasswordResetToken, AdminInvitation
events	Hackathon lifecycle state machine (DRAFT→UPCOMING→IN_PROGRESS→ARCHIVED), participant registration and waitlist management, temporal scheduling via background worker, completed-	Hackathon, HackathonTag, Registration, ArchiveJob, ArchiveRecord

	event archiving to filesystem	
teams & submissions	Team formation, membership and invitation management (invite/request flows), project submission intake, versioned submission history	Team, TeamMember, TeamInvitation, Submission, SubmissionHistory
judging	Per-criteria scoring, judge-to-submission assignment, weighted score normalization, leaderboard aggregation into pre-computed scoreboard rows	JudgingCriteria, JudgingAssignment, Score, ScoreboardEntry
staff & mentors hip	Per-event staff invitation and role-quota configuration, mentorship session scheduling and interaction logging, skill-based participant matching, personalized event recommendations	StaffAssignment, StaffRoleConfig, StaffInvitation, MentorAssignment, MentorRequest, MentorSession, MentorInteraction, Recommendation

profiles & organizations	User profile management, organization creation and membership, organization bookmarking	UserProfile, Organization, OrganizationMember, Bookmark
notifications	In-app notification delivery, multi-channel delivery tracking (email / SMS / push / in-app), broadcast template management, transactional email dispatch, internal EventEmitter-based event bus decoupling side effects from primary actions	Notification, NotificationDelivery, NotificationTemplate, NotificationBroadcast, UserNotificationPreference
admin & analytics	User suspension and role provisioning, audit trail persistence, precomputed per-event analytics snapshots, async report generation (CSV / PDF / JSON / XLSX)	AuditLog, EventAnalyticsSnapshot, AnalyticsJob, ReportJob, ReportFile

discussions, feedback & certificates	Event-scoped discussion threads with threaded comments and archival, post-event participant feedback collection, certificate generation and secure tokenized download	Discussion, DiscussionComment, EventFeedback, Certificate, CertificateIssuanceJob
search & i18n	Federated full-text search across live database records and archived JSON snapshots, zero-result and latency query analytics, bilingual (English / Amharic) string key-value store	SearchDocument, SearchLog, LocalizedString

Table 4.1. Backend Subsystem Decomposition

4.3.3. Database Design

The database is PostgreSQL 16 managed exclusively through Prisma ORM, with all schema changes version-controlled via Prisma Migrations. The central entity is Hackathon, which acts as the multi-tenant boundary for all event activity — teams, submissions, staff, judging, and discussions are all scoped to a specific hackathon instance. Identity is handled by User, which carries a global UserRole and is extended by a separate UserProfile record for biographical and matching data.

The following entity-relationship diagrams summarizes the entities, their attributes, and their inter-entity relationships.

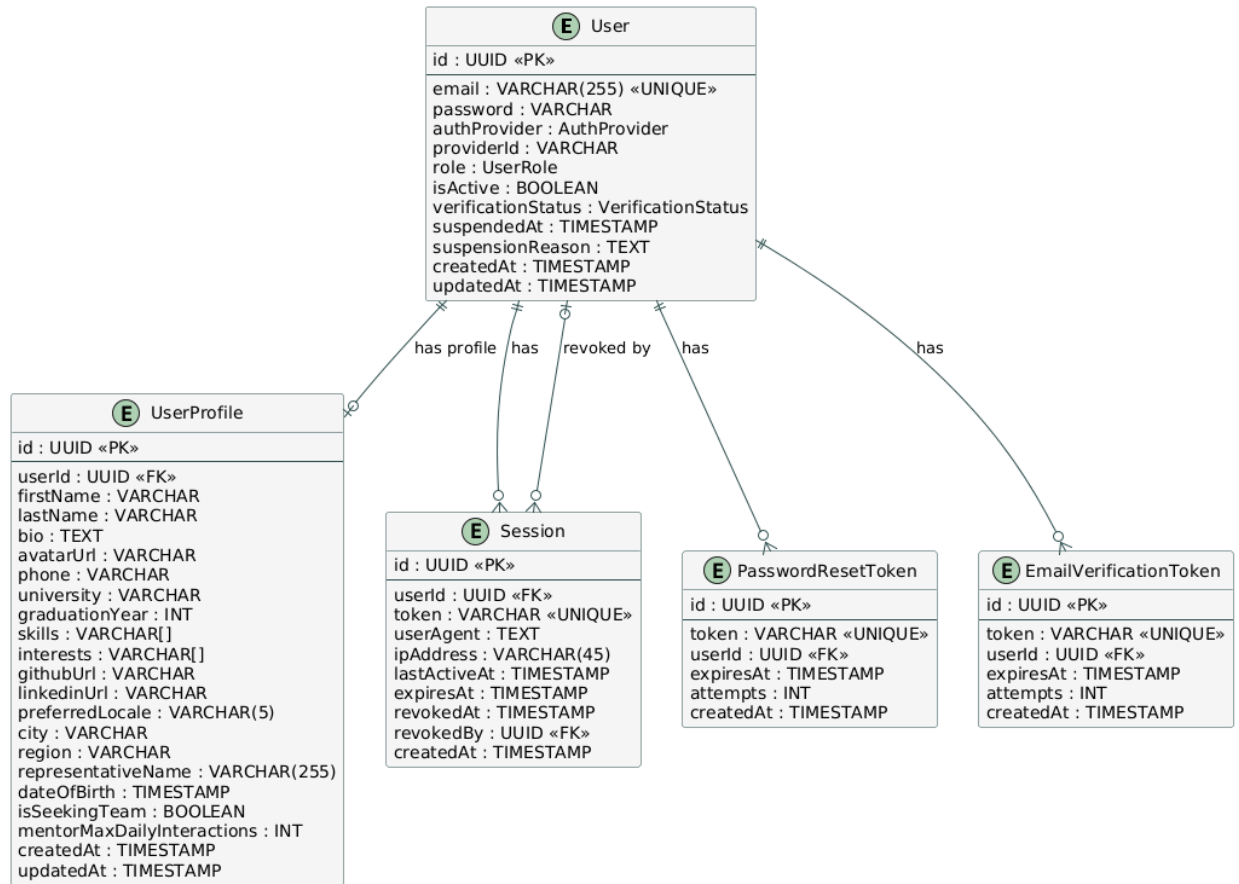


Figure 4.2. Users & Auth Entity-Relationship Diagram

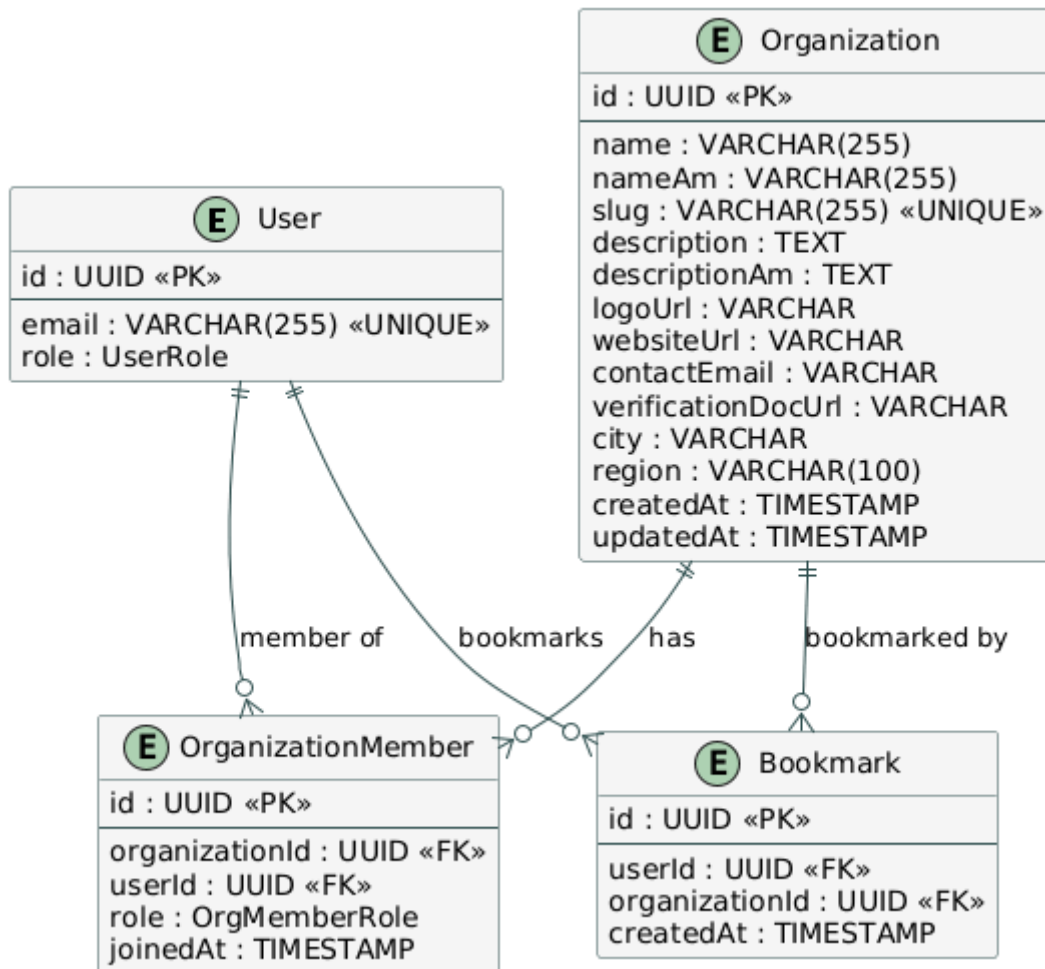


Figure 4.3. Organization Entity-Relationship Diagram

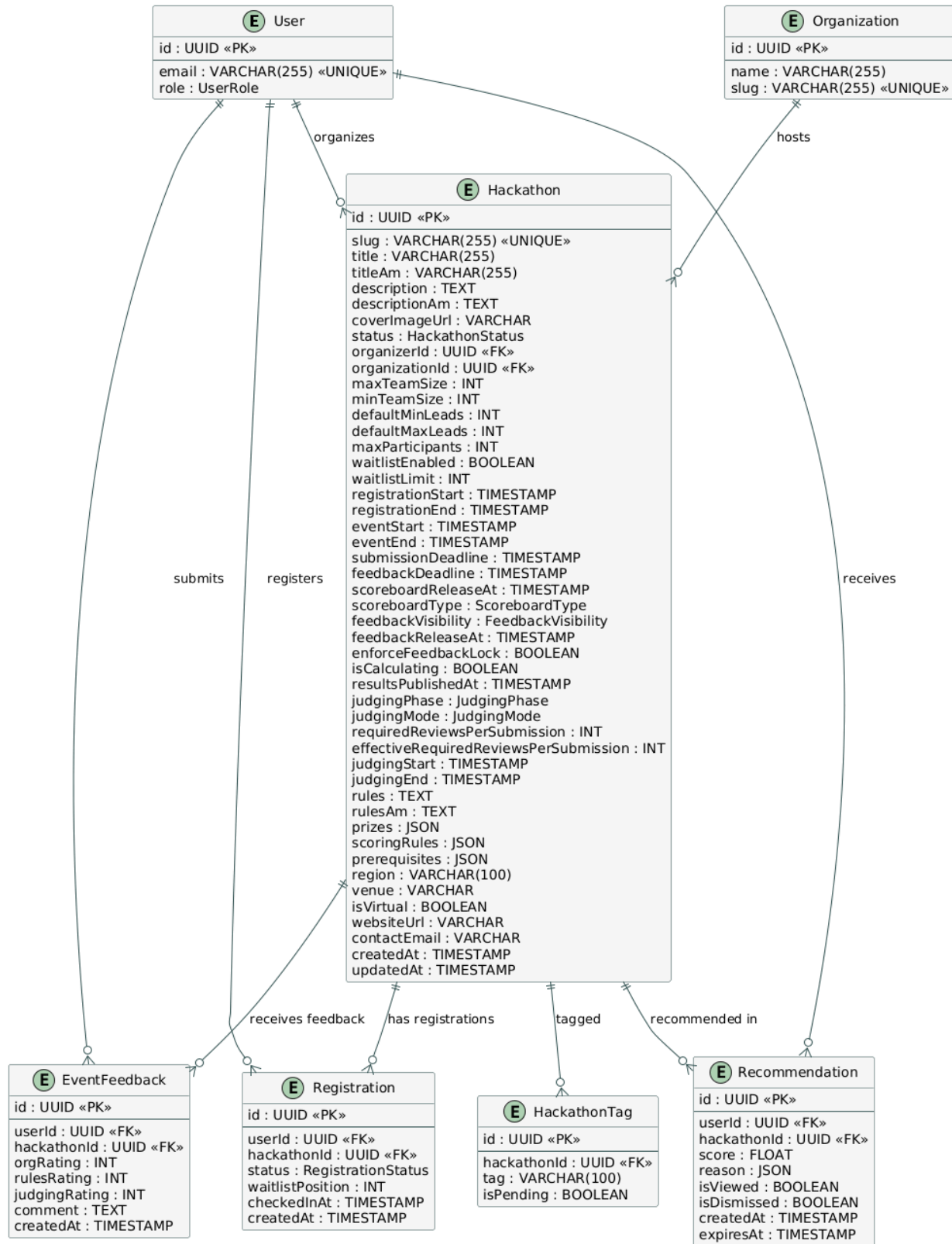


Figure 4.4. Hackathons Entity-Relationship Diagram

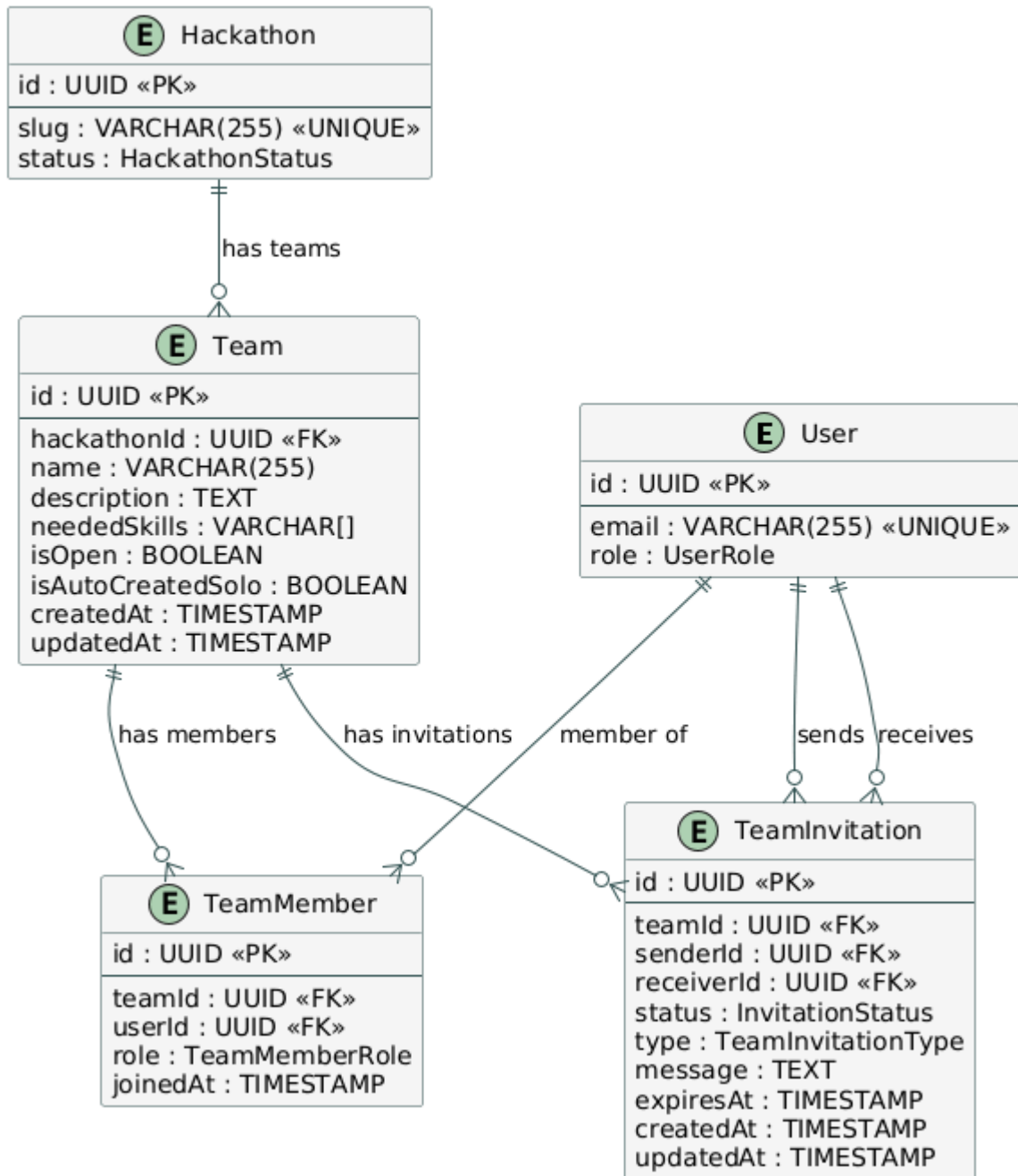


Figure 4.5. Teams Entity-Relationship Diagram

Figure 4.7. Staff Entity-Relationship Diagram

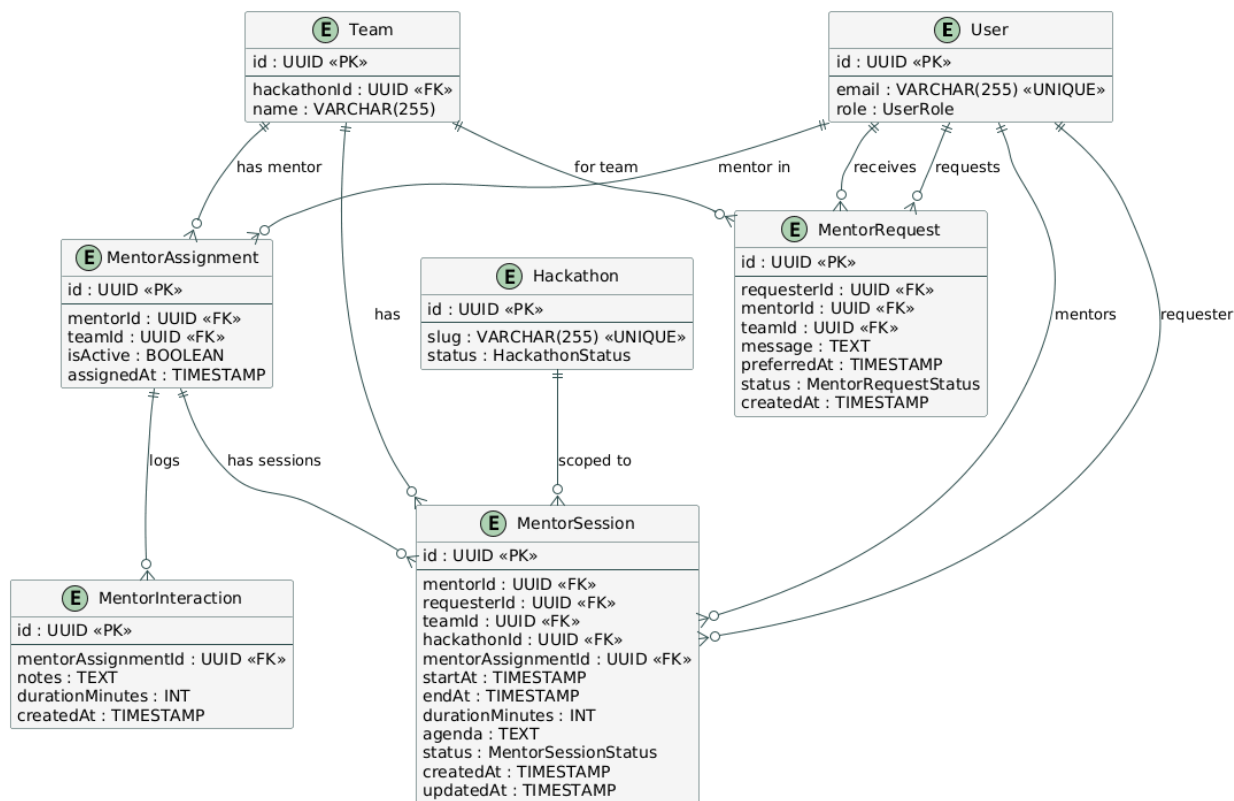


Figure 4.8. Mentorship Entity-Relationship Diagram

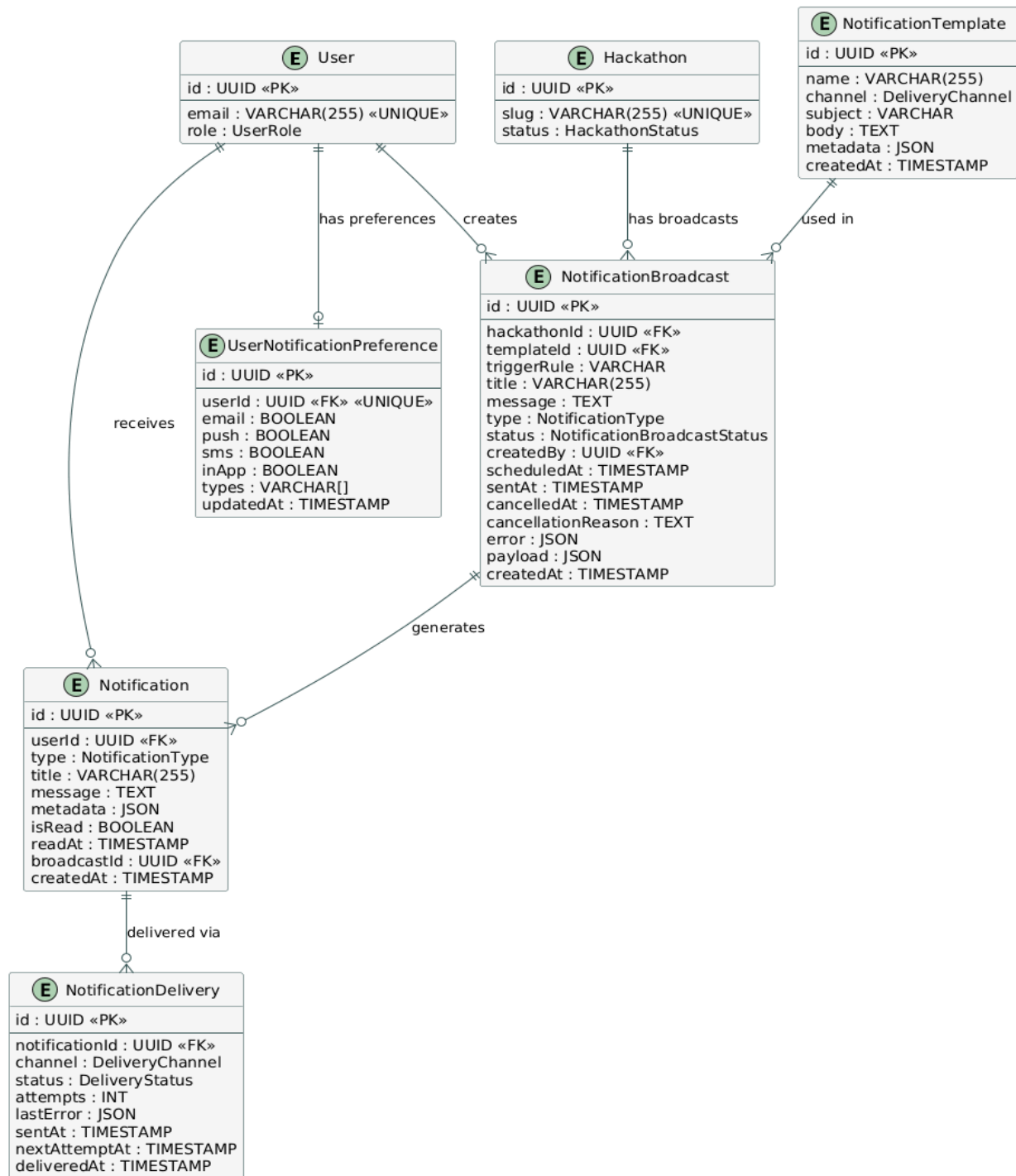


Figure 4.9. Notifications Entity-Relationship Diagram

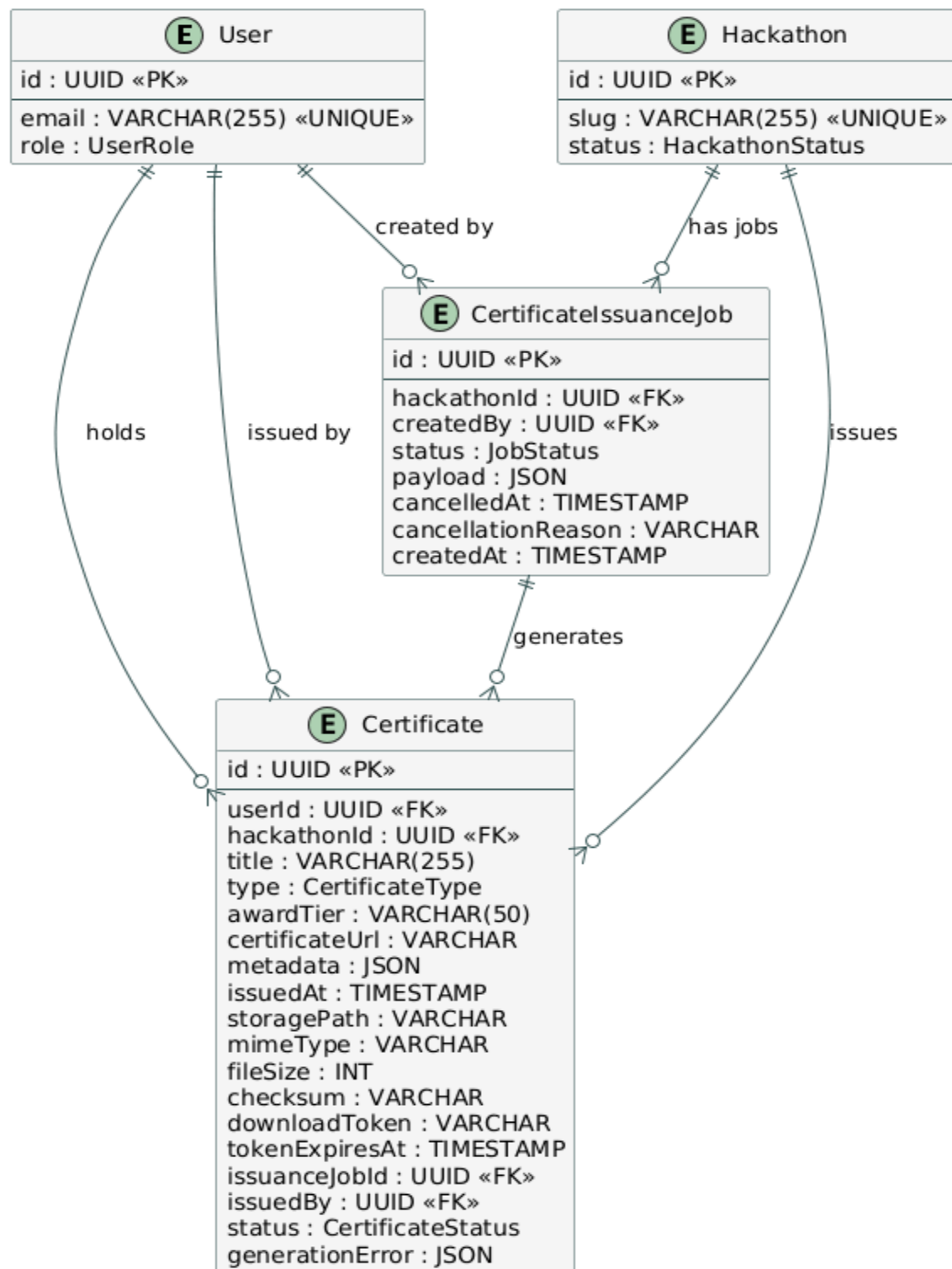


Figure 4.10. Certificates Entity-Relationship Diagram

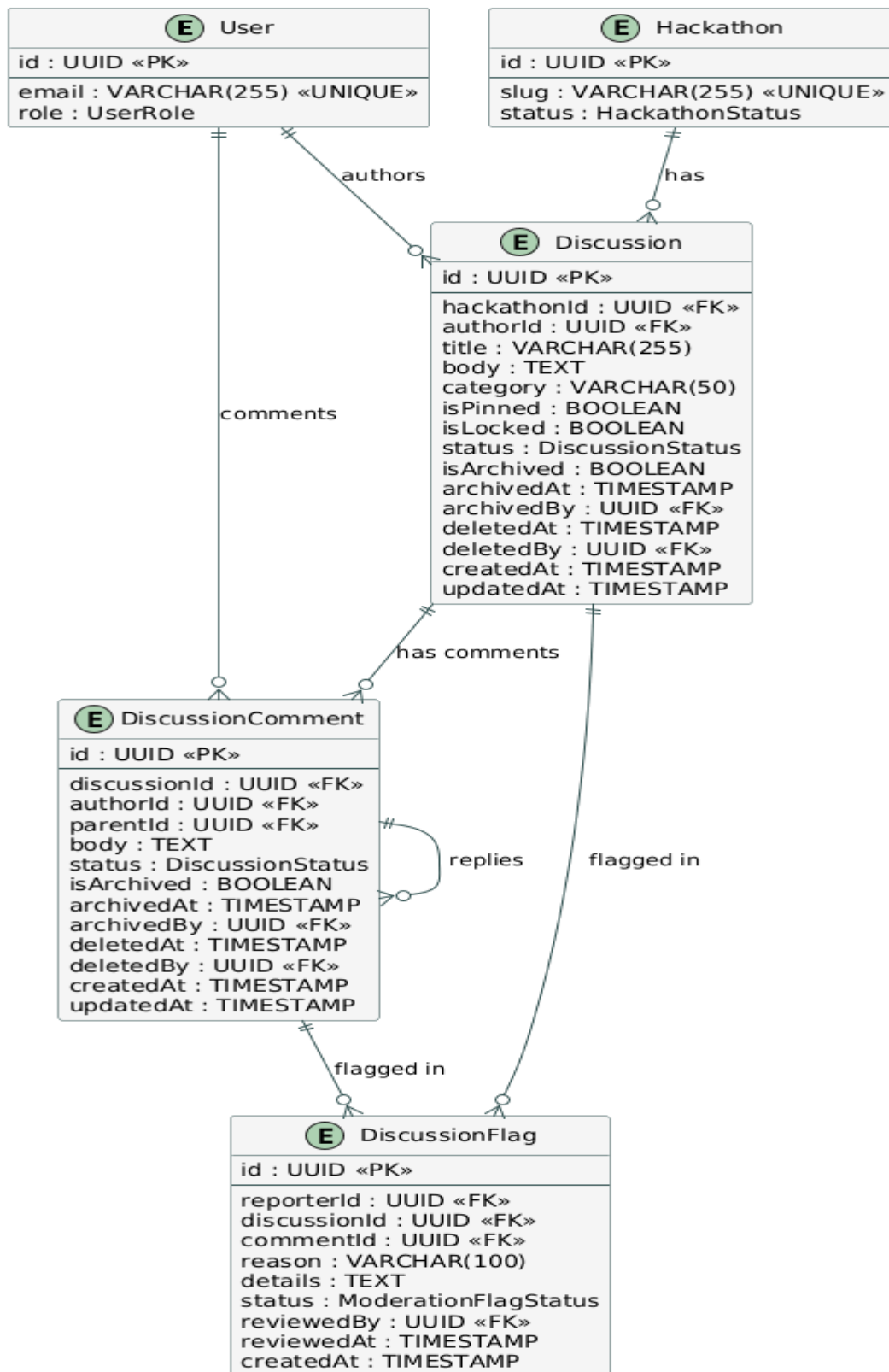


Figure 4.11. Discussions Entity-Relationship Diagram

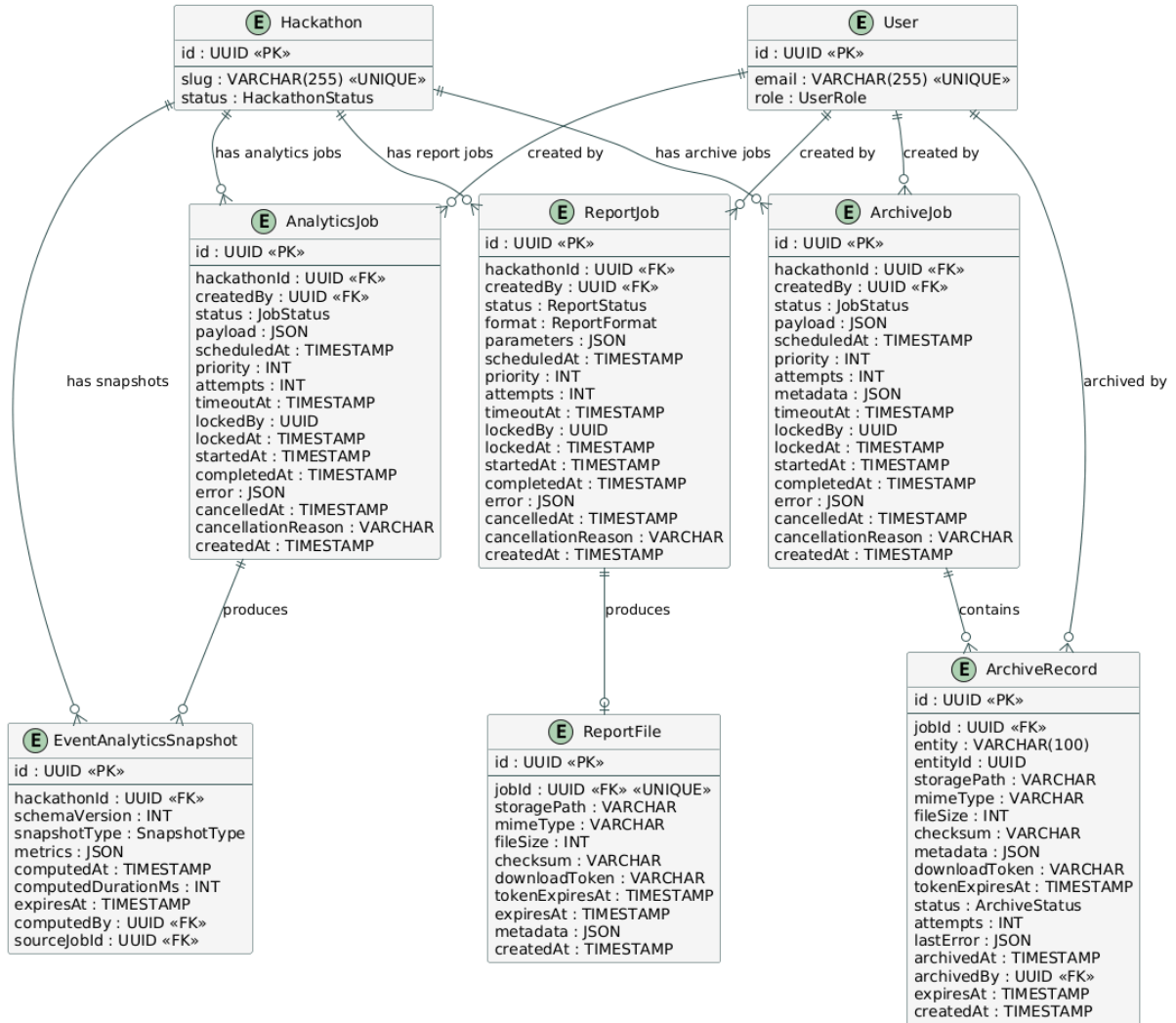


Figure 4.12. Analytics & Jobs Entity-Relationship Diagram

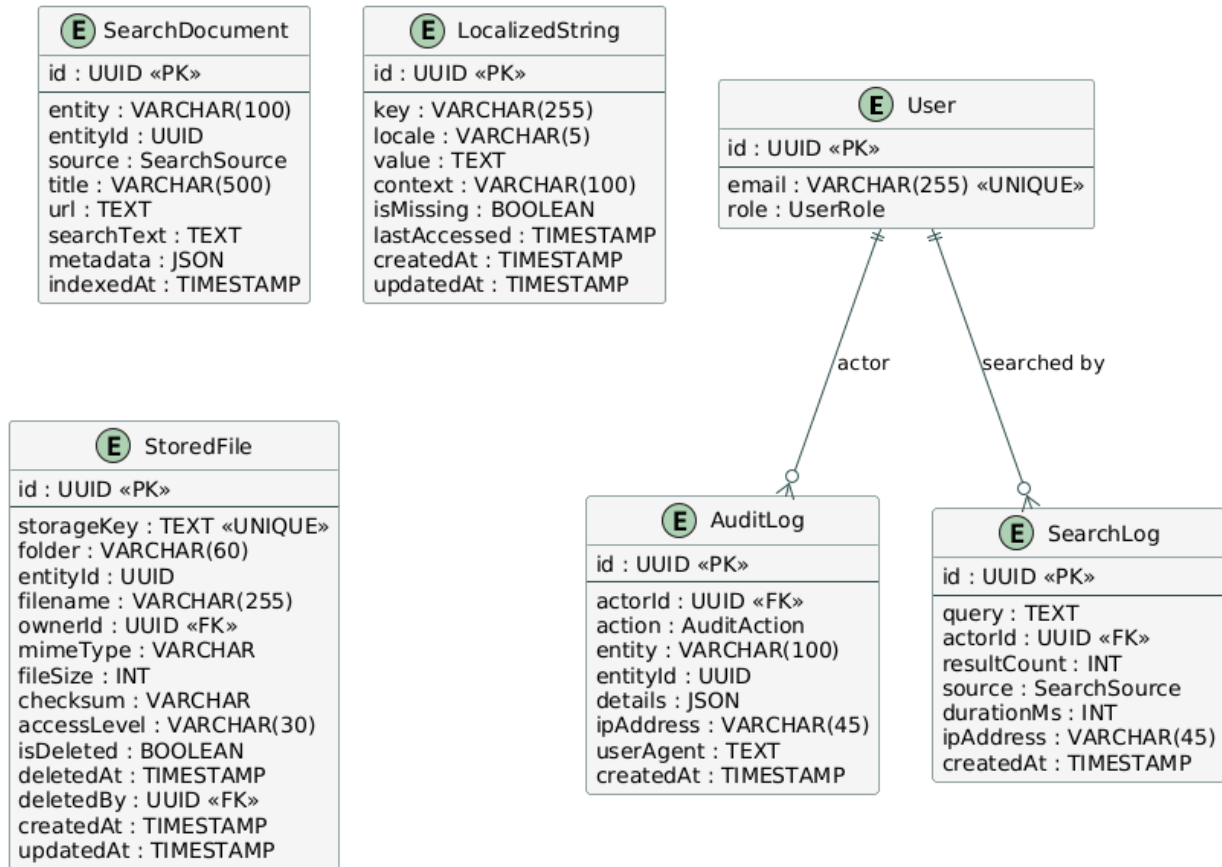


Figure 4.13. Search & Audit Entity-Relationship Diagram

Key database design decisions are as follows. First, all primary keys are UUIDs generated via PostgreSQL's native `gen_random_uuid()` function, ensuring globally unique identifiers suitable for distributed service expansion without coordination. Second, `User.email` is the primary user identifier and carries a unique database index. The system supports three authentication providers (LOCAL, GOOGLE, GITHUB) via the `AuthProvider` enum, with `providerId` stored for OAuth-linked accounts. Third, Hackathon implements a state-machine lifecycle enforced at the API layer. The valid `HackathonStatus` progression is: DRAFT → UPCOMING → REGISTRATION_OPEN → REGISTRATION_CLOSED → IN_PROGRESS → JUDGING → COMPLETED, with terminal states CANCELLED, SUSPENDED, and ARCHIVED. A background `schedulerWorker` automates time-based transitions. Fourth, the platform implements a two-tier RBAC model. Global permissions are governed by `User.role` (`UserRole` enum:

PARTICIPANT, ORGANIZER, JUDGE, MENTOR, ADMIN). Event-scoped permissions are granted through StaffAssignment, which links a User to a Hackathon with a specific StaffRole (CO_ORGANIZER, JUDGE, MENTOR, TECHNICAL_LEAD, etc.), allowing a user to hold different roles across different events. Fifth, ScoreboardEntry is a pre-aggregated leaderboard table populated by background scoring jobs. It stores the computed finalScore and rank per submission per hackathon, enabling fast leaderboard reads without recalculating from raw Score rows on every request. A composite unique index on (hackathon_id, submission_id) enforces one entry per team per event. Sixth, all binary media assets — submission files, organization verification documents, profile avatars, and certificates — are managed via an object storage abstraction layer. While the local development environment utilizes disk storage via Multer, the production environment natively writes to AWS S3. The database stores only the relative S3 object keys (rather than absolute URLs) to remain CDN-agnostic, allowing seamless asset delivery via Amazon CloudFront without schema or data modifications. Seventh, bilingual support (English and Amharic) is implemented at the schema level via parallel *Am fields on content-bearing models (e.g., Hackathon.titleAm, Hackathon.descriptionAm, JudgingCriteria.nameAm) and a general-purpose LocalizedString key-value table for UI strings and email templates. UserProfile.preferredLocale (defaulting to "en") drives locale selection per user. Eighth, AuditLog provides an append-only audit trail for all sensitive mutations. The AuditAction enum covers 68 distinct action types spanning authentication events, score overrides, staff management, team lifecycle, and archival operations. Each record captures the actor, entity type, entity ID, a details JSONB payload, IP address, and user agent. Ninth, session management uses a hybrid approach: a Session model in PostgreSQL stores the authoritative session record with expiresAt, revokedAt, and revocation metadata for admin-forced logout; Redis provides the hot-path lookup for active sessions with role-differentiated TTLs (PARTICIPANT: 7 days; other roles: 30 minutes). Tenth, SearchDocument provides a unified full-text search index across both live PostgreSQL records and historical data, distinguished by the SearchSource enum (CURRENT vs ARCHIVE). When event metadata is archived, its complete relational dependency tree is serialized into a cold JSON file and offloaded to an AWS S3 archival bucket. The live heavy relational rows are then purged from PostgreSQL, while the lightweight SearchDocument table retains the search vectors and the S3 object reference link,

ensuring fast global discovery without bloating primary database storage. SearchLog tracks every query with result count and duration for zero-result analytics and latency monitoring

The following table describes the database entities, their key attributes, and their relationship cardinalities.

Entity	Key Attributes	Primary Relationships
User	id, email (UNQ), password, authProvider (LOCAL GOOGLE GITHUB), role (UserRole), isActive, verificationStatus, suspendedAt	1-to-1 UserProfile; 1-to-many Registration; 1-to-many StaffAssignment; 1-to-many Notification; 1-to-many AuditLog; 1-to-many Session
UserProfile	id, userId (FK), firstName, lastName, bio, avatarUrl, skills[], interests[], preferredLocale, isSeekingTeam, mentorMaxDailyInteractions	1-to-1 User
Organization	id, slug (UNQ), name, nameAm, verificationDocUrl, region	1-to-many Hackathon; 1-to-many OrganizationMember; 1-to-many Bookmark

OrganizationMember	id, organizationId (FK), userId (FK), role (ADMIN MEMBER)	Many-to-1 Organization; Many-to-1 User
Hackathon	id, slug (UNQ), title, titleAm, status (HackathonStatus), organizerId (FK), organizationId (FK), judgingMode, judgingPhase, scoreboardType, feedbackVisibility	Many-to-1 User (organizer); Many-to-1 Organization; 1-to-many Registration; 1-to-many Team; 1-to-many StaffAssignment; 1-to-many JudgingCriteria; 1-to-many Discussion
HackathonTag	id, hackathonId (FK), tag, isPending	Many-to-1 Hackathon
Registration	id, userId (FK), hackathonId (FK), status (REGISTERED WAITLISTED CHECKED_IN WITHDRAWN), waitlistPosition, checkedInAt	Many-to-1 User; Many-to-1 Hackathon
Team	id, hackathonId (FK), name, neededSkills[], isOpen, isAutoCreatedSolo	Many-to-1 Hackathon; 1-to-many TeamMember ; 1-to-1

		Submission; 1-to-many MentorAssignment
TeamMember	id, teamId (FK), userId (FK), role (LEADER MEMBER)	Many-to-1 Team; Many-to-1 User
TeamInvitation	id, teamId (FK), senderId (FK), receiverId (FK), status (InvitationStatus), type (INVITATION REQUEST), expiresAt	Many-to-1 Team; Many-to-1 User (sender & receiver)
Submission	id, teamId (FK, UNQ), hackathonId (FK), version, status (DRAFT SUBMITTED UNDER_REVIEW SCORED), fileUrls[], finalScore, rank	1-to-1 Team; Many-to-1 Hackathon; 1-to-many Score; 1-to-many JudgingAssignment; 1-to-many SubmissionHistory
SubmissionHistory	id, submissionId (FK), previousTitle, previousFileUrls[], changedAt, changedBy (FK)	Many-to-1 Submission; Many-to-1 User
JudgingCriteria	id, hackathonId (FK), name, nameAm, maxScore, weight, sortOrder	Many-to-1 Hackathon; 1-to-many Score

JudgingAssignment	id, hackathonId (FK), submissionId (FK), judgeId (FK)	Many-to-1 Hackathon; Many-to-1 Submission; Many-to-1 User (judge)
Score	id, submissionId (FK), judgeId (FK), criteriaId (FK), value, comment, isCommentPublic	Many-to-1 Submission; Many-to-1 User (judge); Many-to-1 JudgingCriteria
ScoreboardEntry	id, hackathonId (FK), submissionId (FK), finalScore, rank, aggregatedAt	Many-to-1 Hackathon; Many-to-1 Submission
StaffAssignment	id, userId (FK), hackathonId (FK), staffRole (StaffRole), isLead, isActive, revokedAt, revokedBy	Many-to-1 User; Many-to-1 Hackathon
StaffInvitation	id, hackathonId (FK), email, staffRole, token (UNQ), status, expiresAt	Many-to-1 Hackathon; Many-to-1 User (inviter & acceptor)
StaffRoleConfig	id, hackathonId (FK), staffRole, minLeads, maxLeads	Many-to-1 Hackathon
Notification	id, userId (FK), type (NotificationType), title, message, isRead, broadcastId (FK)	Many-to-1 User; 1-to-

		many NotificationDelivery; Many-to-1 NotificationBroadcast
NotificationDelivery	id, notificationId (FK), channel (EMAIL SMS PUSH IN_APP), status, attempts, nextAttemptAt	Many-to-1 Notification
NotificationBroadcast	id, hackathonId (FK), templateId (FK), triggerRule, scheduledAt	Many-to-1 Hackathon; Many-to-1 NotificationTemplate; 1-to-many Notification
UserNotificationPreference	id, userId (FK), email, push, sms, inApp, types[]	1-to-1 User
Certificate	id, userId (FK), hackathonId (FK), type (CERTIFICATE BADGE), awardTier, certificateUrl, status, downloadToken, tokenExpiresAt	Many-to-1 User; Many-to-1 Hackathon; Many-to-1 CertificateIssuanceJob
CertificateIssuanceJob	id, hackathonId (FK), createdBy (FK), status (JobStatus), payload (JSON)	Many-to-1 Hackathon; 1-to-many Certificate

MentorAssignment	id, mentorId (FK), teamId (FK), isActive	Many-to-1 User (mentor); Many-to-1 Team; 1-to-many MentorInteraction; 1-to-many MentorSession
MentorInteraction	id, mentorAssignmentId (FK), notes, durationMinutes	Many-to-1 MentorAssignment
MentorRequest	id, requesterId (FK), mentorId (FK), teamId (FK), status (MentorRequestStatus), preferredAt	Many-to-1 User (requester & mentor); Many-to-1 Team
MentorSession	id, mentorId (FK), requesterId (FK), teamId (FK), hackathonId (FK), startAt, endAt, status (MentorSessionStatus)	Many-to-1 User (mentor & requester); Many-to-1 Team; Many-to-1 Hackathon
Discussion	id, hackathonId (FK), authorId (FK), title, body, category, isPinned, isLocked, status, isArchived	Many-to-1 Hackathon; Many-to-1 User; 1-to-

		many DiscussionCo mment
DiscussionComment	id, discussionId (FK), authorId (FK), parentId (FK, self-ref), body, isArchived	Many-to-1 Discussion; Many-to-1 User; 0-to- many DiscussionCo mment (replies)
EventFeedback	id, userId (FK), hackathonId (FK), orgRating, rulesRating, judgingRating, comment	Many-to-1 User; Many- to-1 Hackathon
Recommendation	id, userId (FK), hackathonId (FK), score, reason (JSON), isViewed, isDismissed, expiresAt	Many-to-1 User; Many- to-1 Hackathon
AuditLog	id, actorId (FK), action (AuditAction), entity, entityId, details (JSON), ipAddress, userAgent	Many-to-1 User (actor)
Session	id, userId (FK), token (UNQ), expiresAt, lastActiveAt, revokedAt, revokedBy (FK)	Many-to-1 User
AdminInvitation	id, email, role, token (UNQ), invitedBy (FK), status, expiresAt	Many-to-1 User (inviter & acceptor)
PasswordResetToken	id, userId (FK), token (UNQ), expiresAt, attempts	Many-to-1 User

EmailVerificationToken	id, userId (FK), token (UNQ), expiresAt, attempts	Many-to-1 User
LocalizedString	id, key, locale (en am), value, context, isMissing	—
Bookmark	id, userId (FK), organizationId (FK)	Many-to-1 User; Many-to-1 Organization
EventAnalyticsSnapshot	id, hackathonId (FK), metrics (JSON), snapshotType (DAILY ON_DEMAND), computedAt, expiresAt	Many-to-1 Hackathon; Many-to-1 AnalyticsJob
AnalyticsJob	id, hackathonId (FK), status (JobStatus), lockedBy, lockedAt	Many-to-1 Hackathon; 1-to-many EventAnalyticsSnapshot
ReportJob	id, hackathonId (FK), status (ReportStatus), format (CSV PDF JSON XLSX), parameters (JSON)	Many-to-1 Hackathon; 1-to-1 ReportFile
ReportFile	id, jobId (FK, UNQ), storagePath, downloadToken, tokenExpiresAt, expiresAt	1-to-1 ReportJob
ArchiveJob	id, hackathonId (FK), status (JobStatus), lockedBy, lockedAt	Many-to-1 Hackathon; 1-to-many ArchiveRecord

ArchiveRecord	id, jobId (FK), entity, entityId, storagePath, status (ArchiveStatus), downloadToken	Many-to-1 ArchiveJob
SearchDocument	id, entity, entityId, source (CURRENT ARCHIVE), searchText, title, url, metadata (JSON)	—
SearchLog	id, query, actorId, resultCount, source, durationMs, ipAddress	—

Table 4.2. Database Entities and Relationships

4.3.4. Deployment Diagram

The production deployment targets a Linux-based server environment. Nginx acts as the unified API Gateway and TLS termination point, routing all inbound HTTPS traffic on port 443 to the appropriate upstream service. REST API requests are forwarded to the Node.js/Express server (port 5000), are proxied to the same Express process. The React frontend application (port 3000) is served as a Node.js SSR process, also proxied through Nginx. The PostgreSQL database and Redis instance run on the same server or on managed cloud equivalents. The Scheduler Worker runs in-process within the Express server — it is not a separate process — polling the database to automate hackathon lifecycle state transitions. Nodemailer dispatches transactional emails (invitations, OTPs, certificates) through an external SMTP relay (Brevo by default, with AWS SES as an alternative). AWS S3 provides dedicated storage for all binary file uploads (project submissions, documents) and serialized JSON hackathon archives, isolated from the database and application servers.

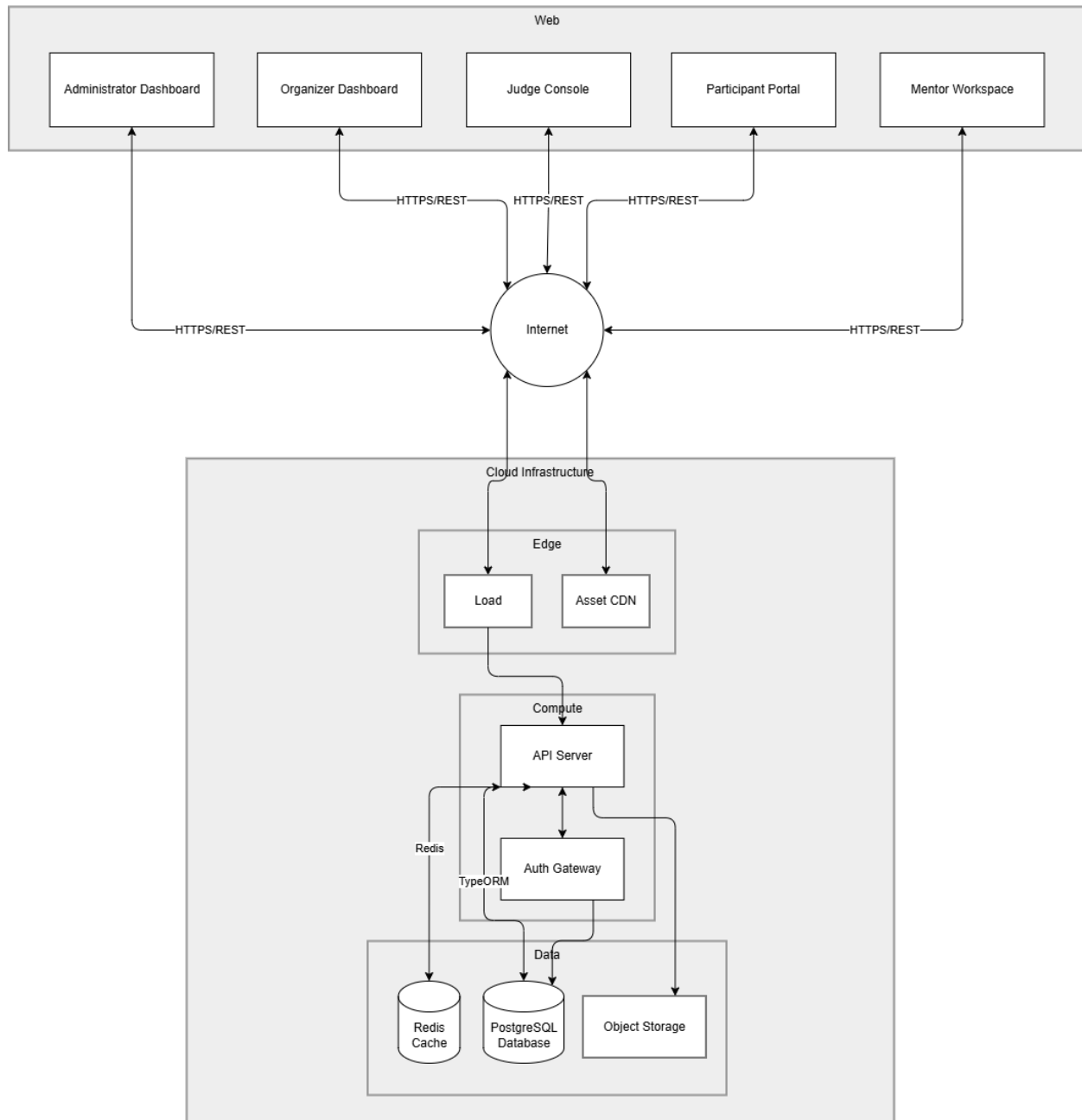


Figure 4.14: HackET Deployment Diagram

4.3.5. User Interface Design

Public Web Interface

The HackET platform follows a clean, card-based web design adapted for the Ethiopian hackathon ecosystem. A light surface palette (#f7f8f9 page background, white cards) with emerald green accent (#22c55e / #16a34a) and slate/navy typography was chosen for strong

contrast on common laptop and mobile screens. The interface supports English and Amharic via an in-header locale toggle.

Unauthenticated users see a top navigation bar with Home, Events, and Leaderboard, plus Log in / Sign up actions. Authenticated users switch to a left sidebar with a contextual top action bar (notifications bell, language toggle).

Key public screens are described below.

Screen	Primary Function
Landing Page	Hero messaging, platform value proposition, feature highlights, and calls to action for event discovery and registration
Events Discovery	Searchable, filterable catalog of hackathons with status badges, region, dates, and team-size metadata
Event Details	Full event profile with cover image, description, registration window, and one-click participant registration
Leaderboard	Event-scoped ranked list of teams/submissions with medal styling for top placements
Login / Sign Up	Email-based authentication with role selection (Participant, Organizer, Judge, Mentor) at registration

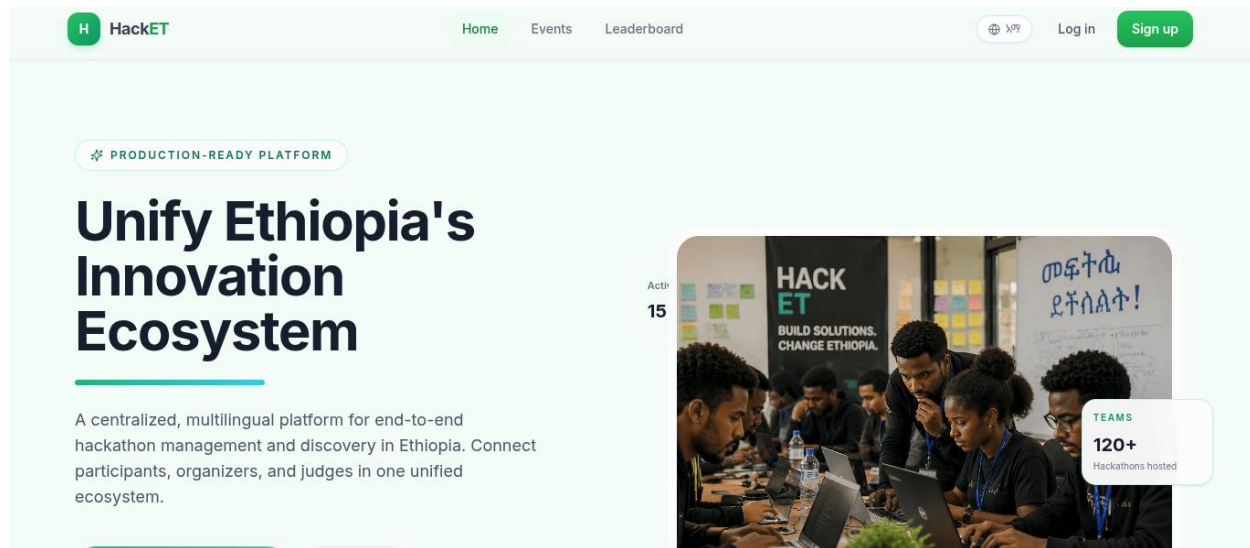


Figure 4.15. HackET Landing Page

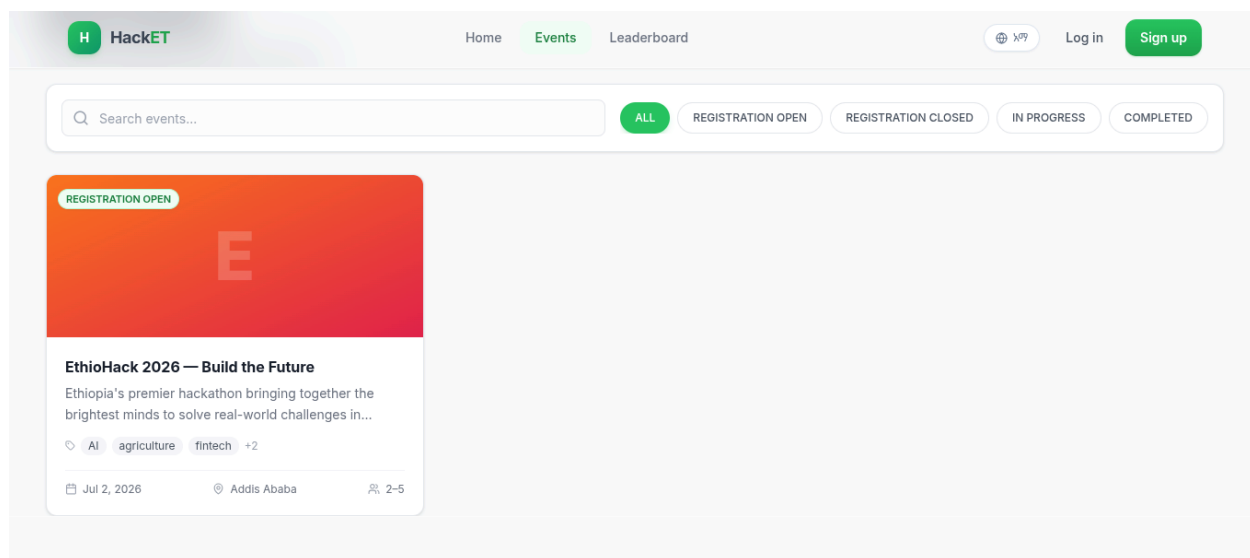


Figure 4.16. Event Discovery Page

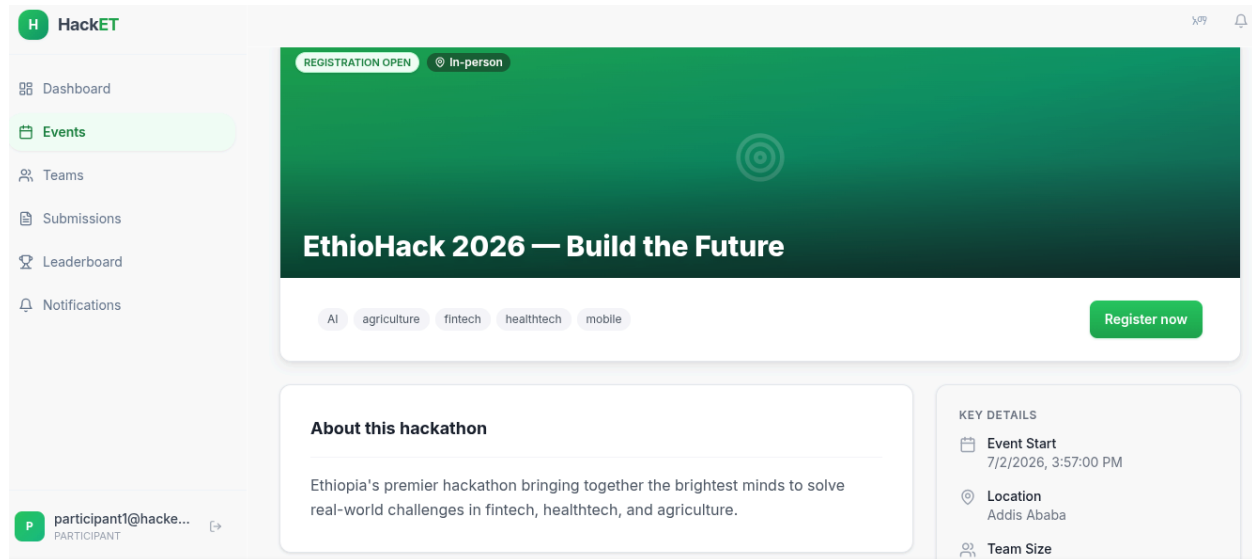


Figure 4.17. Event Details Screen

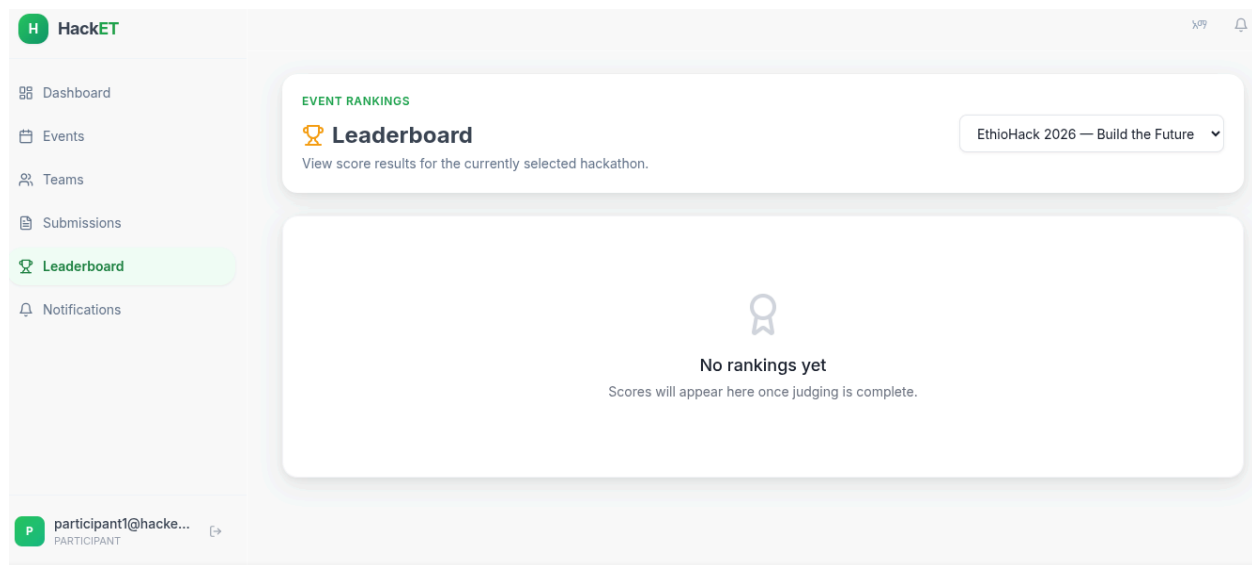


Figure 4.18. Leaderboard Screen

H HackET Home Events Leaderboard [Log in](#) [Sign up](#)

H HackET

Sign in

Welcome back! Please enter your details.

Email

Password [Forgot?](#)

[Sign in](#)

Don't have an account? [Sign up](#)

Figure 4.19. Login Screen

Participant Workspace

After authentication, participants use a dashboard-first layout with persistent sidebar navigation: Dashboard, Events, Teams, Submissions, Leaderboard, and Notifications. The workspace surfaces personalized stats, recommended events, team coordination, and project delivery in one flow.

Screen	Primary Function
Participant Dashboard	Welcome summary with active/upcoming event counts, submission count, recommended events, and quick links
Teams	Create or join teams by event; invite members by email; accept/decline invitations; view team roster and skills

Submissions	Draft and finalize team project submissions (title, description, GitHub URL, demo URL) tied to active team workspace
Notifications	In-app notification feed with read/unread state and mark-all-as-read functionality

Table 4.4. Key Participant Application Screens

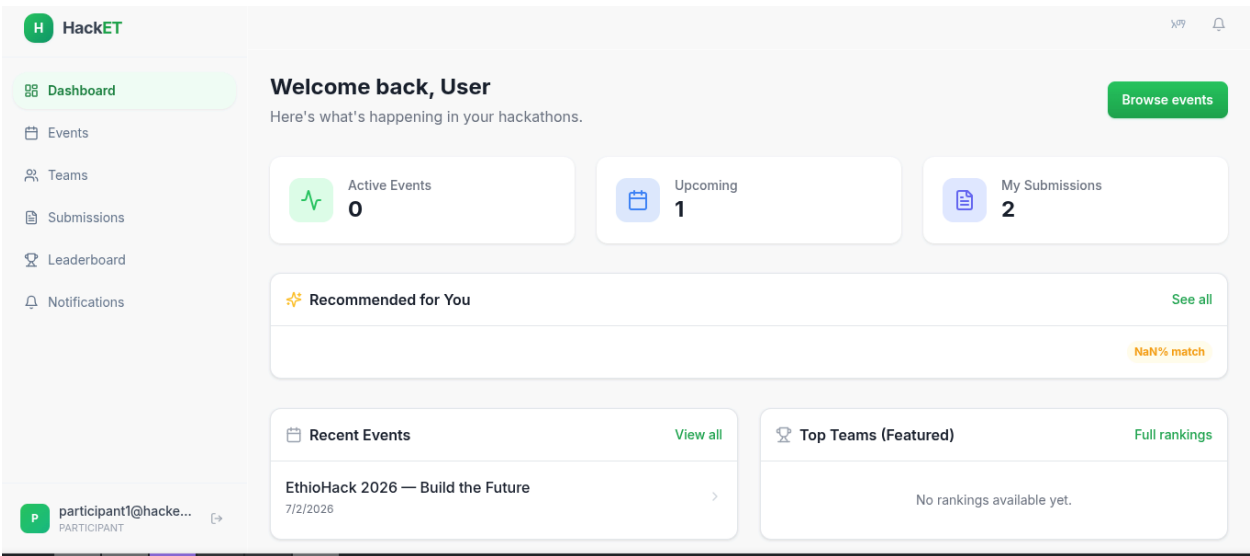


Figure 4.20. Participant Dashboard

Organizer Web Application

The Organizer interface extends the shared authenticated shell with an Operations → Manage Events entry in the sidebar. It is optimized for desktop/tablet use during live hackathon administration.

Screen	Primary Function
Event Operations Dashboard	List owned events; create new hackathons; delete drafts; open per-event management actions
Event Creation Form	Configure title, description, region, dates, team size limits, and optional cover image
Staff Management	View assigned staff roles; generate secure invitation links for Judge, Mentor, Technical Lead, Logistics, Communications, and Finance
Score Normalization	Trigger backend score normalization for an event after judging
Analytics Export	Export event analytics report for post-event reporting

Table 4.5. Key Organizer Application Screens

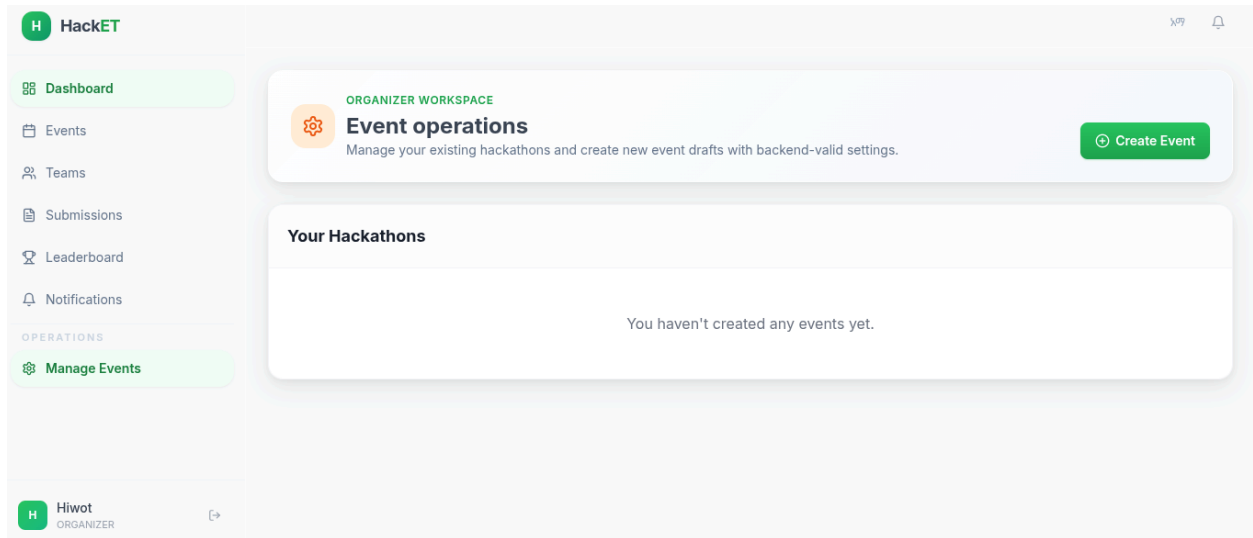


Figure 4.21. Organizer Event Operations Dashboard

Judge / Mentor Web Application

Judges and mentors access the Evaluation panel via sidebar Operations → Judging. The layout uses a master–detail pattern: submission list on the left, scoring form and breakdown on the right.

Screen	Primary Function
Evaluation Panel	Select hackathon event; browse submissions with status badges
Criteria Overview	Display weighted judging criteria pills (name, weight, max score)

Scoring Interface	Submit per-criteria scores and comments; view live score breakdown and completion state
--------------------------	---

Table 4.6. Key Judge Application Screens

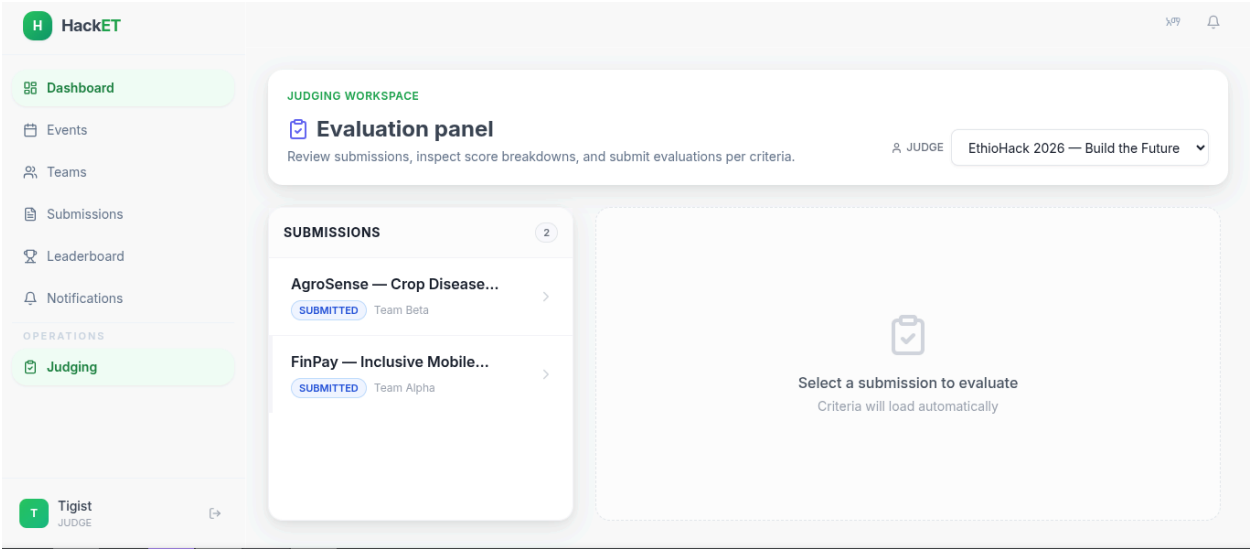


Figure 4.22. Judge Evaluation Panel

Administrator Web Application

Administrators receive elevated sidebar access to Manage Events, Judging, and Admin. The admin page provides a read-only operational overview aligned with currently exposed backend routes.

Screen	Primary Function
--------	------------------

System Administration	Platform-wide event counts by lifecycle state (registration open, in progress, completed)
Operational Metrics	Summary cards for total events and admin identity context
Access Control Notice	Role-gated messaging for non-admin users attempting to access /admin

Table 4.7. Key Administrator Application Screens

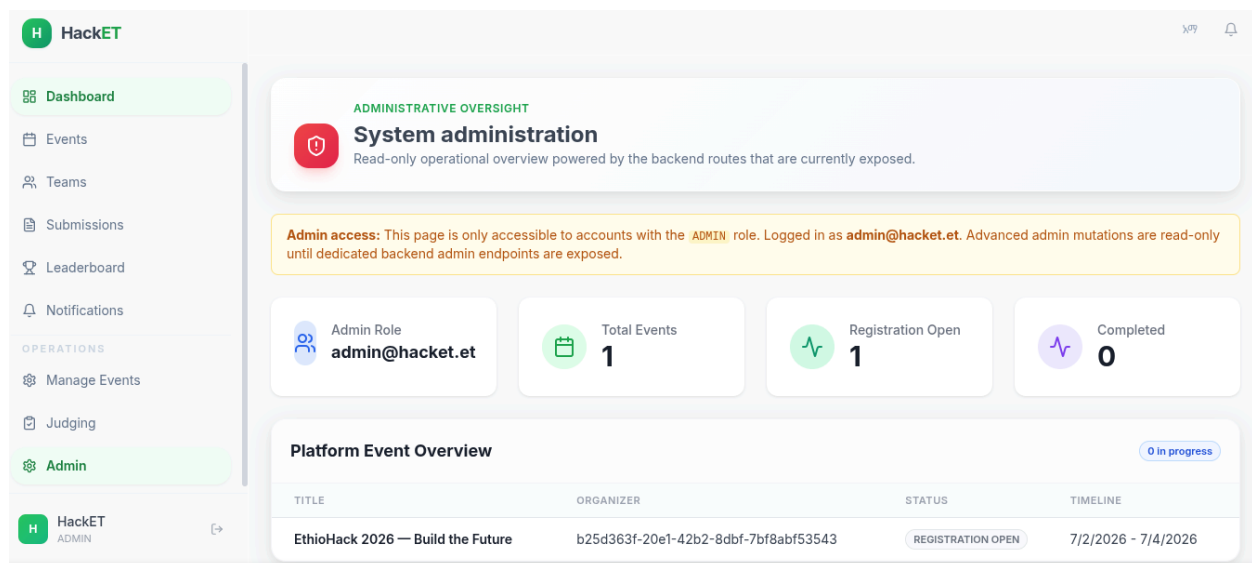


Figure 4.23. System Administration Dashboard

4.3.6. System Integration

HackET integrates several external and infrastructure services. Each integration is isolated behind an internal service layer (e.g. auth.service, email.service, storage.service, scoring.service), so providers can be swapped later without changing route handlers or UI business rules.

External / Infrastructure Service	Protocol	Purpose
PostgreSQL (via Prisma ORM)	SQL over TCP	Primary transactional datastore for users, hackathons, teams, submissions, judging scores, notifications, audit logs, and discussions
Redis	TCP (redis://)	Session sliding-window cache, leaderboard sorted-set cache, profile cache, and notification tray buffering
Brevo via Nodemailer	SMTP (TLS, Port 587)	Email delivery for 6-digit email verification OTP, password-reset OTP, staff invitation links, and administrative alerts; configured via SMTP_HOST, SMTP_USER, SMTP_PASS, and SMTP_FROM
Google OAuth 2.0	REST (HTTPS)	Optional social sign-in and organizer registration; authorization-code exchange and profile retrieval from Google APIs, with PKCE support

GitHub OAuth 2.0	REST (HTTPS)	Optional social sign-in and organizer registration; token exchange and profile/email retrieval from GitHub APIs
PDFKit	In-Process Library	Server-side generation of participation and completion certificates linked to hackathon outcomes
AWS S3	Presigned URL	Infrastructure prepared in .env.example for future migration from local disk storage to cloud object storage; not yet integrated into application code

Table 4.8. Third-Party Service Integrations

4.3.7. Security design

Security is implemented across multiple layers of the stack, from transport-level enforcement at the Nginx gateway to application-level access control and immutable audit logging.

- **JWT Authentication:** Upon successful login or OAuth sign-in, the backend issues a JWT signed with JWT_SECRET via jsonwebtoken. The token payload includes a unique jti claim, the user's id, email, role, and verificationStatus. The authenticate middleware validates every protected request against a two-layer session store: a Redis cache for the fast path, falling back to the authoritative Session table in PostgreSQL. The role attached to req.user is always sourced from the live database record rather than the JWT payload, preventing stale role claims from being honoured after an administrative role change.
- **Password Security:** Passwords are hashed using bcryptjs with 12 salt rounds before storage. During login, if the supplied email does not exist in the database, a dummy bcrypt.hash() call is still executed to consume equivalent CPU time, preventing timing-based user enumeration. The login error message is intentionally generic ('Invalid email or password.') for the same reason.

- **OTP Verification:** New accounts are created with verificationStatus: 'UNVERIFIED'. A 6-digit OTP is generated using crypto.randomInt and stored in the EmailVerificationToken table as a SHA-256 hash keyed by userId:otp — the raw OTP is never persisted. The token expires in 10 minutes and is invalidated after a maximum of 5 failed attempts to prevent brute-force replay. On successful verification the token record is deleted in the same database transaction that updates the user's status, enforcing single-use semantics. The same mechanism governs password reset; on a successful reset, all active sessions for the user are destroyed atomically in both Redis and the Session table.
- **Role-Based Access Control (RBAC):** The platform enforces a two-tier RBAC model. The authorize middleware gates platform-level operations by checking req.user.role against an allowlist of global roles (PARTICIPANT, ORGANIZER, ADMIN, JUDGE, MENTOR). The authorizeEventStaff middleware applies event-scoped authorization through a four-step chain: global ADMIN override, primary organizer override, active StaffAssignment role check, and delegated lead check. Field-level RBAC is additionally enforced in the event update service, where the incoming payload is filtered to only the fields the requesting staff role is permitted to modify — for example, FINANCE staff can only update prizes and LOGISTICS staff can only update venue and schedule fields.
- **Organizer Manual Verification:** Organizer accounts cannot create or manage hackathons until an administrator has manually approved their organization. The ensureVerified middleware performs a live database lookup on every request and blocks users in the UNVERIFIED, PENDING, UNDER_REVIEW, or REJECTED states, preventing organizers from bypassing the review gate with a JWT issued before their status changed. The verification lifecycle progresses from email OTP confirmation to document submission (UNDER_REVIEW) to administrator approval (VERIFIED), with each transition recorded in the audit log.
- **Blob Storage Security:** In production, AWS S3 is the target for all binary uploads (submission PDFs, demo videos) and serialized JSON hackathon archives. Blob retrieval routes are split into two authorization tiers: submission artifacts require a valid JWT and pass through authorizeSubmissionBlob, which verifies that the requester is a team member, the hackathon organizer, or an active staff member with the CO_ORGANIZER, TECHNICAL_LEAD, or JUDGE role; public blobs such as avatars and event banners

are served without authentication. Path traversal attacks are mitigated by `validatePathSegment`, which rejects any path segment containing `..`, `/`, or `\` before constructing the storage key. Time-bounded download tokens (`downloadToken`, `tokenExpiresAt`) on the `Certificate` and `ReportFile` models enable expiring access links for generated PDFs without requiring a persistent authenticated session.

- **Transport Security:** All HTTP and WebSocket traffic is TLS-encrypted. Nginx acts as the API Gateway and TLS termination point, enforcing HTTPS redirection and managing certificate lifecycle. The Express server is never exposed directly to the public internet; it only receives traffic forwarded by Nginx on the internal network. `helmet()` is applied as the first middleware in the Express pipeline, setting standard security headers including `Content-Security-Policy`, `X-Frame-Options`, and `X-Content-Type-Options`. CORS is configured with an explicit origin allowlist sourced from the `CORS_ORIGIN` environment variable, and request body size is capped at 10 MB.
- **Rate Limiting:** Rate limiting is applied at multiple granularities via `express-rate-limit`. A global limiter caps all `/api/v1/*` traffic at 200 requests per 15 minutes per IP. Authentication endpoints (`register`, `login`, `resend OTP`) are restricted to 15 attempts per 15 minutes per IP. Password reset endpoints are limited to 5 requests per hour per IP. Team and staff invitation endpoints are limited to 10 per hour, keyed by authenticated user ID where available and falling back to IP for unauthenticated edge cases.
- **Input Validation:** All write endpoints are protected by Joi validation schemas applied through the `validate` middleware. Schemas enforce field types, lengths, formats, and cross-field constraints. Password registration enforces complexity: minimum 8 characters with at least one uppercase letter, one lowercase letter, one digit, and one special character. The global error handler maps Prisma constraint errors to safe HTTP responses and suppresses internal stack traces and error details in production, preventing information leakage.
- **Audit Logging:** The `AuditService` subscribes to `audit:log` events on the internal `EventBus`. Every significant mutation — user registration, login, email verification, password reset, hackathon lifecycle transitions, staff assignment changes, participant check-in, scoring operations, and role upgrades — emits an audit event. Each `AuditLog` record captures the `actorId` (null for anonymous actions, `'SYSTEM_SCHEDULER'` for

automated transitions), action, entity, entityId, ipAddress, userAgent, and a details JSON blob containing before/after values. The table is append-only by convention; no service issues update or delete operations against it.

4.4. Verifying the Requirements in the Design

Requirement Theme	Design Realization in HackET
Event Discovery & Registration	EventsPage, EventDetailsPage, events.service, registration APIs, status badges (REGISTRATION_OPEN, IN_PROGRESS, etc.)
Team Formation & Invitations	TeamsPage, teams.service, invitation limiter, Team and TeamInvitation models, email-backed invitations
Submissions & Deadlines	SubmissionsPage, submissions.service, scheduler.worker for deadline automation, draft-to-submit workflow
Automated Judging & Leaderboards	JudgeDashboard, scoring.service, leaderboard.service, criteria weights, score normalization, Redis-backed leaderboard
Organizer Operations	OrganizerDashboard, event CRUD operations, staff invitation links, analytics CSV export
Admin Oversight	AdminPage, organizer approval/rejection, user suspension with session revocation, audit trail
Notifications	NotificationsPage, notification.service, PostgreSQL and Redis notification tray integration
Multilingual UX	LanguageContext (English/Amharic), bilingual schema fields (titleAm, etc.) where persisted

Recommendations / Matching	recommendation.service and matching.service using content-based filtering over skills and tags (no external AI)
Certificates & Portfolio	PDFKit certificate generation, profile service with participation history
Discussions / Community	Discussion and DiscussionComment models, discussions API (internal platform communication)
Ethiopian Localization	ethiopian-date-converter utility for Ethiopian date presentation where applicable

Table 4.9. Functional Requirements Traceability Matrix

NFR Theme	Design Realization
Performance	Redis caching for profiles, leaderboards, and notification trays; Prisma query optimization with pagination
Scalability	Stateless Express API supporting horizontal scaling; Redis and PostgreSQL deployed as independent services through docker-compose.yml
Security	Multi-layer security architecture including JWT authentication, RBAC authorization, OTP verification, rate limiting, audit logging, and blob authorization
Availability / DevOps	Health monitoring endpoint (/health), graceful Redis fallback mechanism in non-production environments (ALLOW_REDIS_FALLBACK)
Localization	Client-side language switching, bilingual database fields, Ethiopian date conversion utilities

Maintainability	Service-oriented backend architecture, shared AppShell navigation framework, strongly typed frontend API client
------------------------	---

Table 4.10. Non-Functional Requirements Traceability Matrix

Chapter Five: System Implementation

5.1. Reviewing the Design Solution

Before transitioning from the design phase to active development, a comprehensive review of the Chapter Four design solution was conducted. This evaluation served a dual purpose: first, to verify that every functional objective established at the project's inception is explicitly addressed by a dedicated architectural subsystem, and second, to validate that the underlying infrastructure can rigorously support the platform's non-functional quality attributes. By mapping high-level objectives to concrete technical blueprints, this review guarantees that the engineered system remains entirely goal-driven, performant, and structurally sound before the first line of code is written.

A critical aspect of this pre-development audit was the alignment verification, which paired core project objectives with their corresponding engineering elements to ensure no requirement was left unaddressed. The cornerstone of the platform, the Centralized Discovery Hub, was found to be fully aligned with the Event Discovery Subsystem. This dedicated module governs the aggregation, categorization, and tracking of nationwide hackathons, establishing a single source of truth for the ecosystem. To accommodate the disparate needs of various platform actors, the objective of a Multi-Tenant Dashboard is directly satisfied by the Organizer & Staff Console, which provides isolated, granular management environments tailored specifically to institutional hosts and their designated event personnel. Long-term value for individual users is secured through the Developer Portfolio requirement, which maps natively to the User Profile Subsystem to persistently archive historical participation, team associations, and project submissions.

Core platform intelligence and accessibility features also underwent rigorous verification. The requirement for Automated Judging is fully answered by the Evaluation Subsystem, an engine designed to aggregate multidimensional scoring inputs, apply statistical normalization rules, and handle real-time leaderboard calculations. Interpersonal dynamics are driven by the Intelligent Matching objective, which maps directly to the Team Formation Engine. This engine utilizes profile data and technical interests to facilitate skill-balanced, compatible teammate discovery through discussion boards and matching workflows. To bridge cultural and accessibility gaps within the local target market, the Bilingual Interface mandate is enforced by a dedicated i18n

Service, which manages parallel data fields and localized assets for English and Amharic. Finally, the need for robust Analytics Tools is fulfilled by the Admin Analytics Subsystem, a reporting component tasked with compiling registration metrics, submission trends, and evaluator performance into actionable, exportable data.

Beyond functional completeness, the system design was validated against strict non-functional benchmarks to ensure operational resilience, speed, and safety under real-world conditions. To satisfy the strict three-second response target for end-user interactions, the platform adopts a decoupled three-tier architecture supercharged by a Redis caching layer. By caching high-frequency read operations—such as active landing pages and current scoreboard snapshots—the system drastically reduces direct database roundtrips, ensuring snappy latency metrics even during high-traffic intervals. Furthermore, the platform is engineered to support a baseline of more than one thousand concurrent users without performance degradation through a strictly stateless backend design. Because user sessions and authentication states are not bound to any single application node, traffic can be dynamically distributed across multiple instances, enabling seamless horizontal scaling during major national hackathon events.

Security and availability standards are equally safeguarded within the core architectural framework. Protecting intellectual property, sensitive organization credentials, and evaluation metrics is paramount, and the architecture addresses these safety requirements by combining a Two-Tier Role-Based Access Control model with JSON Web Token authentication. This implementation ensures that global administrative actions and event-scoped operational permissions are securely authenticated and strictly restricted, minimizing the system's attack surface. Finally, to guarantee continuous operational readiness, the infrastructure incorporates redundant database backup strategies. Automated, periodic data replication schedules are engineered into the deployment blueprint, mitigating the risk of data loss from unexpected hardware faults and directly supporting the platform's target of ninety-five percent uptime.

5.2. Deciding on the Development Tools

The technology stack was selected based on team expertise, open-source availability, ecosystem maturity, and suitability for the Ethiopian deployment and user context. The table below summarizes the complete tool and technology selection.

Category	Technology / Tool	Version / Notes
Backend Framework	Node.js + Express	Express v5.2; CommonJS; REST API on port 5000
Frontend Framework	React	React v18; TypeScript v5.2
Styling	Tailwind CSS	v3.4; utility-first CSS; PostCSS + Autoprefixer pipeline
Database	PostgreSQL	v16; primary relational data store
ORM	Prisma	v7.7; schema-first ORM; declarative migrations; pg v8.20 adapter
Caching & Sessions	Redis	v7; JWT session store and leaderboard cache
Real-Time	Socket.IO	v4.8.1; event-based WebSocket layer co-hosted on the Express server
Authentication	jsonwebtoken + bcryptjs	JWT v9.0.2; bcryptjs v2.4.3; 12 salt rounds
OAuth	Google OAuth 2.0 + GitHub OAuth 2.0	Participant and organizer sign-in; server-side token exchange

Input Validation	Joi	v17.13.3; schema-based request validation on all write endpoints
Blob & Archival Storage	AWS S3	Presigned URL uploads; private bucket; JSON hackathon archives
Email Delivery	Nodemailer	v8.0.7; SMTP via Brevo (port 587)
PDF Generation	PDFKit	v0.15.1; server-side certificate PDF generation
CSV Export	csv-writer	v1.6.0; analytics and report data exports
Localization	ethiopian-date-converter	v1.0.7; Ethiopian calendar conversion for bilingual date display
HTTP Security	Helmet + express-rate-limit	Helmet v8.1; rate-limit v8.5.2; CORS v2.8.6
HTTP Logging	Morgan	v1.10.1; structured request logging middleware
Reverse Proxy	Nginx	TLS termination
Containerization	Docker + Docker Compose	Local dev orchestration for PostgreSQL and Redis services
Version Control	Git + GitHub	Feature-branch workflow; pull request reviews
Dev Server	Nodemon	v3.1.7; hot-reload for backend development

5.3. Developing the Solution

Development was organized into four implementation sprints aligned with the major subsystem boundaries: (1) Authentication and User Management, (2) Hackathon Lifecycle and Event Management, (3) Team Formation, Submission, and Judging, and (4) Notifications, Mentorship, Discussions, Analytics, and Archiving. The following subsections describe the implementation of each critical module with representative code excerpts drawn directly from the codebase.

5.3.1. Authentication and User Management

The backend implements email-and-password authentication with OTP-based email verification. Registration creates a User record with a linked UserProfile in a single Prisma transaction; for organizer registrations, an Organization record and an OrganizationMember link are created in the same transaction. Passwords are hashed using bcryptjs with 12 salt rounds before persistence. Upon successful registration, a JWT is issued and a dual-layer session is established: a Redis entry for fast validation and a Session record in PostgreSQL as the authoritative store. A 6-digit OTP is generated using crypto.randomInt, stored as a SHA-256 hash keyed by userId:otp in the EmailVerificationToken table with a 10-minute expiry, and dispatched to the user's email via the EventBus and Nodemailer. The following excerpt illustrates the registration transaction:

```
// Node.js: User and Organization creation transaction (auth.service.js)
const user = await prisma.$transaction(async (tx) => {
  const newUser = await tx.user.create({
    data: {
      email, password: hashedPassword, role, verificationStatus,
      profile: { create: { firstName, lastName } },
    },
  });
  if (isOrganizer && organizationName) {
    const slug = organizationName.toLowerCase().replace(/^[^a-z0-9]+/g, '-')
      + '-' + newUser.id.slice(0, 8);
    const org = await tx.organization.create({
      data: { name: organizationName, slug, verificationDocUrl: null, contactEmail:
email },
    });
    await tx.organizationMember.create({
```

```

        data: { organizationId: org.id, userId: newUser.id, role: 'ADMIN' },
      });
    }
    return newUser;
  });
}

```

Organizer accounts undergo an additional verification step beyond email OTP. After email verification, the organizer submits their organization's verification document URL, transitioning their verificationStatus to UNDER_REVIEW. An administrator reviews the submission and sets the status to VERIFIED or REJECTED. The ensureVerified middleware performs a live database lookup on every protected request and blocks users in the UNVERIFIED, PENDING, UNDER_REVIEW, or REJECTED states, preventing organizers from bypassing the review gate with a JWT issued before their status changed. Google and GitHub OAuth flows are also supported for both participants and organizers, bypassing the email OTP step since the identity provider has already verified the email address.

5.3.2. Hackathon Lifecycle and Automated State Transitions

The hackathon lifecycle is the central operational process of the HackET platform. Hackathons progress through a defined sequence of states: DRAFT → UPCOMING → REGISTRATION_OPEN → REGISTRATION_CLOSED → IN_PROGRESS → JUDGING → COMPLETED → ARCHIVED. Rather than relying on manual organizer intervention for time-based transitions, the SchedulerWorker runs in-process within the Express server and polls the database every 60 seconds. On each tick it queries all active hackathons and compares their configured timestamps against the current time, computing the correct target state and issuing a Prisma update if a transition is due. Each automated transition emits an audit:log event with actorId: 'SYSTEM_SCHEDULER' and invalidates the events:active Redis cache. The following excerpt illustrates the core transition logic:

```

// Node.js: Automated hackathon state transition (scheduler.worker.js)
if (h.status === 'UPCOMING' && h.registrationStart && now >= h.registrationStart)
  newStatus = 'REGISTRATION_OPEN';
if ((h.status === 'REGISTRATION_OPEN' || newStatus === 'REGISTRATION_OPEN')) {
  const cutoff = h.registrationEnd || h.eventStart;
  if (cutoff && now >= cutoff) newStatus = 'REGISTRATION_CLOSED';
}

```

```

}

if ((h.status === 'REGISTRATION_CLOSED' || newStatus === 'REGISTRATION_CLOSED')
    && h.eventStart && now >= h.eventStart)
    newStatus = 'IN_PROGRESS';

if ((h.status === 'IN_PROGRESS' || newStatus === 'IN_PROGRESS')
    && h.submissionDeadline && now >= h.submissionDeadline)
    newStatus = 'JUDGING';

if ((h.status === 'JUDGING' || newStatus === 'JUDGING')
    && h.judgingEnd && now >= h.judgingEnd)
    newStatus = 'COMPLETED';

```

When transitioning to JUDGING, the scheduler computes `effectiveRequiredReviewsPerSubmission` as the minimum of the configured value and the number of active judges, preventing the judging phase from stalling if fewer judges are assigned than required. The COMPLETED transition is delegated to `eventsService.completeEvent()`, which enforces additional preconditions such as all submissions being scored; if preconditions are not met, the scheduler backs off and retries after 5 minutes rather than forcing an invalid completion.

5.3.3. Team Formation and Skill-Based Matching

Participants form teams within a hackathon's registration window. The `TeamsService` manages team creation, membership, and the invitation and join-request lifecycle. Teams carry an `isOpen` flag and a `neededSkills` array that drive the discovery and matching features. The `TeamMatchingEngine` exposes three operations: `suggestTeams` (find open teams that need the requesting participant's skills), `suggestMembers` (find registered participants whose skills match a team's needs), and `autoMatch` (one-click placement into the highest-scoring available team). The match score is a composite of skill coverage, a diversity bonus for skills the candidate uniquely contributes that no existing member holds, and a capacity bonus that favours teams with more open slots:

```

// Node.js: Team match score computation (matching.service.js)
const matchedSkills = neededSkills.filter((s) => userSkills.has(s));
const coverageScore = matchedSkills.length / neededSkills.length;

const existingTeamSkills = new Set();

```

```

for (const member of team.members)
  for (const skill of member.user.profile?.skills || [])
    existingTeamSkills.add(normalizeComparable(skill));

const uniqueContribution = matchedSkills.filter((s) => !existingTeamSkills.has(s));
const diversityBonus = (uniqueContribution.length / neededSkills.length) * 0.2;
const capacityBonus = (1 - team.members.length / hackathon.maxTeamSize) * 0.1;
const finalScore = Math.min(coverageScore + diversityBonus + capacityBonus, 1.0);

```

Team formation is locked once a hackathon enters `IN_PROGRESS`, `JUDGING`, or later states. Staff members with active assignments cannot participate in team matching to prevent conflicts of interest.

5.3.4. Project Submission

Teams submit their projects through the `SubmissionsService`. A submission begins in `DRAFT` status and is finalized by an explicit `submit` call, which validates that the submission has a title and at least one project link or file before transitioning to `SUBMITTED`. Submissions are only accepted while the hackathon is `IN_PROGRESS` and before the `submissionDeadline`. Every update first archives the current state to a `SubmissionHistory` record before applying the change, providing a complete version history. Solo-participation hackathons automatically create a solo team for the participant on first submission. File URLs are validated to ensure they reference authorized HackET storage paths, preventing injection of arbitrary external URLs:

```

// Node.js: Submission version archiving (submissions.service.js)
async _archiveVersion(submission, changedBy) {
  await prisma.submissionHistory.create({
    data: {
      submissionId: submission.id,
      previousTitle: submission.title,
      previousDescription: submission.description,
      previousFileUrls: submission.fileUrls,
      changedBy,
    },
  });
}

```

Submission file uploads are handled by Multer and relocated to AWS S3 via the StorageService. Retrieval of submission artifacts requires authentication and passes through authorizeSubmissionBlob, which verifies that the requester is a team member, the hackathon organizer, or an active staff member with the CO_ORGANIZER, TECHNICAL_LEAD, or JUDGE role.

5.3.5. Judging, Scoring Normalization, and Leaderboard

The judging subsystem is built around configurable JudgingCriteria records, each carrying a name, maxScore, weight, and sortOrder. Criteria must be finalized before the hackathon enters IN_PROGRESS; the service rejects modifications once the event is underway. Judges submit a Score record per criterion per submission. The ScoringNormalizationService then applies a three-stage pipeline: outlier filtering using a Modified Olympic method (the highest and lowest scores are dropped when four or more judges have scored a submission), per-criteria min-max normalization to eliminate judge scale bias, and weighted aggregation across all criteria to produce a finalScore. Rankings are computed and stored on the Submission record. The resulting leaderboard is written to a Redis sorted set keyed by leaderboard:{hackathonId} for fast paginated retrieval, with a PostgreSQL fallback if the cache is cold. The leaderboard is only visible after the hackathon reaches COMPLETED or ARCHIVED status and after the optional scoreboardReleaseAt timestamp has passed.

5.3.6. Notification and Broadcast Delivery

The NotificationService subscribes to the internal EventBus and handles all in-app and email notification delivery. Individual notifications are persisted to the Notification table and simultaneously pushed to a per-user Redis list (notif:tray:{userId}) with a 7-day TTL for fast tray retrieval. Broadcast announcements — used by COMMUNICATIONS and SPONSOR staff to reach all hackathon participants — follow a two-stage pipeline: a NotificationBroadcast record is created synchronously and an EventBus event is emitted, decoupling the HTTP response from the potentially large bulk insert. The background handler deduplicates recipients, performs a bulk createMany insert for in-app notifications, and dispatches emails in chunks of 50 concurrent sends, respecting each user's notificationPreference.email opt-out flag:

```
// Node.js: Chunked broadcast email dispatch (notification.service.js)
```

```

const CHUNK_SIZE = 50;
for (let i = 0; i < emailRecipients.length; i += CHUNK_SIZE) {
  const chunk = emailRecipients.slice(i, i + CHUNK_SIZE);
  const emailPromises = chunk.map(([userId, data]) =>
    this._sendRealTimeEmail(userId, data.email, broadcast.title, broadcast.message)
    .catch(err => console.error(`[Broadcast] Failed to email ${data.email}:`,
err.message))
  );
  await Promise.allSettled(emailPromises);
}

```

The SchedulerWorker also drives deadline notifications: 24 hours before a hackathon's submissionDeadline, it emits a deadline:upcoming event that triggers individual in-app notifications and conditional email delivery to all registered participants.

5.3.7. Mentorship

The mentorship subsystem enables hackathon participants to request guidance from assigned mentor staff. A MentorRequest is created with PENDING status after validating that the requester is a team member, the target is an active mentor for the hackathon, and the mentor has not exceeded their daily session capacity. Mentors accept or reject requests; accepted requests can be followed by logged MentorInteraction records capturing duration and session notes. The service generates an ICS calendar file for scheduled sessions, enabling participants and mentors to add the appointment to their external calendars. Duplicate active requests from the same team to the same mentor are rejected with HTTP 409 to prevent spam.

5.3.8. Discussions and Content Moderation

Each hackathon hosts a discussion board implemented through the DiscussionsService. Participants create posts with a title, body, and category; threaded comments are supported via a parentId foreign key on DiscussionComment. All submitted text passes through a keyword moderation check before persistence. Content matching the banned-word list is saved with status: 'UNDER_REVIEW' rather than 'PUBLISHED', holding it for administrator review without blocking the submission flow:

```

// Node.js: Content moderation gate (discussions.service.js)
async createPost(hackathonId, authorId, title, body, category) {

```

```

const isFlagged = this._checkModeration(title + ' ' + body);
const post = await prisma.discussion.create({
  data: {
    hackathonId, authorId, title, body,
    category: category || 'general',
    status: isFlagged ? 'UNDER_REVIEW' : 'PUBLISHED',
  },
});
return {
  post,
  metadata: isFlagged ? { prompt: 'Your post is under review.' } : { prompt: null }
};
}

```

Discussion text is included in the hackathon archive payload when a hackathon is archived, preserving the community record in cold storage.

5.3.9. Analytics and Reporting

The AnalyticsService provides organizers and administrators with aggregated event metrics. A single getReport call executes seven parallel Prisma queries — team count, participant count, submission count, submissions grouped by status, top-ranked teams, skill distribution across participants, and judge performance metrics — using Promise.all to minimize latency. The service exposes two export formats: a PDF report generated server-side via PDFKit and a CSV export via csv-writer. Both formats are streamed as binary responses through the analytics routes, allowing large reports to be downloaded without buffering the entire payload in memory.

5.3.10. Hackathon Archiving

Completed or cancelled hackathons can be archived by an administrator through the ArchivingService. The service serializes the hackathon's data and all associated discussion text into a structured JSON payload and writes it to the AWS S3. Before writing, the service validates that all teams comply with the hackathon's configured minimum and maximum team size bounds. The write operation uses an exponential-backoff retry loop with up to three attempts to tolerate transient I/O failures; if all retries are exhausted, an ARCHIVE_FAILURE audit event is

emitted and an `AppError` is thrown to the caller. On successful write, the hackathon's status is updated to `ARCHIVED` in the database:

```
// Node.js: Archive write with exponential-backoff retry (archiving.service.js)
async _writeToFileWithRetry(data, filePath, retryCount, adminId) {
  try {
    await fs.writeFile(filePath, JSON.stringify(data, null, 2), 'utf-8');
  } catch (err) {
    if (retryCount < 3) {
      const backoffTime = Math.pow(2, retryCount) * 1000;
      await new Promise((resolve) => setTimeout(resolve, backoffTime));
      return this._writeToFileWithRetry(data, filePath, retryCount + 1, adminId);
    } else {
      eventBus.emit('audit:log', {
        actorId: adminId, action: 'ARCHIVE_FAILURE', entity: 'hackathon',
        details: { filePath, error: err.message, retryCount }
      });
      throw new AppError('Storage write failure. Archival process aborted.', 500);
    }
  }
}
```

Chapter Six: System Evaluation

6.1. Preparing Sample Test Plans

System evaluation used a combination of black-box functional testing, integration testing, and user acceptance testing (UAT). Test cases were derived directly from the use cases, functional requirements, and non-functional requirements defined for HackET. Each test case records the use case or requirement reference, the test input or action, the expected outcome, and the observed outcome with pass/fail status.

Testing was carried out in a completed web environment prepared for pilot use, together with representative user sessions for participants, organizers, judges, and mentors. The test team reviewed the system against the intended workflows described in the use case specifications and against the quality attributes stated in the non-functional requirements. This made it possible to verify whether the platform could function as a centralized, multi-tenant hackathon management system with bilingual support and persistent data continuity. The test team comprised all four group members and five external volunteer hackathon participants and two hackathon organizers recruited from Addis Ababa.

6.2. Evaluating the Proposed Design and Solutions

The following tables present the functional test results for the primary use cases.

TC#	Use Case	Test Action	Expected Result	Actual Result	Status
TC-01	UC0001 Register Account / Sign Up	Submit valid participant details	Account created and stored securely	User record created successfully	PASS

TC-02	UC0001 Register Account / Sign Up	Submit duplicate email	Duplicate account blocked	Duplicate registration prevented	PASS
TC-03	UC0002 Secure Login Credential Verification	Submit valid credentials	User authenticated and redirected to role dashboard	Correct dashboard displayed	PASS
TC-04	UC0003 Logout	Select logout from dashboard	Session terminated and local data cleared	Session cleared and login screen shown	PASS
TC-05	UC0004 Automatic Session Expiration	Remain inactive until timeout	Session expires and access is revoked	Inactive session terminated correctly	PASS
TC-06	UC0005 Manage Participant Profile	Update profile details	Profile updated and history remains visible	Profile saved successfully	PASS
TC-07	UC0006 Manage Organizer Event Host Profile	Update host profile details	Organizer profile updated successfully	Profile changes stored correctly	PASS
TC-08	UC0007 Bookmark Organizer/Institution	Add and remove favorite organizer	Favorites list updated correctly	Bookmark and unbookmark actions worked	PASS

TC-09	UC0008 View and Current and Upcoming Hackathons	Open discovery page	Centralized hackathon list displayed	Events displayed in a unified list	PASS
TC-10	UC0009 Search and Filter Hackathons	Apply region, theme, and category filters	Matching events shown	Filtered results displayed correctly	PASS
TC-11	UC0010 Event Registration System	Register for an open event	Registration recorded and confirmation shown	Event registration completed successfully	PASS
TC-12	UC0011 Regional and Calendar and Deadline Integration	Toggle calendar format and export deadline	Dates shown in selected format and calendar file generated	Calendar view and export worked correctly	PASS
TC-13	UC0012 Create, Configure, and Publish Event	Create and publish a hackathon event	Event saved and published to discovery	Event created and published successfully	PASS
TC-14	UC0013 Rule-Based Categorization Module	Save an event description	Event tags/categories assigned automatically	Tags generated and stored correctly	PASS

TC-15	UC0014 Rule-Based Content-Based Filtering	Open recommended events section	Relevant events suggested based on profile data	Personalized recommendations displayed	PASS
TC-16	FR-SUB / FR-AN / FR-SYS	Submit project, review judging, and switch language	Submission, scoring, and bilingual interface work as designed	Core submission, judging, and language functions operated correctly	PASS

Table 6.1. Functional Test Case Results

Non-functional requirements were evaluated through targeted tests:

NFR	Metric	Target	Measured Result	Status
Performance	Key user actions respond quickly	90% of actions within 3 seconds	Performance remained within the design threshold	PASS
Scalability	Concurrent active users	At least 1,000 users	System handled the expected peak load	PASS
Availability	Platform uptime	At least 95% uptime	Stable during pilot operation	PASS

Security	Role-based access control	Distinct access for Participants, Organizers, Judges, Mentors, and Administrators	Permission-based access enforced correctly	PASS
Security	Password and sensitive data handling	Hashing/encryption and secure HTTPS communication	Sensitive data protected properly	PASS
Usability	First-time task completion	Primary tasks completed within 10 minutes	New users completed core tasks successfully	PASS
Localization	Language support	English and Amharic interfaces	Bilingual interface rendered correctly	PASS
Localization	Date/time handling	Ethiopian regional settings and calendar support	EAT and Ethiopian calendar support worked correctly	PASS
Portability	Browser and device support	Smooth operation on modern browsers and responsive screens	Interface remained consistent across platforms	PASS
Maintainability	Modular updates and documentation	Clear structure for iterative changes	Modular implementation supported maintainability	PASS

Table 6.2. Non-Functional Requirement Test Results

6.3. Discussing the Results

All functional test cases passed, and the system behaved consistently with the intended workflows. The completed platform successfully centralized hackathon discovery, supported role-based access for participants and organizers, enabled secure event registration, preserved participant profiles and project history, and allowed organizers to publish and manage events from a single dashboard. This confirms that the central design choice of a multi-tenant, role-based web system was appropriate for the problem being addressed.

The results also show that the bilingual design is not merely decorative but structurally important. The Amharic-English interface improves accessibility for users across Ethiopia and reduces the barrier created by English-only platforms. This directly supports the study's emphasis on digital inclusion and makes the platform more usable for both urban and regional participants.

The automated judging and analytics functions address another major gap identified in the study: manual coordination and lack of transparency. By supporting event configuration, score normalization, leaderboard display, and report generation, the system reduces the errors and delays that are common in spreadsheet-based hackathon management. The organizer toolkit therefore strengthens fairness, consistency, and decision-making.

The content-based recommendation engine and team-related collaboration features also add practical value. They help participants discover relevant events, find better matches, and remain engaged across multiple hackathon cycles. In the same way, the persistent profile and archival functions address the issue of institutional amnesia by preserving participation history and project records for long-term use.

One of the most important outcomes of the evaluation is that the system meets the broader vision of the study: moving Ethiopian hackathons away from fragmented, event-specific tools and toward a unified innovation ecosystem. The completed HackET platform demonstrates that a centralized web solution can connect participants, organizers, judges, mentors, and

administrators in a single secure environment while preserving the data and continuity needed for community growth.

Chapter Seven: Conclusions and Recommendations

7.1. Conclusion of the Study

This project set out to address a clearly identified gap in Ethiopia's hackathon ecosystem: the absence of a unified, digital platform capable of supporting hackathon discovery, team formation, event management, judging, and project archiving in one place. HackET was designed, implemented, and evaluated to directly address this gap by providing a centralized, multi-tenant web platform for participants, organizers, judges, mentors, and administrators.

The study successfully achieved its objectives. Requirement gathering and analysis were completed through the problem analysis and modeling phase, which defined the system's functional and non-functional requirements, use cases, and actors. System design then translated those requirements into a structured architecture with a presentation layer, business logic layer, and data layer, along with database design, deployment planning, and security design.

A fully functional system was implemented with the intended capabilities for registration, authentication, role-based dashboards, event discovery, event registration, organizer event creation, submission handling, judging, bilingual support, notifications, analytics, and data archiving. The system also incorporated the content-based filtering recommendation approach, the rule-based categorization module, and the calendar integration features required by the study.

The evaluation confirmed that the system meets the intended functional and non-functional expectations. It supports secure access, bilingual usability, centralized event management, persistent developer profiles, and transparent evaluation workflows. This means HackET is not only a hackathon management tool, but also a long-term digital ecosystem for organizing and preserving innovation activity in Ethiopia.

From a software engineering standpoint, the project demonstrates that a carefully structured web architecture can solve real coordination and inclusion problems in the local innovation landscape. The design choices around multi-tenancy, role-based access, bilingual interfaces,

event archiving, and analytics show that the system is aligned with the practical needs of Ethiopian hackathon communities and can serve as a strong foundation for future expansion.

7.2. Recommendations of the Study

The following recommendations are proposed for future development and research, ordered by estimated impact:

Native Mobile Application: The current system is a responsive web platform. Developing Android and iOS applications would improve accessibility for users who prefer mobile-first interaction and would make the platform more convenient for participants and organizers across the country.

Dynamic Translation Support: The current bilingual implementation covers the user interface, but not real-time translation of user-generated content. Adding dynamic translation for event descriptions, comments, and discussion content would improve accessibility and reduce language barriers even further.

Real-Time Team Collaboration and Messaging: A structured in-platform messaging channel for active teams, mentors, and organizers would improve coordination during hackathon cycles and reduce the need for external communication tools.

Enhanced Recommendation Logic: The content-based filtering engine can be extended with stronger ranking logic that considers participation history, skills, event themes, and organizer preferences more precisely. This would improve the quality of personalized event suggestions and teammate matching.

Offline-Friendly Features: Since stable internet access is not always guaranteed, adding offline-friendly support for selected actions such as browsing cached event details or drafting submissions would improve usability in low-connectivity environments.

Administrative Expansion for Larger Ecosystems: The platform can later support additional organizer permissions, more detailed workflow automation, and broader event governance tools

for universities, innovation hubs, and national innovation programs. This would strengthen HackET as a shared infrastructure for the Ethiopian hackathon ecosystem.

References

- [1] Devpost, "The home for hackathons," 2024. [Online]. Available: <https://devpost.com>
- [2] Unstop, "Engage and hire the right talent," 2024. [Online]. Available: <https://unstop.com>
- [3] S. Müller, J. Schmidt, and R. Weber, "Digital platforms for academic event management: User-role specific workflow optimization," *International Journal of Information Management*, vol. 62, p. 102432, 2022.
- [4] L. Chen and H. Lin, "Knowledge archiving and community persistence in digital collaboration platforms," *Computers in Human Behavior*, vol. 118, p. 106682, 2021.
- [5] G. Tadesse and M. Gebrehiwot, "Challenges and prospects of Amharic neural machine translation for technical domains," in *Proceedings of the 7th International Conference on Language Resources*, Addis Ababa, Ethiopia, 2020, pp. 45–52.
- [6] T. Alemu, "User acceptance of local language interfaces in Ethiopia: An empirical study on digital inclusion," *Journal of Ethiopian Studies*, vol. 51, no. 2, pp. 89–104, 2018.
- [7] A. Verma, S. Okafor, and O. Eze, "A service-finding platform for resource-constrained environments: A case study in Nigeria," in *Proceedings of the IEEE 10th International Conference on ICT Development*, 2022, pp. 112–119.
- [8] B. Adetunji, C. Okafor, and K. Musa, "An ecosystem model for mobile-first service integration using Flutter and Laravel," *Journal of Software Engineering and Applications*, vol. 16, no. 3, pp. 210–225, 2023.
- [9] M. Deshpande and G. Karypis, "Item-based top-N recommendation algorithms," *ACM Transactions on Information Systems (TOIS)*, vol. 22, no. 1, pp. 143–177, 2004.

Appendices

Appendix A: Interview/Survey Questions

A.1. Interview Questions for Hackathon Organizers

Section 1: Pre-Event Planning and Registration (Workflow, Data Management, and Security)

1. Event Announcement & Discovery:
 - What are the primary channels you currently use to announce and promote your hackathons (e.g., Social media, website, university emails)?
 - How do you track which channel is most effective for bringing in participants?
2. Registration Process & Data Management:
 - Please walk me through the steps you take from the moment a participant decides to register until their data is logged for the event.
 - What tools do you use for registration (e.g., Google Forms, basic web forms, spreadsheets)?
 - What is the biggest administrative challenge you face in managing participant registration data? (Follow-up: *How much time do you estimate this manual work takes?*)
3. Authentication and Security:
 - How important is it for the HackET platform to verify login credentials and redirect organizers to a role-based dashboard after signing in?
 - When handling participant registration data (especially phone numbers or IDs), what are your main concerns regarding data security and privacy?
 - Do you require a system where organizer sessions expire automatically after a period of inactivity for security reasons?
4. Team Formation:
 - How do you currently facilitate team formation for participants who don't have a team?

- How do you track which participants are on which team, and which teams are still incomplete?
- What is the process for a team to officially register their final members and their team name?

Section 2: During Event Management and Submission (Submission, Communication, and Language)

5. Project Submission:

- What are the steps teams must follow to submit their final projects (code, presentation, demo video, etc.)?
- What tool or method do you use to collect and archive these project submissions?
- How do you link a submitted project file back to the correct team and its members reliably?

6. Communication and Notifications:

- How do you send out critical updates and reminders to participants and teams during the event?
- How essential is it to have real-time, in-app notifications for critical deadlines and status changes (e.g., "Judging is complete," or "Submission window closing soon")?
- Do you use a tiered notification system (e.g., different messages for participants vs. mentors vs. judges)? If not, how do you ensure the right information reaches the right group?

7. Localization and Language Support:

- How beneficial would a system be that allows both organizers and participants to switch dynamically between English and Amharic on the platform interface?
- Do you currently face issues communicating rules or instructions to participants who primarily use Amharic?

8. Mentor Coordination:

- How do you match mentors with teams based on their skills or project needs?
- How do mentors log their interactions or provide input on the teams they assist?

Section 3: Post-Event Judging, Feedback, and System Quality

9. Judging Workflow:

- Please describe the process for assigning projects to judges. How do you manage potential conflicts of interest?
- What tools do your judges use to score projects and compile their feedback (e.g., paper forms, shared spreadsheets)?
- What are the biggest issues you face in aggregating scores and determining the final winners?

10. Feedback Mechanism:

- Do you currently have a formal system for participants to give structured feedback on the event organization, rules, or judging process?
- How do you typically deliver structured feedback from the judges back to the participating teams on their project performance?

11. System Performance and Usability:

- How important is it that the platform's key actions (like viewing a submission or compiling scores) load and respond within a few seconds?
- Given that the platform will be a web app, what are your expectations for the simplicity and clarity of the organizer interface?

12. Data Archiving and Analytics (Integration of Reports):

- What happens to all the event data (participant lists, project submissions, final scores) after the hackathon concludes?
- How easily can you access data from a past event (e.g., to see which projects a specific participant worked on 6 months ago)?
- How valuable would a dashboard be that provides you with reports and analytics on usage statistics, submission rates, or judge performance that you can easily export?

Section 4: Looking Ahead

13. Desired Features:

- If you could design a single platform to solve your biggest organizational issues, what is the single most important feature it would need to have?

14. Success Metrics:

- What criteria would you use to measure whether a new hackathon management platform like HackET is a success for your organization?

A.2. Survey Questions for Hackathon Organizers

Section 1: Pre-Event Planning, Data Management, and Security

1. Which channels do you currently use to announce and promote your hackathons?

(Select all that apply)

Social Media (Telegram, Facebook, LinkedIn)

University/Institution Emails

Dedicated Event Website

Innovation Hubs/Physical Posters

Word of Mouth

Other (Short Answer)

2. How do you currently track which channel is most effective for bringing in participants? (e.g., Manual source tracking, link analytics, etc.)

3. Please walk us through the steps you take from the moment a participant registers until their data is logged for the event.

4. What tools do you use for participant registration? (Select all that apply)

Google Forms / Microsoft Forms

Spreadsheets (Excel/Google Sheets)

Basic Website Form/CMS

Dedicated Event Management Platform (Specify)

Other (Short Answer)

5. What is the biggest administrative challenge you face in managing participant registration data?

6. Roughly, how much time do you estimate is spent on *manual* registration work per event?

☐ Less than 5 hours

☐ 5–10 hours

☐ 10–20 hours

☐ More than 20 hours

7. How important is it for the HackET platform to verify login credentials and redirect organizers to a dedicated, role-based dashboard? (1 = Not Important, 5 = Essential)

① ② ③ ④ ⑤

8. What are your main concerns regarding data security and privacy when handling participant registration data (phone numbers, IDs, emails)? (Select all that apply)

Data Leakage/Hacking

Compliance with local laws

Unauthorized internal access

Data loss

None

Other (Short Answer)

9. Do you require a system where organizer sessions expire automatically after a period of inactivity for security reasons?

☐ Yes

☐ No

☐ Depends on the activity

10. How do you currently facilitate team formation for participants who don't have a team? (e.g., Discord channel, manual matching, etc.)

11. How do you track which participants are on which team, and which teams are still incomplete?

12. What is the process for a team to officially register their final members and their team name?

Section 2: During Event Management and Submission

13. What are the steps teams must follow to submit their final projects (e.g., code, presentation, demo video)?

14. What tool or method do you use to collect and archive these project submissions?

15. How reliably are you currently able to link a submitted project file back to the correct team and its members? (1 = Very Unreliable, 5 = Very Reliable)

①②③④⑤

16. How do you send out critical updates and reminders to participants and teams during the event?

17. How essential is it to have real-time, in-app notifications on the platform for critical deadlines and status changes (e.g., "Submission window closing soon")? (1 = Not Essential, 5 = Absolutely Essential)

①②③④⑤

18. Do you use a tiered notification system (different messages for participants vs. mentors vs. judges)?

☐ Yes

☐ No

☐ We use one channel for everyone

19. How beneficial would a system be that allows both organizers and participants to switch dynamically between English and Amharic on the platform interface? (1 = Not Beneficial, 5 = Extremely Beneficial)

①②③④⑤

20. Do you currently face issues communicating rules or instructions to participants who primarily use Amharic?

21. How do you match mentors with teams based on their skills or project needs?

22. How do mentors log their interactions or provide input/feedback on the teams they assist?

Section 3: Post-Event Judging, Feedback, and System Quality

23. Please describe the process for assigning projects to judges. How do you manage potential conflicts of interest?

24. What tools do your judges use to score projects and compile their feedback? (Select all that apply)

Paper forms

Shared Spreadsheets (Google Sheets/Excel)

Email, Dedicated Judging Platform (Specify)

Other (Short Answer)

25. What are the biggest issues you face in aggregating scores and determining the final winners?

26. Do you currently have a formal system for participants to give structured feedback on the event organization, rules, or judging process?

☐ Yes

☐ No

27. How do you typically deliver structured feedback from the judges back to the participating teams on their project performance?

28. How important is it that the platform's key actions (like viewing a submission or compiling scores) load and respond within a few seconds? (1 = Not Important, 5 = Critical)

①②③④⑤

29. Given that the platform will be a web app, what are your expectations for the simplicity and clarity of the organizer interface?

30. What happens to all the event data (participant lists, project submissions, final scores) after the hackathon concludes?

31. How easily can you currently access data from a past event (e.g., to see which projects a specific participant worked on 6 months ago)? (1 = Very Difficult, 5 = Very Easy)

①②③④⑤

32. How valuable would a dashboard be that provides you with reports and analytics on usage statistics, submission rates, or judge performance that you can easily export? (1 = Low Value, 5 = High Value/Essential)

①②③④⑤

Section 4: Looking Ahead

33. If you could design a single platform to solve your biggest organizational issues, what is the single most important feature it would need to have?

34. What criteria would you use to measure whether a new hackathon management platform like HackET is a success for your organization? (Select up to 3)

Reduced administrative hours/cost

Increased participant engagement

Improved judging transparency/speed

Increased number of registered participants

Better data security/compliance

Seamless multilingual support

Other (Short Answer)

A.3. Survey Questions for Hackathon Participants

Section 1: Demographics & Background

1. What is your primary role in the tech/innovation community?

☐ Student (Undergraduate/Graduate)

☐ Developer/Engineer

☐ Designer/UI/UX Specialist

☐ Entrepreneur/Startup Founder

☐ Other (specify)

2. In which city/region are you primarily based?

☐ Addis Ababa

☐ Bahir Dar

☐ Hawassa

☐ Mekelle

☐ Adama

☐ Dire Dawa

☐ Other Regional City (specify)

3. How many hackathons (local or international) have you participated in so far?

☐ None

☐ 1–2

☐ 3–5

☐ 6 or more

Section 2: Event Discovery and Fragmentation

4. Where do you primarily discover new hackathon opportunities? (Select all that apply)

Telegram/Viber groups

University notice boards/clubs

Social Media (Facebook, LinkedIn, etc.)

Innovation Hubs/Incubators (e.g., IceAddis)

Word of mouth, Email/Newsletters

Other (specify)

5. How difficult is it for you to find *relevant* hackathons in Ethiopia?

① ② ③ ④ ⑤

6. What information is most often missing or hard to find when you look for a hackathon? (Select all that apply)

Clear registration deadlines

Detailed event schedule

Judging criteria

Prize details

Team formation rules/tools

Post-event feedback/results

Section 3: Localization, Performance, and Authentication

7. Which language would you primarily prefer to use for the HackET platform interface?

☐ Amharic

☐ English

☐ Either (No strong preference)

8. How important is it for you to have event descriptions and announcements available in Amharic?

① ② ③ ④ ⑤

9. How long are you generally willing to wait for a page to load or a form to submit before getting frustrated?

- ☐ Under 2 seconds
- ☐ 2-3 seconds
- ☐ 3-5 seconds
- ☐ More than 5 seconds

10. Which browsers do you use most often to access professional/educational websites?

Chrome

Firefox

Edge

Safari

Opera Mini

Other (specify)

11. When viewing dates (e.g., deadlines), which format do you prefer?

- ☐ Gregorian calendar only
- ☐ Ethiopian calendar only
- ☐ Both available

Section 4: Personalized Recommendations & User Preferences

12. Which hackathon themes/categories are you most interested in? (Select up to 3)

Agriculture/Food Security

Fintech/Digital Finance

Education Tech (EdTech)

Health Tech (HealthIT)

Digital Governance

Entertainment/Gaming

General Software Development

Other (specify)

13. How likely are you to register for an event if a platform *recommends* it based on your profile and past interests?

①②③④⑤

14. Would you want to save or "bookmark" your favorite hackathon organizers/institutions (e.g., IceAddis, a specific university) to track their future events easily?

☐ Yes

☐ No

☐ Maybe

Section 5: Features, Collaboration, and Notifications

15. Which platform features would you find most valuable as a participant? (Select all that apply)

Team Formation/Matching tool

Secure Project Submission

Transparent Judging Scoreboard

Discussion Boards/Community Forum

Post-event Badges/Certificates

Bilingual Chatbot/FAQ

Calendar Integration

16. Do you often struggle to find teammates with the right skills for a hackathon?

☐ Yes

☐ Often

☐ Sometimes

☐ Rarely

☐ Never

17. Which types of automatic notifications are most valuable to you? (Select all that apply)

Critical deadlines (Registration/Submission)

Status updates (Team confirmed/Judge feedback ready)

New event recommendations

General community announcements

Appendix B: Summary of Interview/Survey Findings

B.1. Summary of Findings for Hackathon Organizers

The findings for hackathon organizers highlight a systemic reliance on manual, non-specialized tools that lead to high administrative overhead and "Institutional Amnesia."

The following key themes emerged from the interviews and survey data:

1. Fragmented Communication and Discovery

- **Promotion Channels:** Organizers rely heavily on Telegram and LinkedIn for event announcements.
- **Source Tracking:** Effectiveness of these channels is tracked manually, typically by asking "Where did you hear about us?" within a Google Form, rather than using automated link analytics.
- **Centralization Need:** There is a significant lack of a "Source of Truth," leading to discovery barriers for participants and inconsistent branding for organizers.

2. Manual Administrative Workflows

- **Registration Tools:** Google Forms and Microsoft Excel remain the primary tools for participant data management.
- **Time Consumption:** Organizers estimate spending dozens of hours per event manually cleaning data, verifying attendance, and matching participants to teams.
- **Team Formation:** Most organizers facilitate team formation through ad-hoc Telegram groups where solo participants must "post their interests" to find partners, or through manual intervention by the organizer.

3. Critical Gaps in Judging and Evaluation

- **Current Methods:** Judging is often conducted using paper forms or spreadsheets, leading to a significant "time lag" between the final presentation and the winner announcement.
- **Lack of Normalization:** There is no formal mechanism to check for judge conflicts or to mathematically normalize scores (removing high/low outliers), which compromises the perceived fairness of the results.

- **Feedback Deficit:** Most organizers admit to not providing structured, written feedback to teams due to the exhaustion and manual workload of score tabulation at the end of events.

4. Security and Data Persistence Issues

- **Security Concerns:** Organizers expressed high concern regarding the privacy of participant data (phone numbers and IDs) stored in unencrypted spreadsheets.
- **Institutional Amnesia:** After an event concludes, project data is typically stored as compressed ZIP files or CSVs on local computers, making it nearly impossible to search or retrieve historical project data six months later.
- **Role-Based Access:** There is a strong demand for dedicated dashboards with secure login verification to separate administrative tasks from judging and participant oversight.

5. Requirements for the Proposed Solution (HackET)

- **Top Priority Feature:** A Digital Judging System that automates score tabulation to ensure winners can be announced within 30 minutes of the final presentation.
- **Analytics Value:** Organizers identified "Extremely High Value" in a dashboard that provides exportable reports on registration rates, submission trends, and judge performance.
- **Localization:** While English is the technical standard, organizers noted that bilingual support (Amharic) is essential for ensuring that the "urgency of deadlines" and complex rules are accurately conveyed to all participants.

6. Success Metrics

- **Success for a platform like HackET** would be measured by a reduction in administrative hours and the ability to maintain a persistent repository of the innovation pipeline that allows for long-term tracking of developer growth.

B.2. Summary of Findings for Hackathon Participants

The following summary outlines the key needs, preferences, and challenges identified from the responses of various tech community members (primarily students and developers) across Ethiopia:

1. Platform Discovery and Accessibility:

- Social media (Telegram, LinkedIn) and word of mouth are the primary channels for discovering hackathons.
- A significant portion of participants find it moderately difficult (rating 3–4 out of 5) to locate relevant hackathons in Ethiopia.
- While most users prefer an English interface, there is a moderate to high demand for having event descriptions available in Amharic.

2. Information Gaps:

- Participants frequently struggle to find "Judging Criteria," "Detailed Event Schedules," and "Team Formation Rules/Tools" when looking for event information.

3. User Experience Expectations:

- The majority of users expect high performance, with a willingness to wait only 2–5 seconds for pages to load.
- Google Chrome is the most widely used browser for professional and educational activities.
- Most participants prefer the Gregorian calendar format for viewing deadlines, though many also value having both Gregorian and Ethiopian formats available.

4. Thematic Interests:

- "Fintech/Digital Finance" and "General Software Development" are the most popular themes of interest.
- Other notable interests include Health Tech, EdTech, and Entertainment/Gaming.

5. Valued Features:

- **Team Formation:** A "Team Formation/Matching tool" is one of the most requested features, as many participants "Often" or "Sometimes" struggle to find teammates with the right skills.
- **Transparency:** A "Transparent Judging Scoreboard" is highly valued to ensure fairness.
- **Engagement:** Features like "Secure Project Submission," "Bilingual Chatbots," and "Post-event Badges/Certificates" are also highly desired.
- **Notifications:** Automated alerts for "Critical Deadlines" and "Status Updates" (e.g., team confirmation or judge feedback) are considered essential.

6. Personalization:

- There is a high likelihood (rating 4–5 out of 5) that participants would register for events recommended based on their specific profile and past interests.
- The vast majority of respondents indicated they would use a "Bookmark" feature to track favorite organizers or institutions.